

Port-Hamiltonian Neural ODE Networks on Lie Groups for Robot Dynamics Learning and Control

Thai Duong[✉], *Graduate Student Member, IEEE*, Abdullah Altawaitan[✉], *Student Member, IEEE*,
Jason Stanley[✉], *Student Member, IEEE*, and Nikolay Atanasov[✉], *Senior Member, IEEE*

Abstract—Accurate models of robot dynamics are critical for safe and stable control and generalization to novel operational conditions. Hand-designed models, however, may be insufficiently accurate, even after careful parameter tuning. This motivates the use of machine learning techniques to approximate the robot dynamics over a training set of state-control trajectories. The dynamics of many robots are described in terms of their generalized coordinates on a matrix Lie group, e.g., on $SE(3)$ for ground, aerial, and underwater vehicles, and generalized velocity, and satisfy conservation of energy principles. This article proposes a port-Hamiltonian formulation over a Lie group of the structure of a neural ordinary differential equation (ODE) network to approximate the robot dynamics. In contrast to a black-box ODE network, our formulation embeds energy conservation principle and Lie group's constraints in the dynamics model and explicitly accounts for energy-dissipation effect such as friction and drag forces in the dynamics model. We develop energy shaping and damping injection control for the learned, potentially under-actuated Hamiltonian dynamics to enable a unified approach for stabilization and trajectory tracking with various robot platforms.

Index Terms— $SE(3)$ manifold, dynamics learning, Hamiltonian dynamics, neural ordinary differential equation (ODE) networks.

I. INTRODUCTION

MOTION planning and optimal control algorithms depend on the availability of accurate system dynamics models. Models obtained from first principles and calibrated over a small set of parameters via system identification [1] are widely used for unmanned ground vehicles (UGVs), unmanned aerial

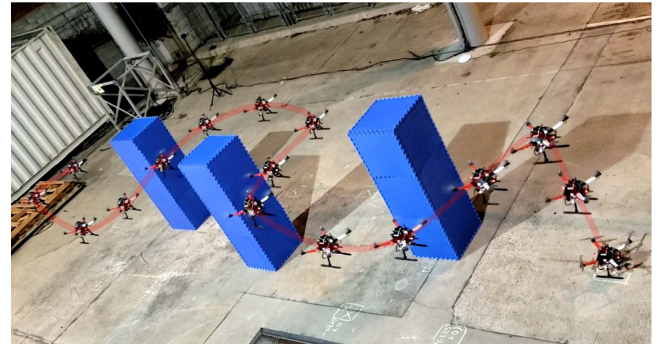


Fig. 1. Quadrotor trajectory tracking using a learned port-Hamiltonian dynamics model.

vehicles (UAVs), and unmanned underwater vehicles (UUVs). Such models may oversimplify or even incorrectly describe the underlying structure of the dynamical system, leading to bias and modeling errors that cannot be corrected by adjusting a few parameters. Data-driven techniques [2], [3], [4], [5], [6] have emerged as a powerful approach to approximate system dynamics with an overparameterized machine learning model, trained over a dataset of system state and control trajectories. Neural networks are expressive function approximation models, capable of identifying and generalizing interaction patterns from the training data. Training neural network models, however, typically requires large amounts of data and computation time, which may be impractical in mobile robotics applications. Recent works [7], [8], [9], [10], [11], [12], [13], [14] have considered a hybrid (gray-box) approach, where prior knowledge of the physics, governing the system dynamics, is used to assist the learning process. The dynamics of physical systems obey kinematic constraints and energy conservation laws. These laws are known to be universally true but a black-box machine learning model might struggle to infer them from the training data, causing poor generalization. Instead, prior knowledge may be encoded into the learning model, e.g., using a prior distribution [3], a graph-network forward kinematic model [15], or a network architecture reflecting the structure of Lagrangian [16] or Hamiltonian [10] mechanical systems. Moreover, many physical robot platforms are composed of rigid-body interconnections and their state evolution respects the structure of a Lie group [17], e.g., the position and orientation kinematics of a rigid body evolve on the $SE(3)$ Lie group [18]. Existing works [19], [20], [21] on Lie group neural ordinary differential equations (ODEs)

Manuscript received 11 January 2024; revised 7 June 2024; accepted 18 June 2024. Date of publication 15 July 2024; date of current version 7 August 2024. This work was supported by NSF CCF-2112665 (TILOS) and NSF FRR CAREER 2045945. This article was recommended for publication by Associate Editor C. Della Santina and Editor J. Bohg upon evaluation of the reviewers' comments. (Corresponding author: Thai Duong.)

Thai Duong, Jason Stanley, and Nikolay Atanasov are with the Department of Electrical and Computer Engineering, University of California San Diego, La Jolla, CA 92093 USA (e-mail: tduong@ucsd.edu; jstanle@ucsd.edu; natanasov@ucsd.edu).

Abdullah Altawaitan is with the Department of Electrical and Computer Engineering, University of California San Diego, La Jolla, CA 92093 USA, and also with Kuwait University 13060, Kuwait (e-mail: aaltawaitan@ucsd.edu).

This video illustrates the approach to solve the problem of learning robot dynamics, whose states evolve on a matrix Lie group, from a training set of state-control trajectories and the problem of tracking a trajectory with the learned dynamics. The video also shows the proposed method working in simulation and real-world demonstrations with quadrotors. Software and videos are available at <https://thaipduong.github.io/LieGroupHamDL>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TRO.2024.3428433>, provided by the authors.

Digital Object Identifier 10.1109/TRO.2024.3428433

李群上的端口-哈密顿神经 ODE 网络用于机器人动力学学习与控制

Thai Duong^{ID}, *Graduate Student Member, IEEE*, Abdullah Altawaitan^{ID}, *Student Member, IEEE*, Jason Stanley, *Student Member, IEEE*, 和 Nikolay Atanasov, *Senior Member, IEEE*

Abstract——精确的机器人动力学模型对于安全、稳定的控制以及在新操作条件下的泛化至关重要。然而，即使经过仔细的参数调优，手工设计的模型可能仍然不够准确。这促使人们使用机器学习技术来在一组状态-控制轨迹的训练集上近似机器人动力学。许多机器人的动力学可以用矩阵李群上的广义坐标来描述，例如地面、空中和水下车辆的 $SE(3)$ ，并用广义速度表示，同时满足能量守恒原理。本文提出了一种基于李群的 port-Hamiltonian 形式，结合神经微分方程（ODE）网络的结构来近似机器人动力学。与黑箱 ODE 网络相比，我们的形式在动力学模型中嵌入了能量守恒原理和李群的约束，并明确考虑了能量耗散效应，如摩擦。

Index Terms— $SE(3)$ 流形、动力学学习、哈密顿动力学、神经微分方程（ODE）网络。

一、引言

运动规划和最优控制算法依赖于精确系统动力学模型的可用性。从第一性原理获得并通过系统辨识在少量参数上校准的模型 [1] 被广泛用于无人地面车辆（UGV）、无人航空

稿件收到日期：2024年1月11日；修订日期：2024年6月7日；接受日期：2024年6月18日。出版日期：2024年7月15日；当前版本日期：2024年8月7日。本工作得到了 NSF CCF-2112665（TILOS）和 NSF FFR CAREER 2045945 的资助。该文章在评审意见的基础上，由副编辑 C. Della Santina 和编辑 J. Bohg 推荐发表。（Corresponding author: Thai Duong.）

Thai Duong、Jason Stanley 和 Nikolay Atanasov 隶属于加利福尼亚大学圣地亚哥分校（University of California San Diego）电气与计算机工程系，地址为 La Jolla, CA 92093, 美国（电子邮件：tduong@ucsd.edu; jstanle@ucsd.edu; natanasov@ucsd.edu）。

阿卜杜拉·阿尔塔韦坦隶属于加利福尼亚大学圣地亚哥分校电气与计算机工程系，美国加利福尼亚州拉霍亚 92093，同时也隶属于科威特大学 13060，科威特（电子邮件：aaltawaitan@ucsd.edu）。

本视频展示了如何解决从状态-控制轨迹训练集学习机器人动力学的问题，其中状态在矩阵李群上演化，以及使用学到的动力学跟踪轨迹的问题。视频还展示了所提出的方法在四旋翼仿真和实际演示中的效果。软件和视频可在 <https://thaipduong.github.io/LieGroupHamDL> 获取。

本文提供了由作者提供的可下载补充材料，网址为 <https://doi.org/10.1109/TRO.2024.3428433>。数字对象唯一标识符 10.1109/TRO.2024.3428433

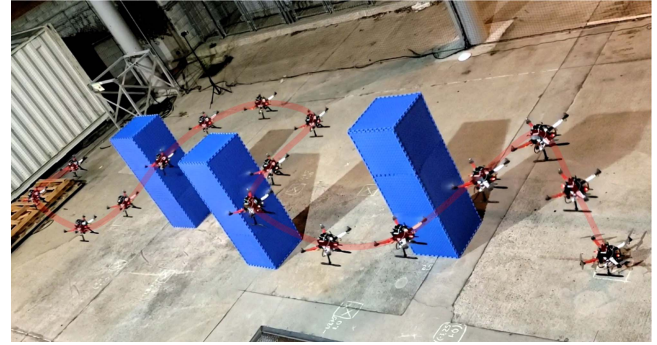


图1. 使用学习到的端口-哈密顿动力学模型进行四旋翼轨迹跟踪。

车辆（无人机）和无人水下航行器（UUV）。这种模型可能过于简化，甚至错误地描述动力系统的基本结构，导致无法通过调整几个参数来纠正的偏差和建模错误。数据驱动技术 [2]、[3]、[4]、[5]、[6] 已成为一种强有力的方法，可通过在系统状态和控制轨迹数据集上训练的过参数化机器学习模型来逼近系统动力学。神经网络是具有表达能力的函数逼近模型，能够从训练数据中识别并泛化交互模式。然而，训练神经网络模型通常需要大量的数据和计算时间，这在移动机器人应用中可能不切实际。近期的研究 [7]、[8]、[9]、[10]、[11]、[12]、[13]、[14] 考虑了一种混合（灰盒）方法，其中利用控制系统动力学的先验物理知识来辅助学习

networks for learning dynamics and normalizing flows focus on preservation of the Lie group structure during backpropagation using an adjoint method, either via a higher dimensional space [19], [20] or local coordinates [21], [22]. The goal of this article is to incorporate both the kinematic structure and the energy conservation properties of physical systems with Lie group states into the structure of a dynamics learning model. We also aim to design a control approach that achieves stabilization or trajectory tracking using the learned model without requiring prior knowledge of its parameters. In other words, the same control design should enable trajectory tracking for learned models of different rigid-body UGVs, UAVs, or UUVs.

Lagrangian and Hamiltonian mechanics [23], [24] provide physical system descriptions that can be integrated into the structure of a neural network [10], [11], [25], [26], [27], [28]. Prior work, however, has only considered vector-valued states, when designing Lagrangian- or Hamiltonian-structured neural networks. This limits the applicability of these techniques as many common robot systems have states on a Lie group. For example, Hamiltonian equations of motion are available for orientation but existing formulations rely predominantly on 3-D vector parametrizations, such as Euler angles [29], [30], which suffer from singularities. Similar to [19] and [20], our work embeds matrix Lie groups in a higher dimensional space $\mathbb{R}^{n \times n}$, allowing us to train the model via the widely used neural ODE network on Euclidean space [31]. However, we focus on embedding the Hamiltonian formulation of robot dynamics in the model and deriving a control policy for trajectory tracking. Concurrently, Wotte et al. [22] offer an exciting approach to learn SE(3) Hamiltonian dynamics from data using a neural ODE on Lie group, trained via local coordinates [21] with guarantees of satisfaction of Lie group constraints. The authors designed an adjoint method on the Lie algebra for neural ODE training, and a potential shaping controller for stabilization of a fully actuated rigid body.

Our preliminary work [32] designed a neural ODE network [31] to capture Hamiltonian dynamics on the SE(3) manifold [33], and derived a trajectory tracking control policy for potentially underactuated systems. Our model is shown to provide accurate long-term trajectory predictions, respecting SE(3) constraints and conserve total energy with high precision. Inspired by [27] and [34], we model kinetic energy and potential energy by separate neural networks, each governed by a set of Hamiltonian equations on SE(3). However, our preliminary work [32] is developed specifically for the SE(3) manifold and does not model dissipation elements that drain energy from the system, such as friction or drag forces in real robot systems. The Hamiltonian formulation of robot dynamics can be generalized to port-Hamiltonian formulation, where the dynamics are governed by energy exchange, i.e., the law of energy conservation, energy dissipation, e.g., from friction, and energy injection, e.g., from control inputs. In this article, we generalize our dynamics learning and control method in [32] using a port-Hamiltonian neural ODE network to embed general matrix Lie group constraints and introduce an energy dissipation term, represented by another neural network, to model friction and air drag in physical systems. We compensate for energy dissipation

in the trajectory tracking control design to provide accurate tracking performance. We verify our approach extensively with simulated robot systems, including a pendulum and a Crazyflie quadrotor, and with several real quadrotor platforms (see Fig. 1). In summary, this article makes the following *contributions*.

- 1) We design a neural ODE model that respects port-Hamiltonian dynamics over a *matrix Lie group* to enable data-driven learning of robot dynamics.
- 2) We develop a unified control policy for port-Hamiltonian dynamics on a Lie group that achieves trajectory tracking if permissible by the system's degree of underactuation.
- 3) We demonstrate our dynamics learning and control approach extensively with simulated robot systems (a pendulum and a quadrotor) and several real quadrotor robots.

II. RELATED WORK

Data-driven techniques [35], [36], [37], [38], [39] have shown impressive results in learning robot dynamics models from state-control trajectories. Neural networks offer especially expressive system models but their training requires large amounts of data, which may be impractical in mobile robot applications. Recently, a hybrid approach [7], [8], [9], [10], [11], [12], [13], [14], [28], [40], [41], [42] has been considered where prior knowledge about a physical system is integrated into the design of a machine learning model. Models designed with structure respecting kinematic constraints [15], symmetry [43], [44], Lagrangian mechanics [7], [8], [9], [12], [13], [16], [45], or Hamiltonian mechanics [10], [11], [14], [25], [26], [27], [28], [46], [47] guarantee that the laws of physics are satisfied by construction, regardless of the training data.

Sanchez-Gonzalez et al. [15] design graph neural networks to represent the kinematic structure of complex dynamical systems and demonstrate forward model learning and online planning via gradient-based trajectory optimization. Ruthotto et al. [43] propose a partial differential equation (PDE) interpretation of convolutional neural networks and derive new parabolic and hyperbolic ResNet architectures guided by PDE theory. Wang et al. [44] design symmetry equivariant neural network models, encoding rotation, scaling, and uniform motion, to learn physical dynamics that are robust to symmetry group distributional shifts.

Lagrangian-based methods [7], [8], [9], [12], [13], [16], [45] design neural network models for physical systems based on the Euler-Lagrange differential equations of motion [23], [24], in terms of generalized coordinates $\mathbf{q}$, their velocity $\dot{\mathbf{q}}$ and a Lagrangian function $\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})$, defined as the difference between the kinetic and potential energies. The energy terms are modeled by neural networks, either separately [7], [16], [45] or together [9].

Hamiltonian-based methods [10], [11], [14], [25], [26], [27], [28] use a Hamiltonian formulation [23], [24] of the system dynamics, instead, in terms of generalized coordinates $\mathbf{q}$, generalized momenta $\mathbf{p}$, and a Hamiltonian function, $\mathcal{H}(\mathbf{q}, \mathbf{p})$, representing the total energy of the system. Greydanus et al. [10] model the Hamiltonian as a neural network and update its parameters by minimizing the discrepancy between its symplectic gradients and the time derivatives of the states $(\mathbf{q}, \mathbf{p})$. This approach,

用于学习动力学和归一化流的网络侧重于在反向传播过程中通过伴随方法保持李群结构, 可以通过更高维空间 [19], [20] 或局部坐标 [21], [22] 实现。本文的目标是将具有李群状态的物理系统的运动学结构和能量守恒特性整合到动力学学习模型的结构中。我们还旨在设计一种控制方法, 利用学习到的模型实现稳定化或轨迹跟踪, 而无需事先知道其参数。换句话说, 相同的控制设计应能够实现不同刚体 UGV、UAV 或 UAV 的学习模型的轨迹跟踪。

拉格朗日力学和哈密顿力学 [23], [24] 提供了可整合到神经网络结构中的物理系统描述 [10], [11], [25], [26], [27], [28]。然而, 以往的工作在设计具有拉格朗日或哈密顿结构的神经网络时, 仅考虑了向量值状态。这限制了这些技术的适用性, 因为许多常见机器人系统的状态存在于李群上。例如, 哈密顿运动方程可用于描述方向, 但现有的表达式主要依赖于三维向量参数化, 如欧拉角 [29], [30], 其存在奇异性。类似于 [19] 和 [20] 的方法, 我们的工作是在更高维空间中嵌入了矩阵李群 $\mathbb{R}^{n \times n}$, 使我们能够在欧几里得空间上广泛使用的神经ODE网络 [31] 训练模型。然而, 我们关注的是在模型中嵌入机器人动力学的哈密顿形式, 并推导轨迹跟踪的控制策略。同时, Wotte 等人

我们的初步工作[32]设计了一个神经ODE网络[31]以捕捉SE(3)流形[33]上的哈密顿动力学, 并为可能是欠驱动的系统推导了轨迹跟踪控制策略。我们的模型被证明能够提供准确的长期轨迹预测, 遵守SE(3)约束, 并以高精度保持总能量。受[27]和[34]的启发, 我们通过独立的神经网络对动能和势能进行建模, 每个网络由SE(3)上的一组哈密顿方程控制。然而, 我们的初步工作[32]是专门针对SE(3)流形开发的, 并未对耗散元件(如实际机器人系统中的摩擦或阻力)导致的能量损失进行建模。机器人的哈密顿动力学表达式可以推广为端口哈密顿表达式, 其动力学由能量交换控制, 即能量守恒定律、能量耗散(例如摩擦)和能量注入(例如控制输入)决定。

在轨迹跟踪控制设计中以提供精确的跟踪性能。我们通过包括单摆和Crazyflie四旋翼在内的仿真机器人系统以及多个实际四旋翼平台对我们的方法进行了广泛验证(见图1)。总之, 本文做出了以下contributions。

- 1) 我们设计了一个神经ODE模型, 该模型遵循 a matrix Lie group 上的端口-哈密顿动力学, 以实现机器人动力学的数据驱动学习。
- 2) 我们为李群上的端口-哈密顿动力学开发了统一的控制策略, 在系统允许的欠驱动度下实现轨迹跟踪。
- 3) 我们通过模拟机器人系统(一个摆和一个四旋翼)以及几台实际四旋翼机器人广泛展示了我们的动力学学习和控制方法。

二、相关工作

数据驱动的技术 [35], [36], [37], [38], [39] 在从状态-控制轨迹学习机器人动力学模型方面显示出令人印象深刻的结果。神经网络提供了特别表达能力强的系统模型, 但其训练需要大量数据, 这在移动机器人应用中可能不可行。最近, 一种混合方法 [7], [8], [9], [10], [11], [12], [13], [14], [28], [40], [41], [42] 得到了考虑, 其中将关于物理系统的先验知识整合到机器学习模型的设计中。采用尊重运动学约束 [15]、对称性 [43], [44]、拉格朗日力学 [7], [8], [9], [12], [13], [16], [45] 或哈密顿力学 [10], [11], [14], [25], [26], [27], [28], [46], [47] 结构设计的模型保证了物理定律在构建时就得到满足, 无论训练数据如何。

Sanchez-Gonzalez 等人 [15] 设计了图神经网络来表示复杂动力系统的运动学结构, 并通过基于梯度的轨迹优化演示了前向模型学习和在线规划。Ruthotto 等人 [43] 提出了卷积神经网络的偏微分方程 (PDE) 解释, 并根据 PDE 理论推导了新的抛物型和双曲型 ResNet 架构。Wang 等人 [44] 设计了对称等变神经网络模型, 编码旋转、缩放和匀速运动, 以学习对称群分布偏移下稳健的物理动力学。

基于拉格朗日的方法 [7], [8], [9], [12], [13], [16], [45] 设计用于物理系统的神经网络模型, 这些模型基于运动的欧拉-拉格朗日微分方程 [23], [24], 在广义坐标 $\mathbf{q}$ 、它们的速度 $\dot{\mathbf{q}}$ 以及一个拉格朗日函数 $\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})$ 的意义下定义, 该函数定义为动能与势能之差。能量项由神经网络建模, 可以分别建模 [7], [16], [45], 也可以共同建模 [9]。

基于哈密顿的方法 [10], [11], [14], [25], [26], [27], [28] 使用系统动力学的哈密顿形式 [23], [24], 而是以广义坐标 $\mathbf{q}$ 、广义动量 $\mathbf{p}$ 和表示系统总能量的哈密顿函数 $\mathcal{H}(\mathbf{q}, \mathbf{p})$ 来表达。Greydanus 等人 [10] 将哈密顿量建模为神经网络, 并通过最小化其辛梯度与状态时间导数 $(\mathbf{q}, \mathbf{p})$ 之间的差异来更新其参数。这种方法,

however, requires that the state time derivatives are available in the training data set. Chen et al. [11] and Zhong et al. [27] relax this assumption by using differentiable leapfrog integrators [48] and differentiable ODE solvers [31], respectively. The need for time derivatives of the states is eliminated by back-propagating a loss function measuring state discrepancy through the ODE solvers via the adjoint method. Our work extends the approach in [27], [34] by formulating the Hamiltonian dynamics over a matrix Lie group, which enforces kinematic constraints in the neural ODE network used to learn the dynamics. Toth et al. [49] and Mason et al. [50] show that, instead of from state trajectories, the Hamiltonian function can be learned from high-dimensional image observations. Finzi et al. [26] show that using Cartesian coordinates with explicit constraints improves both the accuracy and data efficiency for the Lagrangian- and Hamiltonian-based approaches. In a closely related work, Zhong et al. [34] showed that dissipating elements, such as friction or air drag, can be incorporated in a Hamiltonian-based neural ODE network by reformulating the system dynamics in port-Hamiltonian form [51]. The continuous-time equations of motions in Lagrangian or Hamiltonian dynamics can also be discretized using variational integrators [52] to learn discrete-time Lagrangian and Hamiltonian systems [53], [54], [55], [56] and provide long-term prediction for control methods such as model predictive control [57]. This approach eliminates the need to use an ODE solver to roll out the dynamics but its prediction accuracy depends on the discretization time step. Meanwhile, our work encodes not only the Hamiltonian structure but also the Lie group constraints, satisfied by the states of rigid-body robot systems, such as UGVs, UAVs, and UUVs, in a neural ODE network to learn robot dynamics from data.

While most existing dynamics learning approaches focus on Euclidean dynamics, many robot systems have states evolving on a matrix Lie group. Recent works on neural ODE networks [19], [20], [21], [22] on Lie groups are classified into either extrinsic [19], [20] or intrinsic methods [21], [22]. The extrinsic approach [19], [20] embeds a Lie group in a higher dimensional space with Lie group constraints, enabling training with neural ODEs on Euclidean space. Our work belongs to the extrinsic approach by enforcing matrix Lie group constraints on its embedding space $\mathbb{R}^{n \times n}$, but focuses on incorporating the law of energy conservation, via a Hamiltonian formulation, in the dynamics model and providing control design for trajectory tracking. Meanwhile, the intrinsic approach [21], [22] develops adjoint methods for training using local coordinates on the Lie algebra, e.g., via local charts [21], and therefore, guarantees Lie groups constraints by design. Concurrently to our work, Wotte et al. [22] develop a neural ODE network on Lie group to learn Hamiltonian dynamics on the SE(3) manifold by deriving an adjoint method on the Lie algebra, offering an exciting approach for learning structure-preserving dynamics model.

While few dynamics learning articles consider control design based on the learned model, we develop a general trajectory tracking controller for Lie group Hamiltonian dynamics. The Hamiltonian formulation and its port-Hamiltonian generalization [51] are built around the notion of system

energy, and hence, are naturally related to control techniques for stabilization aiming to minimize the total energy. Since the minimum point of the Hamiltonian might not correspond to a desired regulation point, the control design needs to inject additional energy to ensure that the minimum of the total energy is at the desired equilibrium. For fully actuated port-Hamiltonian systems, it is sufficient to shape the potential energy only using an energy-shaping and damping-injection controller [51]. For underactuated systems, both the kinetic and potential energies need to be shaped, e.g., via interconnection and damping assignment passivity-based control (IDA-PBC) [51], [58], [59], [60]. Wang and Goldsmith [61] extend the IDA-PBC controller from stabilization to trajectory tracking. Closely related to our controller design, Souza et al. [62] apply this technique to design a controller for an underactuated quadrotor robot but use Euler angles as the orientation representation. Port-Hamiltonian structure and energy-based control design are also used to learn distributed control policies from state-control trajectories [63], [64], [65].

We connect Hamiltonian-dynamics learning with the idea of IDA-PBC control to allow stabilization of any rigid-body robot without relying on its model parameters a priori. We design a trajectory-tracking controller for underactuated systems, e.g., quadrotor robots, based on the IDA-PBC approach and show how to construct desired pose and momentum trajectories given only desired position and yaw. We demonstrate the tight integration of dynamics learning and control to achieve closed-loop trajectory tracking with underactuated quadrotor robots.

III. PROBLEM STATEMENT

Consider a robot with state $\mathbf{x}$ consisting of generalized coordinates $\mathbf{q}$ evolving on a Lie group G and generalized velocity ξ on the Lie algebra $\mathfrak{g}$ of G . Let $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ characterize the robot dynamics with control input $\mathbf{u} \in \mathbb{R}^m$. For example, the state of rigid-body mobile robot, such as a UGV or UAV, may be modeled by its pose on the SE(3) group, consisting of position and orientation, and its twist on the $\mathfrak{se}(3)$ Lie algebra, consisting of linear and angular velocity. The control input of an Ackermann-drive UGV may include its linear acceleration and steering angle rate, and that of a quadrotor UAV may include the total thrust and moment generated by the propellers; see Section IV-C for more details.

We assume that the function $\mathbf{f}$ specifying the robot dynamics is unknown and aim to approximate it using a dataset $\mathcal{D}$ of state and control trajectories. Specifically, let $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ consist of D state sequences $\mathbf{x}_{0:N}^{(i)}$, obtained by applying a constant control input $\mathbf{u}^{(i)}$ to the system with initial condition $\mathbf{x}_0^{(i)}$ at time $t_0^{(i)}$ and sampling its state $\mathbf{x}^{(i)}(t_n^{(i)}) =: \mathbf{x}_n^{(i)}$ at times $t_0^{(i)} < t_1^{(i)} < \dots < t_N^{(i)}$. Using the dataset $\mathcal{D}$, we aim to find a function $\bar{\mathbf{f}}_\theta$ with parameters θ that approximates the true dynamics $\mathbf{f}$ well. To optimize θ , we roll out the approximate dynamics $\bar{\mathbf{f}}_\theta$ with initial state $\mathbf{x}_0^{(i)}$ and constant control $\mathbf{u}^{(i)}$ and minimize the discrepancy between the computed state sequence $\bar{\mathbf{x}}_{1:N}^{(i)}$ and the true state sequence $\mathbf{x}_{1:N}^{(i)}$ in $\mathcal{D}$.

然而, 这要求训练数据集中提供状态的时间导数。Chen 等人 [11] 和 Zhong 等人 [27] 分别通过使用可微分的跳跃积分器 [48] 和可微分的 ODE 求解器 [31] 放宽了这一假设。通过伴随方法在 ODE 求解器中反向传播衡量状态差异的损失函数, 从而消除了对状态时间导数的需求。我们的工作扩展了 [27], [34] 中的方法, 将哈密顿动力学在矩阵 Lie 群上进行了表述, 这在用于学习动力学的神经 ODE 网络中强制施加了运动学约束。Toth 等人 [49] 和 Mason 等人 [50] 表明, 哈密顿函数可以不仅从状态轨迹中学习, 还可以从高维图像观测中学习。Finzi 等人 [26] 显示, 使用具有明确约束的笛卡尔坐标可以提高拉格朗日和哈密顿方法的准确性和数据效率。在一项密切相关的工作中, Z

能量, 因此, 自然与旨在最小化总能量的稳定控制技术相关。由于哈密顿量的最小点可能不对应于所需的调节点, 控制设计需要注入额外的能量以确保总能量的最小值位于所需的平衡点。对于完全驱动的端口-哈密顿系统, 仅使用能量整形和阻尼注入控制器[51]来整形势能即可。对于欠驱动系统, 需要整形动能和势能, 例如, 通过互联和阻尼分配基于能量控制 (IDA-PBC) [51], [58], [59], [60]。Wang 和 Goldsmith [61] 将 IDA-PBC 控制器从稳定扩展到轨迹跟踪。与我们的控制器设计密切相关的是, Souza 等[62] 将此技术应用于设计欠驱动四旋翼机器人的控制器, 但使用欧拉角作为姿态表示。端口-哈密顿

我们将哈密顿动力学学习与IDA-PBC控制的理念相结合, 以实现任何刚体机器人的稳定控制, 而无需事先依赖其模型参数。我们基于IDA-PBC方法为欠驱动系统(例如四旋翼机器人)设计了轨迹跟踪控制器, 并展示了如何仅根据期望的位置和偏航构建期望的姿态和动量轨迹。我们展示了动力学学习与控制的紧密结合, 以实现欠驱动四旋翼机器人的闭环轨迹跟踪。

三、问题陈述

虽然大多数现有的动力学学习方法专注于欧几里得动力学, 但许多机器人系统的状态是在矩阵李群上演化的。近期关于李群上的神经ODE网络 [19], [20], [21], [22] 的工作被分类为外在方法 [19], [20] 或内在方法 [21], [22]。外在方法 [19], [20] 将李群嵌入到具有李群约束的高维空间中, 从而可以在欧几里得空间上使用神经ODE进行训练。我们的工作属于外在方法, 通过在其嵌入空间 $\mathbb{R}^{n \times n}$ 上强制矩阵李群约束, 但重点是通过哈密顿形式在动力学模型中引入能量守恒定律, 并提供用于轨迹跟踪的控制设计。与此同时, 内在方法 [21], [22] 使用李代数上的局部坐标 (例如通过局部图 [21]) 开发伴随方法进行训练, 因此通过设计保证了李群约束。与我们的工作同时期, Wotte 等人 [

考虑一个具有状态 $\mathbf{x}$ 的机器人, 该状态由泛坐标 $\mathbf{q}$ 组成, 在李群 G 上演化, 并且具有李代数 $\mathfrak{g}$ 上的广义速度 ξ 。设 $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ 表征具有控制输入 $\mathbf{u} \in \mathbb{R}^m$ 的机器人动力学。例如, 刚体移动机器人 (如UGV或UAV) 的状态可以通过其在 $SE(3)$ 群上的位姿建模, 该位姿由位置和方向组成, 以及其在 $\mathfrak{se}(3)$ 李代数上的扭曲, 由线速度和角速度组成。Ackermann 驱动 UGV 的控制输入可能包括其线加速度和转向角速度, 而四旋翼 UAV 的控制输入可能包括螺旋桨产生的总推力和力矩; 更多细节见第 IV-C 节。

我们假设指定机器人动力学的函数 $\mathbf{f}$ 是未知的, 并旨在使用状态和控制轨迹数据集 $\mathcal{D}$ 对其进行逼近。具体来说, 设 $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ 由 D 个状态序列 $\mathbf{x}_{0:N}^{(i)}$ 组成, 这些序列是通过系统施加恒定控制输入 $\mathbf{u}^{(i)}$, 并以初始条件 $\mathbf{x}_0^{(i)}$ 在时间 $t_0^{(i)}$ 采样其状态 $\mathbf{x}^{(i)}(t_n^{(i)}) =: \mathbf{x}_n^{(i)}$ 于时间 $t_0^{(i)} < t_1^{(i)} < \dots < t_N^{(i)}$ 下获得的。使用数据集 $\mathcal{D}$, 我们旨在找到一个具有参数 θ 的函数 $\bar{\mathbf{f}}_\theta$, 以良好地逼近真实动力学 $\mathbf{f}$ 。为了优化 θ , 我们使用初始状态 $\mathbf{x}_0^{(i)}$ 和恒定控制 $\mathbf{u}^{(i)}$ 展开近似动力学 $\bar{\mathbf{f}}_\theta$, 并最小化计算状态序列 $\bar{\mathbf{x}}_{1:N}^{(i)}$ 与真实状态序列 $\mathbf{x}_{1:N}^{(i)}$ 在 $\mathcal{D}$ 中的差异。

尽管很少有动力学学习的文章考虑基于已学习模型的控制设计, 我们开发了一种适用于李群哈密顿动力学的一般轨迹跟踪控制器。哈密顿形式及其端口-哈密顿推广 [51] 是围绕系统的概念建立的

Problem 1: Given a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ and a function $\bar{\mathbf{f}}_\theta$, find the parameters θ that minimize

$$\begin{aligned} \min_{\theta} \quad & \sum_{i=1}^D \sum_{n=1}^N \ell(\mathbf{x}_n^{(i)}, \bar{\mathbf{x}}_n^{(i)}) \\ \text{s.t.} \quad & \dot{\bar{\mathbf{x}}}^{(i)}(t) = \bar{\mathbf{f}}_\theta(\bar{\mathbf{x}}^{(i)}(t), \mathbf{u}^{(i)}), \quad \bar{\mathbf{x}}^{(i)}(t_0) = \mathbf{x}_0^{(i)} \\ & \bar{\mathbf{x}}_n^{(i)} = \bar{\mathbf{x}}^{(i)}(t_n) \quad \forall n = 1, \dots, N \quad \forall i = 1, \dots, D \end{aligned} \quad (1)$$

where ℓ is a distance metric on the state space.

Furthermore, we aim to design a feedback controller capable of tracking a desired state trajectory $\mathbf{x}^*(t), t \geq t_0$, for the learned model $\bar{\mathbf{f}}_\theta$ of the robot dynamics.

Problem 2: Given an initial condition $\mathbf{x}_0$ at time t_0 , desired state trajectory $\mathbf{x}^*(t), t \geq t_0$, and learned dynamics $\bar{\mathbf{f}}_\theta$, design a feedback control law $\mathbf{u} = \pi(\mathbf{x}, \theta, \mathbf{x}^*(t))$ such that $\limsup_{t \rightarrow \infty} \ell(\mathbf{x}(t), \mathbf{x}^*(t))$ is bounded.

We consider robot kinematics on the Lie group G such that when there is no control input, $\mathbf{u} = \mathbf{0}$, the dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u})$ respect the law of energy conservation. We embed these constraints in the structure of the parametric function $\bar{\mathbf{f}}_\theta$. We review matrix Lie groups, with the $SE(3)$ manifold as an example, and Hamiltonian dynamics equations next.

IV. PRELIMINARIES

A. Matrix Lie Groups

In this section, we cover the background needed to define Hamiltonian dynamics on a Lie group. Refer to [17], [33], and [66] for a more detailed overview of matrix Lie groups.

Definition 1 (Dot Product): The dot product $\langle \cdot, \cdot \rangle$ between two matrices ξ and ψ in $\mathbb{R}^{n \times m}$ can be chosen as

$$\langle \xi, \psi \rangle = \text{tr}(\xi^\top \psi). \quad (2)$$

The aforementioned dot product definition is used to define the dual maps in Definitions 6 and 9, and loss functions in Sections V-C and V-D.

Definition 2 (General linear group [17]): The general linear group $GL(n, \mathbb{R})$ is the group of $n \times n$ invertible real matrices.

Definition 3 (Matrix Lie group [17]): A matrix Lie group G is a subgroup of $GL(n, \mathbb{R})$ with identity element $\mathbf{e}$ such that if any sequence of matrices $\{A_n\}_{n=0}^\infty$ in G converges to a matrix A , then either A is in G or A is not invertible. A matrix Lie group is also a smooth embedded submanifold on $\mathbb{R}^{n \times n}$.

Definition 4 (Tangent space and bundle): The tangent space $T_{\mathbf{q}}G$ is the set of all tangent vectors ξ to the manifold G at $\mathbf{q}$. The tangent bundle TG is the set of all the pairs $(\mathbf{q}, \xi)$ with $\mathbf{q} \in G$ and $\xi \in T_{\mathbf{q}}G$.

Definition 5 (Lie algebra and Lie bracket): A Lie algebra is a vector space $\mathfrak{g}$, equipped with a Lie bracket operator $[\cdot, \cdot] : \mathfrak{g} \times \mathfrak{g} \rightarrow \mathfrak{g}$ that satisfies

$$\text{bilinearity: } [a\xi_1 + b\xi_2, \xi_3] = a[\xi_1, \xi_3] + b[\xi_2, \xi_3]$$

$$[\xi_3, a\xi_1 + b\xi_2] = a[\xi_3, \xi_1] + b[\xi_3, \xi_2]$$

$$\text{skew-symmetry: } [\xi_1, \xi_2] = -[\xi_2, \xi_1]$$

$$\text{Jacobi identity:}$$

$$[\xi_1, [\xi_2, \xi_3]] + [\xi_2, [\xi_3, \xi_1]] + [\xi_3, [\xi_1, \xi_2]] = 0.$$

Every matrix Lie group G is associated with a Lie algebra $\mathfrak{g}$, which is the tangent space at the identity element $T_{\mathbf{e}}G$. An element $\mathbf{q} \in G$ is linked with an element $\xi \in \mathfrak{g}$ via the exponential map $\exp_G : \mathfrak{g} \rightarrow G$ and the logarithm map $\log_G : G \rightarrow \mathfrak{g}$ [17]. Since tangent spaces of G , and in particular, the Lie algebra $\mathfrak{g}$, are isomorphic to Euclidean space, we can define a linear mapping $(\cdot)^\wedge : \mathbb{R}^n \rightarrow \mathfrak{g}$ and its inverse $(\cdot)^\vee : \mathfrak{g} \rightarrow \mathbb{R}^n$, where n is the dimension of G . Thus, we can map between G and $\mathbb{R}^n$ using the compositions

$$\exp_G^\wedge = \exp_G \circ \wedge, \quad \log_G^\vee = \vee \circ \log_G. \quad (3)$$

In this article, we consider a matrix Lie group element $\mathbf{q} \in G$ embedded in $\mathbb{R}^{n \times n}$, instead of its n -dimensional representations, e.g., $\log_G^\vee \mathbf{q}$, due to potential issues of discontinuity [67] and singularity [68], [69]. For example, Zhou et al. [67] show that higher dimensional representations of rotations, e.g., $(n^2 - n)$ dimensions for $SO(n)$, are more suitable for learning using neural networks because they ensure continuity.

Definition 6 (Left translation and invariant vectors): The left translation $L_{\mathbf{q}} : G \rightarrow G$ with $\mathbf{q} \in G$ is defined as

$$L_{\mathbf{q}}(\mathbf{h}) = \mathbf{q}\mathbf{h}. \quad (4)$$

The left-invariant vector $T_{\mathbf{e}}L_{\mathbf{q}}(\xi)$ is defined as the derivative of the left translation $L_{\mathbf{q}}$ at $\mathbf{h} = \mathbf{e}$ in the direction of ξ . This vector describes the kinematics of the Lie group, which relates the velocity $\xi \in \mathfrak{g}$ to the change $\dot{\mathbf{q}} \in T_{\mathbf{q}}G$ of coordinates $\mathbf{q}$

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}}(\xi) = \mathbf{q}\xi. \quad (5)$$

Given a pairing $\langle \cdot, \cdot \rangle$ on $\mathfrak{g}^* \times \mathfrak{g}$ (e.g., Definition 1), the dual map $T_{\mathbf{e}}^*L_{\mathbf{q}}$ of $T_{\mathbf{e}}L_{\mathbf{q}}$ satisfies

$$\langle T_{\mathbf{e}}^*L_{\mathbf{q}}(\eta), \xi \rangle = \langle \eta, T_{\mathbf{e}}L_{\mathbf{q}}(\xi) \rangle \quad (6)$$

for any $\eta \in \mathfrak{g}^*$ and $\xi \in \mathfrak{g}$.

Definition 7 (Adjoint Operator): For $\mathbf{q} \in G$, the adjoint $\text{Ad}_{\mathbf{q}} : \mathfrak{g} \rightarrow \mathfrak{g}$ is defined as

$$\text{Ad}_{\mathbf{q}}(\psi) = \mathbf{q}\psi\mathbf{q}^{-1}. \quad (7)$$

The algebra adjoint $\text{ad}_{\xi} : \mathfrak{g} \rightarrow \mathfrak{g}$ is the directional derivative of $\text{Ad}_{\mathbf{q}}$ at $\mathbf{q} = \mathbf{e}$ in the direction of $\xi \in \mathfrak{g}$

$$\text{ad}_{\xi}(\psi) = \left. \frac{d}{dt} \text{Ad}_{\exp_G(t\xi)}(\psi) \right|_{t=0} = [\xi, \psi]. \quad (8)$$

Definition 8 (Cotangent space and bundle): The dual space of the tangent space $T_{\mathbf{q}}G$, i.e., the space of all linear functionals from $T_{\mathbf{q}}G$ to $\mathbb{R}$, is called the cotangent space $T_{\mathbf{q}}^*G$. At the identity $\mathbf{e}$, the cotangent space of the Lie algebra $\mathfrak{g} = T_{\mathbf{e}}G$ is denoted $\mathfrak{g}^*$. The cotangent bundle T^*G is the set of all the pairs $(\mathbf{q}, \mathbf{p})$ with $\mathbf{q} \in G$ and $\mathbf{p} \in T_{\mathbf{q}}^*G$.

Definition 9 (Coadjoint operator): For $\mathbf{q} \in G$, the coadjoint $\text{Ad}_{\mathbf{q}}^* : \mathfrak{g}^* \rightarrow \mathfrak{g}^*$ is defined as $\langle \text{Ad}_{\mathbf{q}}^*(\varphi), \psi \rangle = \langle \varphi, \text{Ad}_{\mathbf{q}}(\psi) \rangle$, where $\varphi \in \mathfrak{g}^*$, $\psi \in \mathfrak{g}$ and $\langle \cdot, \cdot \rangle$ is a pairing on $\mathfrak{g}^* \times \mathfrak{g}$. The algebra coadjoint $\text{ad}_{\xi}^* : \mathfrak{g}^* \rightarrow \mathfrak{g}^*$ is the dual map of ad_{ξ} , satisfying $\langle \text{ad}_{\xi}^*(\varphi), \psi \rangle = \langle \varphi, \text{ad}_{\xi}(\psi) \rangle$.

Problem 1: 给定一个数据集 $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ 和一个函数 $\bar{\mathbf{f}}_\theta$, 找到使 θ 最小化的参数

$$\begin{aligned} \min_{\theta} \quad & \sum_{i=1}^D \sum_{n=1}^N \ell(\mathbf{x}_n^{(i)}, \bar{\mathbf{x}}_n^{(i)}) \\ \text{s.t.} \quad & \dot{\bar{\mathbf{x}}}^{(i)}(t) = \bar{\mathbf{f}}_\theta(\bar{\mathbf{x}}^{(i)}(t), \mathbf{u}^{(i)}), \quad \bar{\mathbf{x}}^{(i)}(t_0) = \mathbf{x}_0^{(i)} \\ & \bar{\mathbf{x}}_n^{(i)} = \bar{\mathbf{x}}^{(i)}(t_n) \quad \forall n = 1, \dots, N \quad \forall i = 1, \dots, D \end{aligned} \quad (1)$$

其中 ℓ 是状态空间上的距离度量。

此外, 我们旨在设计一个反馈控制器, 能够跟踪机器人动力学的学习模型 $\bar{\mathbf{f}}_\theta$ 的期望状态轨迹 $\mathbf{x}^*(t)$ 、 $t \geq t_0$ 。

Problem 2: 给定在时间 t_0 的初始条件 $\mathbf{x}_0$ 、期望状态轨迹 $\mathbf{x}^*(t)$ 、 $t \geq t_0$ 和学习到的动力学 $\bar{\mathbf{f}}_\theta$, 设计一个反馈控制律 $\mathbf{u} = \pi(\mathbf{x}, \theta, \mathbf{x}^*(t))$, 使得 $\limsup_{t \rightarrow \infty} \ell(\mathbf{x}(t), \mathbf{x}^*(t))$ 有界。

我们考虑李群 G 上的机器人运动学, 这样当没有控制输入 $\mathbf{u} = \mathbf{0}$ 时, 动力学 $\mathbf{f}(\mathbf{x}, \mathbf{u})$ 遵循能量守恒定律。我们将这些约束嵌入到参数化函数 $\bar{\mathbf{f}}_\theta$ 的结构中。接下来, 我们回顾矩阵李群, 以 $SE(3)$ 流形为例, 并介绍哈密顿动力学方程。

四、初步工作

A. Matrix Lie Groups

在本节中, 我们介绍在李群上定义哈密顿动力学所需的背景。有关矩阵李群的更详细概述, 请参阅[17]、[33]和[66]。

Definition 1 (Dot Product): $\mathbf{R}^{n \times m}$ 中两个矩阵 ξ 和 ψ 的点积 $\langle \cdot, \cdot \rangle$ 可以选择为

$$\langle \xi, \psi \rangle = \text{tr}(\xi^\top \psi). \quad (2)$$

前述的点积定义用于在定义6和定义9中定义对偶映射, 以及在第V-C和V-D节中定义损失函数。

Definition 2 (General linear group [17]): 一般线性群 $GL(n, \mathbb{R})$ 是 $n \times n$ 可逆实矩阵的群。

Definition 3 (Matrix Lie group [17]): 矩阵李群 G 是 $GL(n, \mathbb{R})$ 的一个子群, 其单位元为 $\mathbf{e}$, 并且如果 G 中的任意矩阵序列 $\{A_n\}_{n=0}^\infty$ 收敛到矩阵 A , 那么要么 A 在 G 中, 要么 A 不可逆。矩阵李群也是 $\mathbb{R}^{n \times n}$ 上的一个光滑嵌入子流形。

Definition 4 (Tangent space and bundle): 切空间 $T_q G$ 是在 q 处的流形 G 的所有切向量 ξ 的集合。切丛 TG 是所有对 (q, ξ) 的集合, 其中 $q \in G$ 和 $\xi \in T_q G$ 。

Definition 5 (Lie algebra and Lie bracket): 李代数是一个向量空间 $\mathfrak{g}$, 配备一个李括号算子 $[\cdot, \cdot]: \mathfrak{g} \times \mathfrak{g} \rightarrow \mathfrak{g}$, 它满足

$$\text{bilinearity: } [a\xi_1 + b\xi_2, \xi_3] = a[\xi_1, \xi_3] + b[\xi_2, \xi_3]$$

$$[\xi_3, a\xi_1 + b\xi_2] = a[\xi_3, \xi_1] + b[\xi_3, \xi_2]$$

$$\text{斜对称: } [\xi_1, \xi_2] = -[\xi_2, \xi_1]$$

雅可比恒等式:

$$[\xi_1, [\xi_2, \xi_3]] + [\xi_2, [\xi_3, \xi_1]] + [\xi_3, [\xi_1, \xi_2]] = 0.$$

每个矩阵李群 G 都与一个李代数 $\mathfrak{g}$ 相关联, 李代数是单位元 $T_e G$ 处的切空间。一个元素 $q \in G$ 通过指数映射 $\exp_G: \mathfrak{g} \rightarrow G$ 和对数映射 $\log_G: G \rightarrow \mathfrak{g}$ 与一个元素 $\xi \in \mathfrak{g}$ 关联[17]。由于 G 的切空间, 特别是李代数 $\mathfrak{g}$, 同构于欧几里得空间, 我们可以定义一个线性映射 $(\cdot)^\wedge: \mathbb{R}^n \rightarrow \mathfrak{g}$ 及其逆映射 $(\cdot)^\vee: \mathfrak{g} \rightarrow \mathbb{R}^n$, 其中 n 是 G 的维数。因此, 我们可以使用组合映射在 G 和 $\mathbb{R}^n$ 之间进行映射。

$$\exp_G^\wedge = \exp_G \circ \wedge, \quad \log_G^\vee = \vee \circ \log_G. \quad (3)$$

在本文中, 我们考虑嵌入在 $\mathbb{R}^{n \times n}$ 中的矩阵李群元素 $q \in G$, 而不是它的 n 维表示, 例如 $\log_G^\vee q$, 这是由于可能存在的不连续性[67]和奇异性[68], [69]问题。例如, Z hou 等人[67]表明, 旋转的高维表示, 例如 $SO(n)$ 的 $(n^2 - n)$ 维, 对于使用神经网络进行学习更为适合, 因为它们能够确保连续性。

Definition 6 (Left translation and invariant vectors): 左翻译 $L_q: G \rightarrow G$ 与 $q \in G$ 被定义为

$$L_q(\mathbf{h}) = q\mathbf{h}. \quad (4)$$

左不变向量 $T_e L_q(\xi)$ 定义为在 $\mathbf{h} = \mathbf{e}$ 处沿 ξ 方向的左平移 L_q 的导数。该向量描述了李群的运动学, 它将速度 $\xi \in \mathfrak{g}$ 与坐标 q 的变化 $\dot{q} \in T_q G$ 联系起来。

$$\dot{q} = T_e L_q(\xi) = q\xi. \quad (5)$$

给定 $\mathfrak{g}^* \times \mathfrak{g}$ (上的配对 $\langle \cdot, \cdot \rangle$, 例如, 定义1), $T_e L_q$ 的对偶映射 $T_e^* L_q$ 满足

$$\langle T_e^* L_q(\eta), \xi \rangle = \langle \eta, T_e L_q(\xi) \rangle \quad (6)$$

针对任何 $\eta \in \mathfrak{g}^*$ 和 $\xi \in \mathfrak{g}$ 。

Definition 7 (Adjoint Operator): 对于 $q \in G$, 伴随 $\text{Ad}_q: \mathfrak{g} \rightarrow \mathfrak{g}$ 被定义为

$$\text{Ad}_q(\psi) = q\psi q^{-1}. \quad (7)$$

代数伴随 $\text{ad}_\xi: \mathfrak{g} \rightarrow \mathfrak{g}$ 是 Ad_q 在 $q = \mathbf{e}$ 点沿 $\xi \in \mathfrak{g}$ 方向的方向导数

$$\text{ad}_\xi(\psi) = \left. \frac{d}{dt} \text{Ad}_{\exp_G(t\xi)}(\psi) \right|_{t=0} = [\xi, \psi]. \quad (8)$$

Definition 8 (Cotangent space and bundle): 切空间 $T_q G$ 的对偶空间, 即从 $T_q G$ 到 $\mathbb{R}$ 的所有线性泛函的空间, 称为余切空间 $T_q^* G$ 。在单位元 $\mathbf{e}$ 处, 李代数 $\mathfrak{g} = T_e G$ 的余切空间记作 $\mathfrak{g}^*$ 。余切丛 $T^* G$ 是所有对 (q, p) 的集合, 其中 $q \in G$ 和 $p \in T_q^* G$ 。

Definition 9 (Coadjoint operator): 对于 $q \in G$, 余伴随 $\text{Ad}_q^*: \mathfrak{g}^* \rightarrow \mathfrak{g}^*$ 定义为 $\langle \text{Ad}_q^*(\varphi), \psi \rangle = \langle \varphi, \text{Ad}_q(\psi) \rangle$, 其中 $\varphi \in \mathfrak{g}^*$ 、 $\psi \in \mathfrak{g}$ 和 $\langle \cdot, \cdot \rangle$ 是 $\mathfrak{g}^* \times \mathfrak{g}$ 上的一种配对。代数余伴随 $\text{ad}_\xi^*: \mathfrak{g}^* \rightarrow \mathfrak{g}^*$ 是 ad_ξ 的对偶映射, 满足 $\langle \text{ad}_\xi^*(\varphi), \psi \rangle = \langle \varphi, \text{ad}_\xi(\psi) \rangle$ 。

The next section describes the $SE(3)$ Lie group to illustrate the aforementioned definitions. The $SE(3)$ Lie group is used to represent the position and orientation of a rigid body.

B. Example: $SE(3)$ Manifold

Consider a fixed world inertial frame of reference and a rigid body with a body-fixed frame attached to its center of mass. The pose of the body-fixed frame in the world frame is determined by the position $\mathbf{p} = [x, y, z]^\top \in \mathbb{R}^3$ of the center of mass and the orientation of the body-fixed frame's coordinate axes

$$\mathbf{R} = [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3]^\top \in SO(3) \quad (9)$$

where $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbb{R}^3$ are the rows of the rotation matrix $\mathbf{R}$. A rotation matrix is an element of the special orthogonal group

$$SO(3) = \{\mathbf{R} \in \mathbb{R}^{3 \times 3} : \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1\}. \quad (10)$$

The rigid-body position and orientation can be combined in a single pose matrix $\mathbf{q} \in SE(3)$, which is an element of the special Euclidean group

$$SE(3) = \left\{ \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} : \mathbf{R} \in SO(3), \mathbf{p} \in \mathbb{R}^3 \right\}. \quad (11)$$

The kinematic equations of motion of the rigid body are determined by the linear velocity $\mathbf{v} \in \mathbb{R}^3$ and angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$ of the body-fixed frame with respect to the world frame, expressed in body-frame coordinates. The generalized velocity $\boldsymbol{\zeta} = [\mathbf{v}^\top, \boldsymbol{\omega}^\top]^\top \in \mathbb{R}^6$ determines the rate of change of the rigid-body pose according to the $SE(3)$ kinematics

$$\dot{\mathbf{q}} = \mathbf{q}\boldsymbol{\xi} = \mathbf{q}\hat{\boldsymbol{\zeta}} =: \mathbf{q} \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (12)$$

where we overload $\hat{\cdot}$ to denote the mapping from a vector $\boldsymbol{\zeta} \in \mathbb{R}^6$ to a 4×4 twist matrix $\boldsymbol{\xi} = \hat{\boldsymbol{\zeta}}$ in the Lie algebra $\mathfrak{se}(3)$ of $SE(3)$ and from a vector $\boldsymbol{\omega} \in \mathbb{R}^3$ to a 3×3 skew-symmetric matrix $\hat{\boldsymbol{\omega}}$ in the Lie algebra $\mathfrak{so}(3)$ of $SO(3)$

$$\hat{\boldsymbol{\omega}} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}. \quad (13)$$

Refer to [68] for an excellent introduction to the use of $SE(3)$ in robot state estimation problems.

C. Hamiltonian Dynamics on Matrix Lie Groups

In this section, we describe Hamilton's equations of motion on a matrix Lie group [33], [66]. Our neural network architecture design in Section V is based on Lie group Hamiltonian dynamics, which encode both kinematic constraints and energy conservation.

Consider a system with generalized coordinates $\mathbf{q}$ in a matrix Lie group G and generalized velocity $\dot{\mathbf{q}} \in T_{\mathbf{q}}G$. The dynamics of the state $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}}) \in TG$ satisfy

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}}(\boldsymbol{\xi}) = \mathbf{q}\boldsymbol{\xi} \quad (14)$$

where $\boldsymbol{\xi}$ is an element in the Lie algebra $\mathfrak{g}$. The Lagrangian on a Lie group $\mathcal{L} : G \times \mathfrak{g} \rightarrow \mathbb{R}$ is defined as the difference between the kinetic energy $\mathcal{T} : G \times \mathfrak{g} \rightarrow \mathbb{R}$ and the potential energy $\mathcal{V} : G \rightarrow \mathbb{R}$

$$\mathcal{L}(\mathbf{q}, \boldsymbol{\xi}) = \mathcal{T}(\mathbf{q}, \boldsymbol{\xi}) - \mathcal{V}(\mathbf{q}). \quad (15)$$

The Hamiltonian is obtained using a Legendre transformation

$$\mathcal{H}(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \boldsymbol{\xi} - \mathcal{L}(\mathbf{q}, \boldsymbol{\xi}) \quad (16)$$

where the momentum $\mathbf{p}$ is defined as

$$\mathbf{p} = \frac{\partial \mathcal{L}(\mathbf{q}, \boldsymbol{\xi})}{\partial \boldsymbol{\xi}}. \quad (17)$$

The state $(\mathbf{q}, \mathbf{p}) \in T^*G$ evolves according to the Hamiltonian dynamics [33] as

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}} \left(\frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \right) \quad (18a)$$

$$\dot{\mathbf{p}} = \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) - T_{\mathbf{e}}^*L_{\mathbf{q}} \left(\frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}} \right) + \mathbf{B}(\mathbf{q})\mathbf{u}. \quad (18b)$$

Obtaining explicit expressions for $\text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p})$ and $T_{\mathbf{e}}^*L_{\mathbf{q}}(\boldsymbol{\eta})$ with $\mathbf{q} \in G$, $\boldsymbol{\xi} \in \mathfrak{g}$, and $\mathbf{p}, \boldsymbol{\eta} \in \mathfrak{g}^*$ depends on the structure of the matrix Lie algebra $\mathfrak{g}$ and the pairing $\langle \cdot, \cdot \rangle$ on $\mathfrak{g}^* \times \mathfrak{g}$. Appendix B provides details and an example for the $SE(3)$ manifold.

By comparing (14) and (18), we have

$$\boldsymbol{\xi} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}}. \quad (19)$$

Let $\boldsymbol{\eta} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}}$. When there is no control input, i.e., $\mathbf{u} = 0$, the conservation of energy is guaranteed as

$$\begin{aligned} \frac{d\mathcal{H}(\mathbf{q}, \mathbf{p})}{dt} &= \langle \boldsymbol{\eta}, \dot{\mathbf{q}} \rangle + \langle \boldsymbol{\xi}, \dot{\mathbf{p}} \rangle \\ &= \langle \boldsymbol{\eta}, T_{\mathbf{e}}L_{\mathbf{q}}(\boldsymbol{\xi}) \rangle - \langle \boldsymbol{\xi}, T_{\mathbf{e}}^*L_{\mathbf{q}}(\boldsymbol{\eta}) \rangle + \langle \boldsymbol{\xi}, \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) \rangle \\ &= 0 \end{aligned} \quad (20)$$

because of (6), and by definition,

$$\langle \boldsymbol{\xi}, \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) \rangle = \langle \text{ad}_{\boldsymbol{\xi}}(\boldsymbol{\xi}), \mathbf{p} \rangle = \langle [\boldsymbol{\xi}, \boldsymbol{\xi}], \mathbf{p} \rangle = 0. \quad (21)$$

D. Reformulation as Port-Hamiltonian Dynamics

The notion of energy in dynamical systems is shared across multiple domains, including mechanical, electrical, and thermal. A port-Hamiltonian generalization [51] of Hamiltonian mechanics is used to model systems with energy-storing elements (e.g., kinetic and potential energy), energy-dissipating elements (e.g., friction or resistance), and external energy sources (e.g., control inputs), connected via energy ports. An input-state-output port-Hamiltonian system has the form

$$\begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} = (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u} \quad (22)$$

where $\mathcal{J}(\mathbf{q}, \mathbf{p})$ is a skew-symmetric interconnection matrix, representing the energy-storing elements, $\mathcal{R}(\mathbf{q}, \mathbf{p}) \succeq 0$ is a positive semidefinite dissipation matrix, representing the energy-dissipating elements, and $\mathcal{G}(\mathbf{q}, \mathbf{p})$ is an input matrix such that

下一节描述了 SE(3) 李群，以说明前面提到的定义。SE(3) 李群用于表示刚体的位置和方向。

B. Example: SE(3) Manifold

考虑一个固定的世界惯性参考系以及一个刚体，其质心处附有一个与刚体一起固定的坐标系。该刚体固定坐标系在世界坐标系中的位姿由质心的位置 $\mathbf{p} = [x, y, z]^T \in \mathbb{R}^3$ 和刚体固定坐标系坐标轴的方向确定。

$$\mathbf{R} = [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3]^T \in \text{SO}(3) \quad (9)$$

其中 $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbb{R}^3$ 是旋转矩阵 $\mathbf{R}$ 的行。旋转矩阵是特殊正交群的一个元素

$$\text{SO}(3) = \{\mathbf{R} \in \mathbb{R}^{3 \times 3} : \mathbf{R}^T \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1\}. \quad (10)$$

刚体的位置和方向可以组合在一个单一的位姿矩阵

$\mathbf{q} \in \text{SE}(3)$ 中，它是特殊欧几里得群的一个元素

$$\text{SE}(3) = \left\{ \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^T & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} : \mathbf{R} \in \text{SO}(3), \mathbf{p} \in \mathbb{R}^3 \right\}. \quad (11)$$

刚体的运动学方程由相对于世界坐标系的刚体固定坐标系的线速度 $\mathbf{v} \in \mathbb{R}^3$ 和角速度 $\boldsymbol{\omega} \in \mathbb{R}^3$ 决定，并以刚体坐标系表示。广义速度 $\boldsymbol{\zeta} = [\mathbf{v}^T, \boldsymbol{\omega}^T]^T \in \mathbb{R}^6$ 根据 SE(3) 运动学确定刚体位姿的变化率

$$\dot{\mathbf{q}} = \mathbf{q}\boldsymbol{\zeta} = \mathbf{q}\hat{\boldsymbol{\zeta}} =: \mathbf{q} \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^T & 0 \end{bmatrix} \quad (12)$$

我们重载 $\hat{\cdot}$ 来表示从向量 $\boldsymbol{\zeta} \in \mathbb{R}^6$ 到 SE(3) 的李代数 $\mathfrak{se}(3)$ 中的 4×4 扭转矩阵 $\hat{\boldsymbol{\zeta}} = \hat{\boldsymbol{\zeta}}$ 的映射，以及从向量 $\boldsymbol{\omega} \in \mathbb{R}^3$ 到 SO(3) 的李代数 $\mathfrak{so}(3)$ 中的 3×3 反对称矩阵 $\hat{\boldsymbol{\omega}}$ 的映射

$$\hat{\boldsymbol{\omega}} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}. \quad (13)$$

请参考 [68] 了解 SE(3) 在机器人状态估计问题中的出色应用介绍。

C. Hamiltonian Dynamics on Matrix Lie Groups

在本节中，我们描述了矩阵李群上的哈密顿运动方程 [33], [66]。我们在第 V 节中的神经网络架构设计基于李群哈密顿动力学，该动力学同时编码了运动学约束和能量守恒。

考虑一个在矩阵李群 G 中具有广义坐标 $\mathbf{q}$ 和广义速度 $\dot{\mathbf{q}} \in T_{\mathbf{q}}G$ 的系统。状态 $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}}) \in TG$ 的动力学满足

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}}(\boldsymbol{\xi}) = \mathbf{q}\boldsymbol{\xi} \quad (14)$$

其中 $\boldsymbol{\xi}$ 是李代数 $\mathfrak{g}$ 中的一个元素。在李群 $\mathcal{L}$ 上的拉格朗日量 $G \times \mathfrak{g} \rightarrow \mathbb{R}$ 定义为动能 $\mathcal{T}: G \times \mathfrak{g} \rightarrow \mathbb{R}$ 与势能 $\mathcal{V}: G \rightarrow \mathbb{R}$ 之差

$$\mathcal{L}(\mathbf{q}, \boldsymbol{\xi}) = \mathcal{T}(\mathbf{q}, \boldsymbol{\xi}) - \mathcal{V}(\mathbf{q}). \quad (15)$$

哈密顿量是通过勒让德变换得到的

$$\mathcal{H}(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \boldsymbol{\xi} - \mathcal{L}(\mathbf{q}, \boldsymbol{\xi}) \quad (16)$$

其中动量 $\mathbf{p}$ 定义为

$$\mathbf{p} = \frac{\partial \mathcal{L}(\mathbf{q}, \boldsymbol{\xi})}{\partial \boldsymbol{\xi}}. \quad (17)$$

状态 $(\mathbf{q}, \mathbf{p}) \in T^*G$ 按照哈密顿动力学 [33] 演化为

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}}\left(\frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}}\right) \quad (18a)$$

$$\dot{\mathbf{p}} = \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) - T_{\mathbf{e}}^*L_{\mathbf{q}}\left(\frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}}\right) + \mathbf{B}(\mathbf{q})\mathbf{u}. \quad (18b)$$

获得 $\text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p})$ 和 $T_{\mathbf{e}}^*L_{\mathbf{q}}(\boldsymbol{\eta})$ 关于 $\mathbf{q} \in G$ 、 $\boldsymbol{\xi} \in \mathfrak{g}$ 和 $\mathbf{p}, \boldsymbol{\eta} \in \mathfrak{g}^*$ 的显式表达式取决于矩阵李代数 $\mathfrak{g}$ 的结构以及 $\mathfrak{g}^* \times \mathfrak{g}$ 上的配对 $\langle \cdot, \cdot \rangle$ 。附录 B 提供了 SE(3) 流形的详细信息和示例。

通过比较 (14) 和 (18)，我们得到

$$\boldsymbol{\xi} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}}. \quad (19)$$

设 $\boldsymbol{\eta} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}}$ 。当没有控制输入时，即 $\mathbf{u} = 0$ ，能量守恒得到保证，如下

$$\begin{aligned} \frac{d\mathcal{H}(\mathbf{q}, \mathbf{p})}{dt} &= \langle \boldsymbol{\eta}, \dot{\mathbf{q}} \rangle + \langle \boldsymbol{\xi}, \dot{\mathbf{p}} \rangle \\ &= \langle \boldsymbol{\eta}, T_{\mathbf{e}}L_{\mathbf{q}}(\boldsymbol{\xi}) \rangle - \langle \boldsymbol{\xi}, T_{\mathbf{e}}^*L_{\mathbf{q}}(\boldsymbol{\eta}) \rangle + \langle \boldsymbol{\xi}, \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) \rangle \\ &= 0 \end{aligned} \quad (20)$$

由于 (6)，根据定义，

$$\langle \boldsymbol{\xi}, \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) \rangle = \langle \text{ad}_{\boldsymbol{\xi}}(\boldsymbol{\xi}), \mathbf{p} \rangle = \langle [\boldsymbol{\xi}, \boldsymbol{\xi}], \mathbf{p} \rangle = 0. \quad (21)$$

D. Reformulation as Port-Hamiltonian Dynamics

动力系统能量概念在多个领域中是共通的，包括机械、电气和热能。哈密顿力学的端口-哈密顿推广 [51] 用于建模具有储能元件（如动能和势能）、耗能元件（如摩擦或电阻）以及通过能量端口连接的外部能量源（如控制输入）的系统。输入-状态-输出端口-哈密顿系统的形式为

$$\begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} = (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u} \quad (22)$$

其中 $\mathcal{J}(\mathbf{q}, \mathbf{p})$ 是一个斜对称互连矩阵，表示储能元件， $\mathcal{R}(\mathbf{q}, \mathbf{p}) \succeq 0$ 是一个半正定耗散矩阵，表示能量消耗元件，且 $\mathcal{G}(\mathbf{q}, \mathbf{p})$ 是一个输入矩阵，使得

$\mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u}$ represents the external energy sources. In the absence of energy-dissipating elements and external energy sources, the skew-symmetry of $\mathcal{J}(\mathbf{q}, \mathbf{p})$ guarantees the energy conservation of the system.

To model energy dissipating elements such as friction or drag forces, we reformulate the Hamiltonian dynamics on a matrix Lie group (18) in port-Hamiltonian form (22). Such elements are often modeled [70] as a linear transformation $\mathbf{D}(\mathbf{q}, \mathbf{p}) \succeq 0$ of the velocity $\dot{\boldsymbol{\xi}}$ and only affect the generalized momenta $\mathbf{p}$, i.e.,

$$\mathcal{R}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{D}(\mathbf{q}, \mathbf{p}) \end{bmatrix}. \quad (23)$$

The Hamiltonian dynamics (18) is a special case of (22), where the dissipation matrix is $\mathbf{D}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, the input matrix is $\mathcal{G}(\mathbf{q}, \mathbf{p}) = [\mathbf{0}^\top \quad \mathbf{B}(\mathbf{q})^\top]^\top$ and the interconnection matrix $\mathcal{J}(\mathbf{q}, \mathbf{p})$ can be obtained by rearranging (18) with $\mathbf{u} = 0$ and is guaranteed to be skew-symmetric due to the energy conservation (20). In an implementation, the coordinates $\mathbf{q}$ and momentum $\mathbf{p}$ may be represented as vectors in $\mathbb{R}^{n^2}$, leading to an interconnection matrix $\mathcal{J}(\mathbf{q}, \mathbf{p}) \in \mathbb{R}^{2n^2 \times 2n^2}$.

E. Example: Hamiltonian Dynamics on the SE(3) Manifold

In this section, we consider the generalized coordinate $\mathbf{q}$ of a mobile robot consisting of its position $\mathbf{p} \in \mathbb{R}^3$ and orientation $\mathbf{R} \in SO(3)$. Let $\mathbf{q} = (\mathbf{p}, \mathbf{R})$ be the generalized coordinates and $\boldsymbol{\zeta} = (\mathbf{v}, \boldsymbol{\omega}) \in \mathbb{R}^6$ be the generalized velocity, consisting of the body-frame linear velocity $\mathbf{v} \in \mathbb{R}^3$ and the body-frame angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$. The coordinate $\mathbf{q}$ evolves on the Lie group $SE(3)$ while the generalized velocity satisfies $\dot{\mathbf{q}} = \mathbf{q}\boldsymbol{\xi}$, where $\boldsymbol{\xi} = \hat{\boldsymbol{\zeta}}$ is a twist matrix in $\mathfrak{se}(3)$, as shown in (12). The isomorphism between $\mathfrak{se}(3)$ and $\mathbb{R}^6$ via (12) simplifies the Hamiltonian (18) and its port-Hamiltonian formulation (22) as follows. The Lagrangian function on $SE(3)$ can be expressed in terms of $\mathbf{q}$ and $\boldsymbol{\zeta}$, instead of $\mathbf{q}$ and $\dot{\mathbf{q}}$ as follows:

$$\mathcal{L}(\mathbf{q}, \boldsymbol{\zeta}) = \frac{1}{2} \boldsymbol{\zeta}^\top \mathbf{M}(\mathbf{q}) \boldsymbol{\zeta} - \mathcal{V}(\mathbf{q}). \quad (24)$$

The generalized mass matrix has a block-diagonal form when the body frame is attached to the center of mass [33]

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} \mathbf{M}_v(\mathbf{q}) & \mathbf{0} \\ \mathbf{0} & \mathbf{M}_\omega(\mathbf{q}) \end{bmatrix} \in \mathbb{S}_{>0}^{6 \times 6} \quad (25)$$

where $\mathbf{M}_v(\mathbf{q}), \mathbf{M}_\omega(\mathbf{q}) \in \mathbb{S}_{>0}^{3 \times 3}$. The generalized momenta are defined, as before, via the partial derivative of the Lagrangian with respect to the twist

$$\mathbf{p} = \begin{bmatrix} \mathbf{p}_v \\ \mathbf{p}_\omega \end{bmatrix} = \frac{\partial \mathcal{L}(\mathbf{q}, \boldsymbol{\zeta})}{\partial \boldsymbol{\zeta}} = \mathbf{M}(\mathbf{q}) \boldsymbol{\zeta} \in \mathbb{R}^6. \quad (26)$$

The Hamiltonian function of the system becomes

$$\mathcal{H}(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \boldsymbol{\zeta} - \mathcal{L}(\mathbf{q}, \boldsymbol{\zeta}) = \frac{1}{2} \mathbf{p}^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p} + \mathcal{V}(\mathbf{q}). \quad (27)$$

By vectorizing the generalized coordinates $\mathbf{q} = [\mathbf{p}^\top \quad \mathbf{r}_1^\top \quad \mathbf{r}_2^\top \quad \mathbf{r}_3^\top]^\top$, the Hamiltonian dynamics on $SE(3)$ can be described

in port-Hamiltonian form (22) [33], [71], [72] with interconnection matrix

$$\mathcal{J}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{0} & \mathbf{q}^\times \\ -\mathbf{q}^{\times\top} & \mathbf{p}^\times \end{bmatrix}, \quad \mathbf{p}^\times = \begin{bmatrix} \mathbf{0} & \hat{\mathbf{p}}_v \\ \hat{\mathbf{p}}_v & \hat{\mathbf{p}}_\omega \end{bmatrix} \quad (28)$$

and input matrix $\mathcal{G}(\mathbf{q}, \mathbf{p}) = [\mathbf{0}^\top \quad \mathbf{B}(\mathbf{q})^\top]^\top$, where $\mathbf{q}^\times = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_1^\top & \hat{\mathbf{r}}_2^\top & \hat{\mathbf{r}}_3^\top \end{bmatrix}^\top$. Note that the kinematics constraints $\dot{\mathbf{q}} = \mathbf{q}\boldsymbol{\xi}$ of the coordinates $\mathbf{q}$ on $SE(3)$, which guarantee that the coordinates $\mathbf{q}$ remain in $SE(3)$, are not affected by vectorization. The port-Hamiltonian formulation allows us to model dissipation elements in the dynamics by the dissipation matrix

$$\mathbf{D}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{D}_v(\mathbf{q}, \mathbf{p}) & \mathbf{0} \\ \mathbf{0} & \mathbf{D}_\omega(\mathbf{q}, \mathbf{p}) \end{bmatrix} \in \mathbb{S}_{>0}^{6 \times 6} \quad (29)$$

where the components $\mathbf{D}_v(\mathbf{q}, \mathbf{p})$ and $\mathbf{D}_\omega(\mathbf{q}, \mathbf{p})$ correspond to $\mathbf{p}_v$ and $\mathbf{p}_\omega$, respectively. The equations of motions on the $SE(3)$ manifold are written in port-Hamiltonian form as

$$\dot{\mathbf{p}} = \mathbf{R} \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} \quad (30a)$$

$$\dot{\mathbf{r}}_i = \mathbf{r}_i \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega}, \quad i = 1, 2, 3 \quad (30b)$$

$$\begin{aligned} \dot{\mathbf{p}}_v &= \mathbf{p}_v \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} - \mathbf{R}^\top \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \\ &\quad - \mathbf{D}_v(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} + \mathbf{b}_v(\mathbf{q})\mathbf{u} \end{aligned} \quad (30c)$$

$$\begin{aligned} \dot{\mathbf{p}}_\omega &= \mathbf{p}_\omega \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} + \mathbf{p}_v \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} \\ &\quad + \sum_{i=1}^3 \mathbf{r}_i \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{r}_i} - \mathbf{D}_\omega(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} + \mathbf{b}_\omega(\mathbf{q})\mathbf{u} \end{aligned} \quad (30d)$$

where the input matrix is $\mathbf{B}(\mathbf{q}) = [\mathbf{b}_v(\mathbf{q})^\top \quad \mathbf{b}_\omega(\mathbf{q})^\top]^\top$.

F. Neural ODE Networks

In this section, we briefly describe neural ODE networks [31], which approximate the closed-loop dynamics $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \boldsymbol{\pi}(\mathbf{x}))$ of a system for some unknown control policy $\mathbf{u} = \boldsymbol{\pi}(\mathbf{x})$ by a neural network $\bar{\mathbf{f}}_\theta(\mathbf{x})$. The parameters of $\bar{\mathbf{f}}_\theta(\mathbf{x})$ are trained using a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}\}_i$ of state trajectory samples $\mathbf{x}_n^{(i)} = \mathbf{x}^{(i)}(t_n^{(i)})$ via forward and backward passes through a differentiable ODE solver, where the backward passes provide the gradient of the loss function. Given an initial state $\mathbf{x}_0^{(i)}$ at time $t_0^{(i)}$, a forward pass returns predicted states at times $t_1^{(i)}, \dots, t_N^{(i)}$ as follows:

$$\{\bar{\mathbf{x}}_1^{(i)}, \dots, \bar{\mathbf{x}}_N^{(i)}\} = \text{ODESolver}(\mathbf{x}_0^{(i)}, \bar{\mathbf{f}}_\theta, t_1^{(i)}, \dots, t_N^{(i)}). \quad (31)$$

The gradient of a loss function, $\sum_{i=1}^D \sum_{j=1}^N \ell(\mathbf{x}_j^{(i)}, \bar{\mathbf{x}}_j^{(i)})$, is back-propagated by solving another ODE with adjoint states. The parameters θ are updated by gradient descent to minimize

$\mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u}$ 表示外部能量来源。在没有耗能元件和外部能量来源的情况下, $\mathcal{J}(\mathbf{q}, \mathbf{p})$ 的反对称性保证了系统的能量守恒。

为了对能量耗散元素(如摩擦或阻力)建模, 我们将矩阵李群上的哈密顿动力学 (18) 重新表述为港口-哈密顿形式 (22)。这些元素通常被建模 [70] 为速度 ξ 的线性变换 $\mathbf{D}(\mathbf{q}, \mathbf{p}) \succeq 0$, 并且仅影响广义动量 $\mathbf{p}$, 即,

$$\mathcal{R}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{D}(\mathbf{q}, \mathbf{p}) \end{bmatrix}. \quad (23)$$

哈密顿动力学 (18) 是 (22) 的一个特例, 其中耗散矩阵为 $\mathbf{D}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, 输入矩阵为 $\mathcal{G}(\mathbf{q}, \mathbf{p}) = [\mathbf{0}^\top \quad \mathbf{B}(\mathbf{q})^\top]^\top$, 互连矩阵 $\mathcal{J}(\mathbf{q}, \mathbf{p})$ 可以通过将 (18) 重新排列并使 $\mathbf{u} = \mathbf{0}$ 得到, 并且由于能量守恒 (20), 该矩阵保证为斜对称。在实现中, 坐标 $\mathbf{q}$ 和动量 $\mathbf{p}$ 可以表示为 $\mathbb{R}^{2n^2}$ 中的向量, 从而得到互连矩阵 $\mathcal{J}(\mathbf{q}, \mathbf{p}) \in \mathbb{R}^{2n^2 \times 2n^2}$ 。

E. Example: Hamiltonian Dynamics on the SE(3) Manifold

在本节中, 我们考虑移动机器人的一般化坐标 $\mathbf{q}$, 它由位置 $\mathbf{p} \in \mathbb{R}^3$ 和方向 $\mathbf{R} \in SO(3)$ 组成。设 $\mathbf{q} = (\mathbf{p}, \mathbf{R})$ 为一般化坐标, $\zeta = (\mathbf{v}, \omega) \in \mathbb{R}^6$ 为一般化速度, 由机体坐标系下的线速度 $\mathbf{v} \in \mathbb{R}^3$ 和机体坐标系下的角速度 $\omega \in \mathbb{R}^3$ 组成。坐标 $\mathbf{q}$ 在李群 $SE(3)$ 上演化, 而一般化速度满足 $\dot{\mathbf{q}} = \mathbf{q}\xi$, 其中 $\xi = \dot{\zeta}$ 是 $\mathfrak{se}(3)$ 中的扭矩阵, 如 (12) 所示。通过 (12) $\mathfrak{se}(3)$ 与 $\mathbb{R}^6$ 之间的同构简化了哈密顿量 (18) 及其端口-哈密顿表示 (22), 如下所示。 $SE(3)$ 上的拉格朗日函数可以用 $\mathbf{q}$ 和 ζ 表示, 而不是 $\mathbf{q}$ 和 ξ , 如下:

$$\mathcal{L}(\mathbf{q}, \zeta) = \frac{1}{2} \zeta^\top \mathbf{M}(\mathbf{q}) \zeta - \mathcal{V}(\mathbf{q}). \quad (24)$$

当刚体坐标系附着在质心时, 广义质量矩阵呈块对角形式 [33]

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} \mathbf{M}_v(\mathbf{q}) & \mathbf{0} \\ \mathbf{0} & \mathbf{M}_\omega(\mathbf{q}) \end{bmatrix} \in \mathbb{S}_{>0}^{6 \times 6} \quad (25)$$

其中 $\mathbf{M}_v(\mathbf{q}), \mathbf{M}_\omega(\mathbf{q}) \in \mathbb{S}_{>0}^{3 \times 3}$ 。广义动量如前所述, 通过对拉格朗日量关于扭转的偏导数来定义

$$\mathbf{p} = \begin{bmatrix} \mathbf{p}_v \\ \mathbf{p}_\omega \end{bmatrix} = \frac{\partial \mathcal{L}(\mathbf{q}, \zeta)}{\partial \zeta} = \mathbf{M}(\mathbf{q}) \zeta \in \mathbb{R}^6. \quad (26)$$

系统的哈密顿函数变为

$$\mathcal{H}(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \zeta - \mathcal{L}(\mathbf{q}, \zeta) = \frac{1}{2} \mathbf{p}^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p} + \mathcal{V}(\mathbf{q}). \quad (27)$$

通过向量化广义坐标 $\mathbf{q} = [\mathbf{p}^\top \quad \mathbf{r}_1^\top \quad \mathbf{r}_2^\top \quad \mathbf{r}_3^\top]^\top$, 可以描述 $SE(3)$ 上的哈密顿动力学

以港-哈密顿形式 (22) [33], [71], [72] 表示, 带有互连矩阵

$$\mathcal{J}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{0} & \mathbf{q}^\times \\ -\mathbf{q}^{\times\top} & \mathbf{p}^\times \end{bmatrix}, \quad \mathbf{p}^\times = \begin{bmatrix} \mathbf{0} & \hat{\mathbf{p}}_v \\ \hat{\mathbf{p}}_v & \hat{\mathbf{p}}_\omega \end{bmatrix} \quad (28)$$

以及输入矩阵 $\mathcal{G}(\mathbf{q}, \mathbf{p}) = [\mathbf{0}^\top \quad \mathbf{B}(\mathbf{q})^\top]^\top$, 其中 $\mathbf{q}^\times = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_1 & \hat{\mathbf{r}}_2 & \hat{\mathbf{r}}_3 \end{bmatrix}$ 。注意, 坐标 $\mathbf{q}$ 在 $SE(3)$ 上的运动学约束 $\dot{\mathbf{q}} = \mathbf{q}\xi$, 它们保证坐标 $\mathbf{q}$ 保持在 $SE(3)$ 中, 不受向量化的影响。端口哈密顿形式允许我们通过耗散矩阵在动力学中建模耗散元件

$$\mathbf{D}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{D}_v(\mathbf{q}, \mathbf{p}) & \mathbf{0} \\ \mathbf{0} & \mathbf{D}_\omega(\mathbf{q}, \mathbf{p}) \end{bmatrix} \in \mathbb{S}_{>0}^{6 \times 6} \quad (29)$$

其中组件 $\mathbf{D}_v(\mathbf{q}, \mathbf{p})$ 和 $\mathbf{D}_\omega(\mathbf{q}, \mathbf{p})$ 分别对应 $\mathbf{p}_v$ 和 $\mathbf{p}_\omega$ 。 $SE(3)$ 流形上的运动方程以端口哈密顿形式写出为

$$\dot{\mathbf{p}} = \mathbf{R} \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} \quad (30a)$$

$$\dot{\mathbf{r}}_i = \mathbf{r}_i \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega}, \quad i = 1, 2, 3 \quad (30b)$$

$$\begin{aligned} \dot{\mathbf{p}}_v &= \mathbf{p}_v \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} - \mathbf{R}^\top \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \\ &\quad - \mathbf{D}_v(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} + \mathbf{b}_v(\mathbf{q})\mathbf{u} \end{aligned} \quad (30c)$$

$$\begin{aligned} \dot{\mathbf{p}}_\omega &= \mathbf{p}_\omega \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} + \mathbf{p}_v \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} \\ &\quad + \sum_{i=1}^3 \mathbf{r}_i \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{r}_i} - \mathbf{D}_\omega(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} + \mathbf{b}_\omega(\mathbf{q})\mathbf{u} \end{aligned} \quad (30d)$$

where the input matrix is $\mathbf{B}(\mathbf{q}) = [\mathbf{b}_v(\mathbf{q})^\top \quad \mathbf{b}_\omega(\mathbf{q})^\top]^\top$.

F. Neural ODE Networks

在本节中, 我们简要描述神经ODE网络 [31], 该网络通过神经网络 $\bar{\mathbf{f}}_\theta(\mathbf{x})$ 近似某个未知控制策略 $\mathbf{u} = \pi(\mathbf{x})$ 下的系统闭环动力学 $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \pi(\mathbf{x}))$ 。 $\bar{\mathbf{f}}_\theta(\mathbf{x})$ 的参数使用状态轨迹样本数据集 $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}\}_i$ 进行训练, 通过可微ODE求解器的前向和反向传播, 其中反向传播提供损失函数的梯度。给定时间 $t_0^{(i)}$ 的初始状态 $\mathbf{x}_0^{(i)}$, 前向传播返回时间 $t_1^{(i)}, \dots, t_N^{(i)}$ 的预测状态如下:

$$\{\bar{\mathbf{x}}_1^{(i)}, \dots, \bar{\mathbf{x}}_N^{(i)}\} = \text{ODESolver}(\mathbf{x}_0^{(i)}, \bar{\mathbf{f}}_\theta, t_1^{(i)}, \dots, t_N^{(i)}). \quad (31)$$

损失函数 $\sum_{i=1}^D \sum_{j=1}^N \ell(\mathbf{x}_j^{(i)}, \bar{\mathbf{x}}_j^{(i)})$ 的梯度通过用伴随状态求解另一个常微分方程进行反向传播。参数 θ 通过梯度下降进行更新以最小化

the loss. For physical systems, Zhong et al. [27] extends the neural ODE by integrating the Hamiltonian dynamics on $\mathbb{R}^n$ into the neural network model $\bar{\mathbf{f}}_\theta(\mathbf{x})$, and consider zero-order hold control input $\mathbf{u}$, leading to a neural ODE network with the following approximated dynamics:

$$\begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{u}} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u}) \\ \mathbf{0} \end{bmatrix}. \quad (32)$$

Recently, neural ODE networks have been extended from Euclidean space to Lie groups [19], [20], [21], [22], guaranteeing that the Lie group constraints are satisfied by the predicted states by design. While it is possible to train our Hamiltonian dynamics model using a Lie group neural ODE network, we leave this investigation for future work due to the lack of suitable open-source software for Lie group ODE integration.

V. LEARNING LIE GROUP HAMILTONIAN DYNAMICS

We consider a Hamiltonian system with unknown kinetic energy $\mathcal{T}(\mathbf{q})$, potential energy $\mathcal{V}(\mathbf{q})$, input matrix $\mathbf{B}(\mathbf{q})$, dissipation matrix $\mathbf{D}(\mathbf{q}, \mathbf{p})$, and design a structured neural ODE network to learn these terms from state-control trajectories.

A. Data Collection

We collect a dataset $\mathcal{D} = \{\mathbf{x}_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ consisting of state sequences $\mathbf{x}_{0:N}^{(i)}$, where $\mathbf{x}_n^{(i)} = [\mathbf{q}_n^{(i)\top} \ \xi_n^{(i)\top}]^\top$ for $n = 0, \dots, N$. Such data are generated by applying a constant control input $\mathbf{u}^{(i)}$ to the system and sampling the state $\mathbf{x}_n^{(i)} = \mathbf{x}^{(i)}(t_n^{(i)})$ at times $t_n^{(i)}$ for $n = 0, \dots, N$. The generalized coordinates $\mathbf{q}$ and velocity ξ may be obtained from a state estimation algorithm, such as odometry algorithm for mobile robots [73], [74], or from a motion capture system. In physics-based simulation, the data can be generated by applying random control inputs $\mathbf{u}^{(i)}$. In real-world applications, where safety is a concern, data may be collected by a human operator manually controlling the robot.

B. Model Architecture

Since robots are physical systems, their dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u})$ satisfy the Hamiltonian formulation (see Section IV-C). To learn the dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u})$ from a trajectory dataset $\mathcal{D}$, we design a neural ODE network (see Section IV-F), approximating the dynamics via a parametric function $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ based on (18).

To integrate the Hamiltonian equations into the structure of $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$, we use four neural networks with parameters $\theta = (\theta_\mathcal{T}, \theta_\mathcal{V}, \theta_\mathbf{D}, \theta_\mathbf{B})$ to approximate the kinetic energy by $\mathcal{T}_\theta(\mathbf{q}, \xi)$, the potential energy by $\mathcal{V}_\theta(\mathbf{q})$, the dissipation matrix by $\mathbf{D}_\theta(\mathbf{q}, \mathbf{p})$, and the input matrix by $\mathbf{B}_\theta(\mathbf{q})$, respectively. Since the generalized momenta $\mathbf{p}$ are not directly available in $\mathcal{D}$, the time derivative of the generalized velocity ξ is obtained from (19). The approximated dynamics function $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ is described with an internal state $\mathbf{p}$ as follows:

$$\dot{\mathbf{q}} = \mathbf{T}_e \mathbf{L}_q \left(\frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \right) \quad (33a)$$

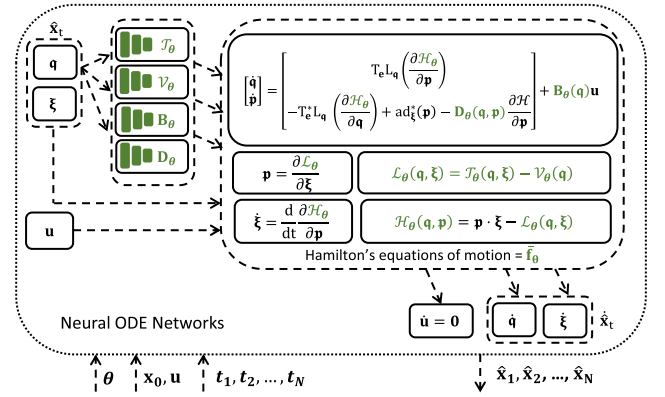


Fig. 2. Architecture of port-Hamiltonian neural ODE network on matrix Lie group. The trainable terms are shown in green.

$$\begin{aligned} \dot{\mathbf{p}} = & \mathbf{ad}_\xi^*(\mathbf{p}) - \mathbf{D}_\theta(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \\ & - \mathbf{T}_e^* \mathbf{L}_q \left(\frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}} \right) + \mathbf{B}_\theta(\mathbf{q}) \mathbf{u} \end{aligned} \quad (33b)$$

$$\dot{\xi} = \frac{d}{dt} \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \quad (33c)$$

where $\mathcal{H}_\theta(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \xi - \mathcal{L}_\theta(\mathbf{q}, \xi)$, and $\mathcal{L}_\theta(\mathbf{q}, \xi) = \mathcal{T}_\theta(\mathbf{q}, \xi) - \mathcal{V}_\theta(\mathbf{q})$. The time derivative $\frac{d}{dt} \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}}$ is calculated using automatic differentiation, e.g., by Pytorch [75]. The approximated dynamics function $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ is implemented in a neural ODE network architecture for training, shown in Fig. 2.

C. Training Process

Let $\bar{\mathbf{x}}^{(i)}(t)$ denote the state trajectory predicted with control input $\mathbf{u}^{(i)}$ by the approximate dynamics $\bar{\mathbf{f}}_\theta$ initialized at $\bar{\mathbf{x}}^{(i)}(t_0^{(i)}) = \mathbf{x}_0^{(i)}$. For sequence i , forward passes through the ODE solver in (31) return the predicted states $\bar{\mathbf{x}}_{0:N}^{(i)}$ at times $t_{0:N}^{(i)}$, where $\bar{\mathbf{x}}_n^{(i)} = [\bar{\mathbf{q}}_n^{(i)\top} \ \bar{\xi}_n^{(i)\top}]^\top$, for $n = 1, \dots, N$. The predicted coordinates $\bar{\mathbf{q}}_n^{(i)}$ and the ground-truth ones $\mathbf{q}_n^{(i)}$ are used to calculate a loss on the Lie group manifold

$$L_q(\theta) = \sum_{i=1}^D \sum_{n=1}^N \left\| \log_{\mathbf{q}_n^{(i)}} \left(\bar{\mathbf{q}}_n^{(i)} \left(\mathbf{q}_n^{(i)} \right)^{-1} \right) \right\|_2^2. \quad (34)$$

We use the squared Euclidean norm to calculate losses for the generalized velocity terms

$$L_\xi(\theta) = \sum_{i=1}^D \sum_{n=1}^N \left\| \xi_n^{(i)} - \bar{\xi}_n^{(i)} \right\|_2^2. \quad (35)$$

The total loss $L(\theta)$ is defined as

$$L(\theta) = L_q(\theta) + L_\xi(\theta). \quad (36)$$

The gradient of the total loss function $L(\theta)$ is back-propagated by solving an ODE with adjoint states [31]. Specifically, let $\mathbf{a} = \frac{\partial L}{\partial \bar{\mathbf{x}}}$ be the adjoint state and $\mathbf{s} = (\bar{\mathbf{x}}, \mathbf{a}, \frac{\partial L}{\partial \theta})$ be the augmented

损失。对于物理系统, Zhong 等人 [27] 通过将 $\mathbb{R}^n$ 上的哈密顿动力学整合到神经网络模型 $\bar{\mathbf{f}}_\theta(\mathbf{x})$ 中扩展了神经 ODE, 并考虑零阶保持控制输入 $\mathbf{u}$, 从而得到具有以下近似动力学的神经 ODE 网络:

$$\begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{u}} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u}) \\ \mathbf{0} \end{bmatrix}. \quad (32)$$

最近, 神经ODE网络已从欧几里得空间扩展到李群[19], [20], [21], [22], 保证预测状态在设计上满足李群约束。虽然可以使用李群神经ODE网络训练我们的哈密顿动力学模型, 但由于缺乏合适的开源李群ODE积分软件, 我们将此研究留待未来进行。

五、学习李群哈密顿动力学

我们考虑一个哈密顿系统, 其动能为 $\mathcal{T}(\mathbf{q})$, 势能为 $\mathcal{V}(\mathbf{q})$, 输入矩阵为 $\mathbf{B}(\mathbf{q})$, 耗散矩阵为 $\mathbf{D}(\mathbf{q}, \mathbf{p})$, 并设计一个结构化的神经 ODE 网络从状态-控制轨迹中学习这些项。

A. Data Collection

我们收集了一个数据集 $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$, 包含状态序列 $\mathbf{x}_{0:N}^{(i)}$, 其中 $\mathbf{x}_n^{(i)} = [\mathbf{q}_n^{(i)\top} \ \xi_n^{(i)\top}]^\top$ 对于 $n = 0, \dots, N$ 。这些数据是通过对系统施加恒定控制输入 $\mathbf{u}^{(i)}$ 并在时间 $t_n^{(i)}$ 对状态 $\mathbf{x}_n^{(i)} = \mathbf{x}^{(i)}(t_n^{(i)})$ 进行采样生成的, 对于 $n = 0, \dots, N$ 。广义坐标 $\mathbf{q}$ 和速度 ξ 可以通过状态估计算法获得, 例如移动机器人里程计算法 [73], [74], 或者通过动作捕捉系统。在基于物理的仿真中, 数据可以通过施加随机控制输入 $\mathbf{u}^{(i)}$ 生成。在现实应用中, 如果涉及安全问题, 数据可能由人工操作员手动控制机器人收集。

B. Model Architecture

由于机器人是物理系统, 它们的动力学 $\mathbf{f}(\mathbf{x}, \mathbf{u})$ 满足哈密顿形式 (参见第 IV-C 节)。为了从轨迹数据集 $\mathcal{D}$ 学习动力学 $\mathbf{f}(\mathbf{x}, \mathbf{u})$, 我们设计了一个神经 ODE 网络 (参见第 IV-F 节), 通过基于 (18) 的参数函数 $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ 来近似动力学。

为了将哈密顿方程整合到 $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ 的结构中, 我们使用四个参数为 $\theta = (\theta_T, \theta_V, \theta_D, \theta_B)$ 的神经网络分别近似动能为 $\mathcal{T}_\theta(\mathbf{q}, \xi)$ 、势能为 $\mathcal{V}_\theta(\mathbf{q})$ 、耗散矩阵为 $\mathbf{D}_\theta(\mathbf{q}, \mathbf{p})$ 以及输入矩阵为 $\mathbf{B}_\theta(\mathbf{q})$ 。由于广义动量 $\mathbf{p}$ 在 $\mathcal{D}$ 中不可直接获得, 广义速度 ξ 的时间导数从公式 (19) 中得到。近似的动力学函数 $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ 通过内部状态 $\mathbf{p}$ 描述如下:

$$\dot{\mathbf{q}} = \mathbf{T}_e \mathbf{L}_q \left(\frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \right) \quad (33a)$$

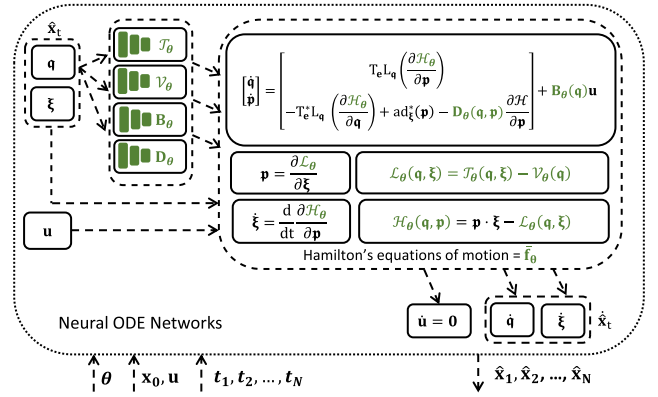


图2. 矩阵李群上端口-哈密顿神经ODE网络的结构。可训练项用绿色表示。

$$\begin{aligned} \dot{\mathbf{p}} = & \mathbf{ad}_\xi^*(\mathbf{p}) - \mathbf{D}_\theta(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \\ & - \mathbf{T}_e^* \mathbf{L}_q \left(\frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}} \right) + \mathbf{B}_\theta(\mathbf{q}) \mathbf{u} \end{aligned} \quad (33b)$$

$$\dot{\xi} = \frac{d}{dt} \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \quad (33c)$$

$\mathcal{H}_\theta(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \xi - \mathcal{L}_\theta(\mathbf{q}, \xi)$ 和 $\mathcal{L}_\theta(\mathbf{q}, \xi) = \mathcal{T}_\theta(\mathbf{q}, \xi) - \mathcal{V}_\theta(\mathbf{q})$ 。时间导数 $\frac{d}{dt} \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}}$ 是使用自动微分计算的, 例如, 通过 Pytorch [75]。近似动力学函数 $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ 在用于训练的神经 ODE 网络架构中实现, 如图 2 所示。

C. Training Process

令 $\bar{\mathbf{x}}^{(i)}(t)$ 表示由近似动力学 $\bar{\mathbf{f}}_\theta$ 在初值 $\bar{\mathbf{x}}^{(i)}(t_0^{(i)}) = \mathbf{x}_0^{(i)}$ 下, 用控制输入 $\mathbf{u}^{(i)}$ 预测的状态轨迹。对于序列 i , 通过 (31) 中的 ODE 求解器的前向传递返回在时间 $t_{0:N}^{(i)}$ 的预测状态 $\bar{\mathbf{x}}_{0:N}^{(i)}$, 其中 $\bar{\mathbf{x}}_n^{(i)} = [\bar{\mathbf{q}}_n^{(i)\top} \ \bar{\xi}_n^{(i)\top}]^\top$, 对于 $n = 1, \dots, N$ 。预测的坐标 $\bar{\mathbf{q}}_n^{(i)}$ 与真实坐标 $\mathbf{q}_n^{(i)}$ 被用来在李群流形上计算损失。

$$L_q(\theta) = \sum_{i=1}^D \sum_{n=1}^N \left\| \log^\vee \left(\bar{\mathbf{q}}_n^{(i)} \left(\mathbf{q}_n^{(i)} \right)^{-1} \right) \right\|_2^2. \quad (34)$$

我们使用平方欧几里得范数来计算广义速度项的损失

$$L_\xi(\theta) = \sum_{i=1}^D \sum_{n=1}^N \left\| \xi_n^{(i)} - \bar{\xi}_n^{(i)} \right\|_2^2. \quad (35)$$

总损失 $L(\theta)$ 定义为

$$L(\theta) = L_q(\theta) + L_\xi(\theta). \quad (36)$$

总损失函数 $L(\theta)$ 的梯度通过求解具有伴随状态的常微分方程 [31] 来进行反向传播。具体来说, 设 $\mathbf{a} = \frac{\partial L}{\partial \bar{\mathbf{x}}}$ 为伴随状态, $\mathbf{s} = (\bar{\mathbf{x}}, \mathbf{a}, \frac{\partial L}{\partial \theta})$ 为扩展

state. The augmented state dynamics are [31]

$$\dot{\mathbf{s}} = \bar{\mathbf{f}}_{\mathbf{s}} = \left(\bar{\mathbf{f}}_{\boldsymbol{\theta}}, -\mathbf{a}^\top \frac{\partial \bar{\mathbf{f}}_{\boldsymbol{\theta}}}{\partial \bar{\mathbf{x}}}, -\mathbf{a}^\top \frac{\partial \bar{\mathbf{f}}_{\boldsymbol{\theta}}}{\partial \boldsymbol{\theta}} \right). \quad (37)$$

The predicted state $\bar{\mathbf{x}}$, the adjoint state $\mathbf{a}$, and the derivatives $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$ can be obtained by a single call to a reverse-time ODE solver starting from $\mathbf{s}_N = \mathbf{s}(t_N)$

$$\mathbf{s}_0 = \left(\bar{\mathbf{x}}_0, \mathbf{a}_0, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} \right) = \text{ODESolver}(\mathbf{s}_N, \bar{\mathbf{f}}_{\mathbf{s}}, t_N) \quad (38)$$

where at each time $t_k, k = 1, \dots, N$, the adjoint state $\mathbf{a}_k$ at time t_k is reset to $\frac{\partial \mathcal{L}}{\partial \bar{\mathbf{x}}_k}$. The resulting derivative $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$ is used to update the parameters $\boldsymbol{\theta}$ using gradient descent. Note that even though the Lie group and Hamiltonian structures are preserved in the continuous-time dynamics function $\bar{\mathbf{f}}$, the Lie group constraints might still be violated by the predicted states when the model is trained with a neural ODE network on the embedding space $\mathbb{R}^{n \times n}$ instead of on the matrix Lie group [21], [22]. To prevent this, we use a high-order integrator, such as fifth-order Runge–Kutta [76], in the Euclidean space neural ODE, for which open-source software is available [31], [77]. Investigating how to train Hamiltonian models with a neural ODE network defined directly on the Lie group [21], [22], [78] is an interesting future direction. For example, the Lie group neural ODE by Wotte et al. [22] offers an approach to guarantee Lie group constraints in the dynamics model. Closely related to this direction, Duruisseaux et al. [54] learn discrete-time Hamiltonian dynamics while guaranteeing Lie group constraints by design using variational integration.

D. Application to SE(3) Hamiltonian Dynamics Learning

This section applies our Lie group Hamiltonian dynamics learning approach to estimate mobile robot dynamics on the SE(3) manifold (see Section IV-E).

1) *Neural ODE Model Architecture*: When the Hamiltonian dynamics in (18) are defined on the SE(3) manifold, the equations of motion become (30). The neural ODE network architecture in (33) is simplified as follows. We use five neural networks with parameters $\boldsymbol{\theta} = (\boldsymbol{\theta}_{\mathbf{v}}, \boldsymbol{\theta}_{\boldsymbol{\omega}}, \boldsymbol{\theta}_V, \boldsymbol{\theta}_{\mathbf{D}}, \boldsymbol{\theta}_{\mathbf{B}})$ to approximate the blocks $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$ of the inverse generalized mass in (25), the potential energy $\mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q})$, the dissipation matrix $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$, and the input matrix $\mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q})$, respectively. The approximated kinetic energy is calculated as $\mathcal{T}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p}) = \frac{1}{2} \mathbf{p}^\top \mathbf{M}_{\boldsymbol{\theta}}^{-1}(\mathbf{q}) \mathbf{p}$, where $\mathbf{M}_{\boldsymbol{\theta}}(\mathbf{q}) = \text{diag}(\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}(\mathbf{q}), \mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}(\mathbf{q}))$.

2) *Neural Network Design*: In many applications, nominal information is available about the generalized mass matrices $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, the potential energy $\mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q})$, the dissipation matrix $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$, and the input matrix $\mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q})$, and can be included in the neural network design.

Let $\mathbf{M}_{\mathbf{v}0}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega}0}^{-1}(\mathbf{q})$, and $\mathbf{D}_0(\mathbf{q}, \mathbf{p})$ be the nominal values of the generalized mass matrices $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$ and the dissipation matrix $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$ with Cholesky decomposition

$$\begin{aligned} \mathbf{M}_{\mathbf{v}0}^{-1}(\mathbf{q}) &= \mathbf{L}_{\mathbf{v}0}(\mathbf{q}) \mathbf{L}_{\mathbf{v}0}^\top(\mathbf{q}) \\ \mathbf{M}_{\boldsymbol{\omega}0}^{-1}(\mathbf{q}) &= \mathbf{L}_{\boldsymbol{\omega}0}(\mathbf{q}) \mathbf{L}_{\boldsymbol{\omega}0}^\top(\mathbf{q}) \\ \mathbf{D}_0(\mathbf{q}) &= \mathbf{L}_{\mathbf{D}0}(\mathbf{q}) \mathbf{L}_{\mathbf{D}0}^\top(\mathbf{q}). \end{aligned} \quad (39)$$

The learned terms $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, and $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$ are obtained using Cholesky decomposition

$$\begin{aligned} \mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q}) &= (\mathbf{L}_{\mathbf{v}0}(\mathbf{q}) + \mathbf{L}_{\mathbf{v}}(\mathbf{q})) (\mathbf{L}_{\mathbf{v}0}(\mathbf{q}) + \mathbf{L}_{\mathbf{v}}(\mathbf{q}))^\top + \varepsilon_{\mathbf{v}} \mathbf{I} \\ \mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q}) &= (\mathbf{L}_{\boldsymbol{\omega}0}(\mathbf{q}) + \mathbf{L}_{\boldsymbol{\omega}}(\mathbf{q})) (\mathbf{L}_{\boldsymbol{\omega}0}(\mathbf{q}) + \mathbf{L}_{\boldsymbol{\omega}}(\mathbf{q}))^\top + \varepsilon_{\boldsymbol{\omega}} \mathbf{I} \\ \mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p}) &= (\mathbf{L}_{\mathbf{D}0}(\mathbf{q}, \mathbf{p}) + \mathbf{L}_{\mathbf{D}}(\mathbf{q}, \mathbf{p})) (\mathbf{L}_{\mathbf{D}0}(\mathbf{q}, \mathbf{p}) + \mathbf{L}_{\mathbf{D}}(\mathbf{q}, \mathbf{p}))^\top \end{aligned} \quad (40)$$

where $\mathbf{L}_{\mathbf{v}}(\mathbf{q})$, $\mathbf{L}_{\boldsymbol{\omega}}(\mathbf{q})$, and $\mathbf{L}_{\mathbf{D}}(\mathbf{q}, \mathbf{p})$ are lower triangular matrices implemented as three neural networks with parameters $\boldsymbol{\theta}_{\mathbf{v}}$, $\boldsymbol{\theta}_{\boldsymbol{\omega}}$, and $\boldsymbol{\theta}_{\mathbf{D}}$, respectively, and $\varepsilon_{\mathbf{v}}, \varepsilon_{\boldsymbol{\omega}} > 0$.

The potential energy $\mathcal{V}(\mathbf{q})$ and the input matrix $\mathbf{B}(\mathbf{q})$ are implemented with nominal values $\mathcal{V}_0(\mathbf{q})$ and $\mathbf{B}_0(\mathbf{q})$ as follows:

$$\begin{aligned} \mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q}) &= \mathcal{V}_0(\mathbf{q}) + \mathcal{V}_{\mathbf{v}}(\mathbf{q}) \\ \mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q}) &= \mathbf{B}_0(\mathbf{q}) + \mathbf{L}_{\mathbf{B}}(\mathbf{q}) \end{aligned} \quad (41)$$

where $\mathcal{V}_{\mathbf{v}}(\mathbf{q})$ and $\mathbf{L}_{\mathbf{B}}(\mathbf{q})$ are two neural networks with parameters $\boldsymbol{\theta}_{\mathbf{v}}$ and $\boldsymbol{\theta}_{\mathbf{B}}$, respectively.

3) *Loss Function*: The orientation loss is calculated as

$$L_{\mathbf{R}}(\boldsymbol{\theta}) = \sum_{i=1}^D \sum_{n=1}^N \left\| \log_{SO(3)}^{\mathbf{v}}(\bar{\mathbf{R}}_n^{(i)} \mathbf{R}_n^{(i)\top}) \right\|_2^2. \quad (42)$$

We use the squared Euclidean norm to calculate losses for the position and generalized velocity terms

$$\begin{aligned} L_{\mathbf{p}}(\boldsymbol{\theta}) &= \sum_{i=1}^D \sum_{n=1}^N \left\| \mathbf{p}_n^{(i)} - \bar{\mathbf{p}}_n^{(i)} \right\|_2^2 \\ L_{\boldsymbol{\zeta}}(\boldsymbol{\theta}) &= \sum_{i=1}^D \sum_{n=1}^N \left\| \boldsymbol{\zeta}_n^{(i)} - \bar{\boldsymbol{\zeta}}_n^{(i)} \right\|_2^2. \end{aligned} \quad (43)$$

The total loss $\mathcal{L}(\boldsymbol{\theta})$ is defined as

$$L(\boldsymbol{\theta}) = L_{\mathbf{R}}(\boldsymbol{\theta}) + L_{\mathbf{p}}(\boldsymbol{\theta}) + L_{\boldsymbol{\zeta}}(\boldsymbol{\theta}). \quad (44)$$

VI. ENERGY-BASED CONTROL DESIGN

The function $\bar{\mathbf{f}}_{\boldsymbol{\theta}}$ (33) learned in Section V satisfies the port-Hamiltonian dynamics in (22) by design. This section extends the interconnection and damping assignment passivity-based control (IDA-PBC) approach [51], [61], [62] to Lie groups to achieve trajectory tracking (Problem 2) based on the learned port-Hamiltonian dynamics. We further derive a tracking controller specifically for learned Hamiltonian dynamics on the SE(3) manifold in Section V-D. In the remainder of this article, we omit the subscript $\boldsymbol{\theta}$ in the learned model for readability.

A. IDA-PBC Control Design for Trajectory Tracking

Consider a desired regulation point $(\mathbf{q}^*, \mathbf{p}^*) \in \mathcal{T}^*\mathcal{G}$ that the system should be stabilized to. The Hamiltonian function $\mathcal{H}(\mathbf{q}, \mathbf{p})$, representing the total energy of the system, generally does not have a minimum at $(\mathbf{q}^*, \mathbf{p}^*)$. An IDA-PBC controller [51], [58], [61] is designed to inject additional energy $\mathcal{H}_a(\mathbf{q}, \mathbf{p})$ such that the desired total energy

$$\mathcal{H}_d(\mathbf{q}, \mathbf{p}) = \mathcal{H}(\mathbf{q}, \mathbf{p}) + \mathcal{H}_a(\mathbf{q}, \mathbf{p}) \quad (45)$$

状态。增强状态动力学为 [31]

$$\dot{\mathbf{s}} = \bar{\mathbf{f}}_{\mathbf{s}} = \left(\bar{\mathbf{f}}_{\boldsymbol{\theta}}, -\mathbf{a}^\top \frac{\partial \bar{\mathbf{f}}_{\boldsymbol{\theta}}}{\partial \bar{\mathbf{x}}}, -\mathbf{a}^\top \frac{\partial \bar{\mathbf{f}}_{\boldsymbol{\theta}}}{\partial \boldsymbol{\theta}} \right). \quad (37)$$

通过从 $\mathbf{s}_N = \mathbf{s}(t_N)$ 开始的一次反向时间 ODE 求解器调用，可以获得预测状态 $\bar{\mathbf{x}}$ 、伴随状态 $\mathbf{a}$ 和导数 $\frac{\partial L}{\partial \boldsymbol{\theta}}$

$$\mathbf{s}_0 = \left(\bar{\mathbf{x}}_0, \mathbf{a}_0, \frac{\partial L}{\partial \boldsymbol{\theta}} \right) = \text{ODESolver}(\mathbf{s}_N, \bar{\mathbf{f}}_{\mathbf{s}}, t_N) \quad (38)$$

在每个时间点 $t_k, k = 1, \dots, N$ ，伴随状态 $\mathbf{a}_k$ 在时间 t_k 被重置为 $\frac{\partial L}{\partial \bar{\mathbf{x}}_k}$ 。得到的导数 $\frac{\partial L}{\partial \boldsymbol{\theta}}$ 用于通过梯度下降更新参数 $\boldsymbol{\theta}$ 。注意，即使在连续时间动力学函数 $\bar{\mathbf{f}}$ 中李群和哈密顿结构得以保留，当模型在嵌入空间 $\mathbb{R}^{n \times n}$ 上用神经 ODE 网络进行训练而不是在矩阵李群上训练时，[21], [22]，预测状态仍可能违反李群约束。为了防止这种情况，我们在欧几里得空间神经 ODE 中使用高阶积分器，例如五阶 Runge-Kutta [76]，并且有开源软件可用 [31], [77]。研究如何在李群上直接定义的神神经 ODE 网络 [21], [22], [78] 上训练哈密顿模型是一个有趣的未来方向。例如，Wotte 等人的李群神经 ODE [22] 提供了一种保证动力学模型中李群约束的方法。密切相关

D. Application to SE(3) Hamiltonian Dynamics Learning

本节将我们的李群哈密顿动力学学习方法应用于估计 SE(3) 流形上的移动机器人动力学（参见第 IV-E 节）。

1) *Neural ODE Model Architecture*: 当式 (18) 中的哈密顿动力学定义在 SE(3) 流形上时，运动方程变为 (30)。式 (33) 中的神经 ODE 网络结构简化如下。我们使用五个带有参数 $\boldsymbol{\theta} = (\boldsymbol{\theta}_v, \boldsymbol{\theta}_\omega, \boldsymbol{\theta}_V, \boldsymbol{\theta}_D, \boldsymbol{\theta}_B)$ 的神经网络来分别近似逆广义质量中的块 $\mathbf{M}_{v;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ 、 $\mathbf{M}_{\omega;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ （如式 (25)）、势能 $\mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q})$ 、耗散矩阵 $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$ 和输入矩阵 $\mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q})$ 。近似的动能计算为 $\mathcal{T}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p}) = \frac{1}{2}$

$\mathbf{p}^\top \mathbf{M}_{\boldsymbol{\theta}}^{-1}(\mathbf{q}) \mathbf{p}$ ，其中 $\mathbf{M}_{\boldsymbol{\theta}}(\mathbf{q}) = \text{diag}(\mathbf{M}_{v;\boldsymbol{\theta}}(\mathbf{q}), \mathbf{M}_{\omega;\boldsymbol{\theta}}(\mathbf{q}))$ 。

2) *Neural Network Design*: 在许多应用中，关于广义质量矩阵 $\mathbf{M}_{v;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ 、 $\mathbf{M}_{\omega;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ 、势能 $\mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q})$ 、耗散矩阵 $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$ 以及输入矩阵 $\mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q})$ 的名义信息是可用的，并且可以包含在神经网络设计中。

设 $\mathbf{M}_{v0}^{-1}(\mathbf{q})$ 、 $\mathbf{M}_{\omega0}^{-1}(\mathbf{q})$ 和 $\mathbf{D}_0(\mathbf{q}, \mathbf{p})$ 为广义质量矩阵 $\mathbf{M}_{v;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ 、 $\mathbf{M}_{\omega;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ 以及耗散矩阵 $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$ 的名义值，其具有 Cholesky 分解

$$\begin{aligned} \mathbf{M}_{v0}^{-1}(\mathbf{q}) &= \mathbf{L}_{v0}(\mathbf{q}) \mathbf{L}_{v0}^\top(\mathbf{q}) \\ \mathbf{M}_{\omega0}^{-1}(\mathbf{q}) &= \mathbf{L}_{\omega0}(\mathbf{q}) \mathbf{L}_{\omega0}^\top(\mathbf{q}) \\ \mathbf{D}_0(\mathbf{q}) &= \mathbf{L}_{D0}(\mathbf{q}) \mathbf{L}_{D0}^\top(\mathbf{q}). \end{aligned} \quad (39)$$

通过 Cholesky 分解获得学习到的术语 $\mathbf{M}_{v;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ 、 $\mathbf{M}_{\omega;\boldsymbol{\theta}}^{-1}(\mathbf{q})$ 和 $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$

$$\begin{aligned} \mathbf{M}_{v;\boldsymbol{\theta}}^{-1}(\mathbf{q}) &= (\mathbf{L}_{v0}(\mathbf{q}) + \mathbf{L}_v(\mathbf{q})) (\mathbf{L}_{v0}(\mathbf{q}) + \mathbf{L}_v(\mathbf{q}))^\top + \varepsilon_v \mathbf{I} \\ \mathbf{M}_{\omega;\boldsymbol{\theta}}^{-1}(\mathbf{q}) &= (\mathbf{L}_{\omega0}(\mathbf{q}) + \mathbf{L}_\omega(\mathbf{q})) (\mathbf{L}_{\omega0}(\mathbf{q}) + \mathbf{L}_\omega(\mathbf{q}))^\top + \varepsilon_\omega \mathbf{I} \\ \mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p}) &= (\mathbf{L}_{D0}(\mathbf{q}, \mathbf{p}) + \mathbf{L}_D(\mathbf{q}, \mathbf{p})) (\mathbf{L}_{D0}(\mathbf{q}, \mathbf{p}) + \mathbf{L}_D(\mathbf{q}, \mathbf{p}))^\top \end{aligned} \quad (40)$$

其中 $\mathbf{L}_v(\mathbf{q})$ 、 $\mathbf{L}_\omega(\mathbf{q})$ 和 $\mathbf{L}_D(\mathbf{q}, \mathbf{p})$ 是作为三个神经网络实现的下三角矩阵，其参数分别为 $\boldsymbol{\theta}_v$ 、 $\boldsymbol{\theta}_\omega$ 和 $\boldsymbol{\theta}_D$ ，且 $\varepsilon_v, \varepsilon_\omega > 0$ 。

势能 $\mathcal{V}(\mathbf{q})$ 和输入矩阵 $\mathbf{B}(\mathbf{q})$ 使用标称值 $\mathcal{V}_0(\mathbf{q})$ 和 $\mathbf{B}_0(\mathbf{q})$ 实现如下：

$$\begin{aligned} \mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q}) &= \mathcal{V}_0(\mathbf{q}) + \mathcal{V}_v(\mathbf{q}) \\ \mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q}) &= \mathbf{B}_0(\mathbf{q}) + \mathbf{L}_B(\mathbf{q}) \end{aligned} \quad (41)$$

其中 $\mathcal{V}_v(\mathbf{q})$ 和 $\mathbf{L}_B(\mathbf{q})$ 是两个神经网络，参数分别为 $\boldsymbol{\theta}_v$ 和 $\boldsymbol{\theta}_B$ 。

3) *Loss Function*: 方向损失的计算方式为

$$L_{\mathbf{R}}(\boldsymbol{\theta}) = \sum_{i=1}^D \sum_{n=1}^N \left\| \log_{SO(3)}^v(\bar{\mathbf{R}}_n^{(i)} \mathbf{R}_n^{(i)\top}) \right\|_2^2. \quad (42)$$

我们使用平方欧几里得范数来计算位置和广义速度项的损失

$$\begin{aligned} L_{\mathbf{p}}(\boldsymbol{\theta}) &= \sum_{i=1}^D \sum_{n=1}^N \left\| \mathbf{p}_n^{(i)} - \bar{\mathbf{p}}_n^{(i)} \right\|_2^2 \\ L_{\boldsymbol{\zeta}}(\boldsymbol{\theta}) &= \sum_{i=1}^D \sum_{n=1}^N \left\| \boldsymbol{\zeta}_n^{(i)} - \bar{\boldsymbol{\zeta}}_n^{(i)} \right\|_2^2. \end{aligned} \quad (43)$$

总损失 $\mathcal{L}(\boldsymbol{\theta})$ 定义为

$$L(\boldsymbol{\theta}) = L_{\mathbf{R}}(\boldsymbol{\theta}) + L_{\mathbf{p}}(\boldsymbol{\theta}) + L_{\boldsymbol{\zeta}}(\boldsymbol{\theta}). \quad (44)$$

VI. 基于能量的控制设计

在第五节中学习的函数 $\bar{\mathbf{f}}_{\boldsymbol{\theta}}$ (33) 通过设计满足了式 (22) 中的端口-哈密顿动力学。本节将互连与阻尼分配基于耗散的控制 (IDA-PBC) 方法 [51], [61], [62] 扩展到李群，以基于所学习的端口-哈密顿动力学实现轨迹跟踪（问题 2）。我们在第五节-D 中进一步推导了一个专门针对 SE(3) 流形上学习的哈密顿动力学的跟踪控制器。在本文的其余部分，为了可读性，我们省略了学习模型中的下标 $\boldsymbol{\theta}$ 。

A. IDA-PBC Control Design for Trajectory Tracking

考虑系统应当稳定到的期望调节点 $(\mathbf{q}^*, \mathbf{p}^*) \in \mathbf{T}^*G$ 。哈密顿函数 $\mathcal{H}(\mathbf{q}, \mathbf{p})$ ，表示系统的总能量，通常在 $(\mathbf{q}^*, \mathbf{p}^*)$ 处没有最小值。IDA-PBC 控制器 [51], [58], [61] 旨在注入额外能量 $\mathcal{H}_a(\mathbf{q}, \mathbf{p})$ ，以使期望的总能量

$$\mathcal{H}_d(\mathbf{q}, \mathbf{p}) = \mathcal{H}(\mathbf{q}, \mathbf{p}) + \mathcal{H}_a(\mathbf{q}, \mathbf{p}) \quad (45)$$

achieves its minimum at $(\mathbf{q}^*, \mathbf{p}^*)$. In other words, the closed-loop system obtained by applying the controller to the port-Hamiltonian dynamics in (22) should have the form

$$\begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} = (\mathcal{J}_d(\mathbf{q}, \mathbf{p}) - \mathcal{R}_d(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}} \end{bmatrix} \quad (46)$$

to ensure that $(\mathbf{q}^*, \mathbf{p}^*)$ is an equilibrium. The control input $\mathbf{u}$ should be chosen so that (22) and (46) are equal. This matching equation design does not directly apply to trajectory tracking, especially for underactuated systems [61], [62].

Consider a desired trajectory $(\mathbf{q}^*(t), \mathbf{p}^*(t))$ that the system should track. Let $(\mathbf{q}_e(t), \mathbf{p}_e(t))$ denote the error in the generalized coordinates and momentum, respectively, where $\mathbf{q}_e = (\mathbf{q}^*)^{-1} \mathbf{q} \in \mathbf{G}$ and $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^* \in \mathbf{T}_{\mathbf{q}_e}^* \mathbf{G}$. For trajectory tracking, the desired total energy $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ is defined in terms of the error state, with desired closed-loop dynamics

$$\begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} = (\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix}. \quad (47)$$

Matching (47) with (22) leads to the following requirement for the control input:

$$\begin{aligned} \mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u} &= (\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \\ &\quad - (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix}. \end{aligned} \quad (48)$$

The control input can be obtained from (48) as the sum $\mathbf{u} = \mathbf{u}_{ES} + \mathbf{u}_{DI}$ of an energy-shaping component $\mathbf{u}_{ES}$ and a damping-injection component $\mathbf{u}_{DI}$

$$\mathbf{u}_{ES} = \mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \left((\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} \right) \quad (49a)$$

$$- (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} \Bigg)$$

$$\mathbf{u}_{DI} = -\mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \quad (49b)$$

where $\mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) = (\mathcal{G}^\top(\mathbf{q}, \mathbf{p})\mathcal{G}(\mathbf{q}, \mathbf{p}))^{-1}\mathcal{G}^\top(\mathbf{q}, \mathbf{p})$ is the pseudoinverse of $\mathcal{G}(\mathbf{q}, \mathbf{p})$. The control input $\mathbf{u}_{ES}$ exists as long as the desired interconnection matrix $\mathcal{J}_d$, dissipation matrix $\mathcal{R}_d$, and total energy $\mathcal{H}_d$ satisfy the following matching condition for all $(\mathbf{q}, \mathbf{p}) \in \mathbf{T}^*\mathbf{G}$ and $(\mathbf{q}_e, \mathbf{p}_e) \in \mathbf{T}^*\mathbf{G}$:

$$\begin{aligned} \mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \left((\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \right. \\ \left. - (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} \right) = 0 \end{aligned} \quad (50)$$

where $\mathcal{G}^\perp(\mathbf{q}, \mathbf{p})$ is a maximal-rank left annihilator of $\mathcal{G}(\mathbf{q}, \mathbf{p})$, i.e., $\mathcal{G}^\perp(\mathbf{q}, \mathbf{p})\mathcal{G}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$.

B. Tracking Control for Port-Hamiltonian Dynamics on the SE(3) Manifold

Consider a desired state trajectory $\mathbf{x}^*(t) = (\mathbf{q}^*(t), \boldsymbol{\zeta}^*(t))$ that the system should track where $\mathbf{q}^*(t) \in \text{SE}(3)$ is the desired pose and $\boldsymbol{\zeta}^*(t) = [\mathbf{v}^*(t)^\top \boldsymbol{\omega}^*(t)^\top]^\top$ is the desired generalized velocity expressed in the desired frame. Let $\mathbf{p}^* = \mathbf{M} \begin{bmatrix} \mathbf{R}^\top \mathbf{R}^* \mathbf{v}^* \\ \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix}$

denote the desired momentum, defined based on (26) with the desired velocity expressed in the body frame. Let $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^*$ and $\mathbf{R}_e = \mathbf{R}^{*\top} \mathbf{R} = [\mathbf{r}_{e1} \ \mathbf{r}_{e2} \ \mathbf{r}_{e3}]^\top$ be the position error and rotation error between the current orientation $\mathbf{R}$ and the desired one $\mathbf{R}^*$, respectively. The vectorized error $\mathbf{q}_e$ in the generalized coordinates is

$$\mathbf{q}_e = [\mathbf{p}_e^\top \ \mathbf{r}_{e1}^\top \ \mathbf{r}_{e2}^\top \ \mathbf{r}_{e3}^\top]^\top. \quad (51)$$

The error in the generalized momenta is $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^*$, described in the body frame. The desired total energy is defined in terms of the error state as

$$\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) = \frac{1}{2} \mathbf{p}_e^\top \mathbf{M}_d^{-1}(\mathbf{q}_e) \mathbf{p}_e + V_d(\mathbf{q}_e) \quad (52)$$

where $\mathbf{M}_d(\mathbf{q}_e)$ and $V_d(\mathbf{q}_e)$ are the desired generalized mass and potential energy. Choosing the following desired interconnection matrix and dissipation matrix:

$$\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) = \begin{bmatrix} \mathbf{0} & \mathbf{J}_1 \\ -\mathbf{J}_1^\top & \mathbf{J}_2 \end{bmatrix}, \quad \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e) = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_d \end{bmatrix} \quad (53)$$

and plugging $\mathcal{J}(\mathbf{q}, \mathbf{p})$ and $\mathcal{R}(\mathbf{q}, \mathbf{p})$ from (28) into the matching equations in (48), leads to

$$\mathbf{0} = \mathbf{J}_1 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{q}} - \dot{\mathbf{q}}_e \quad (54)$$

$$\begin{aligned} \mathbf{B}(\mathbf{q})\mathbf{u} &= \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \\ &\quad - \mathbf{K}_d \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} + \mathbf{D}(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}_e. \end{aligned} \quad (55)$$

Assuming $\mathbf{M}_d(\mathbf{q}_e) = \mathbf{M}(\mathbf{q})$, (54) is satisfied if we choose $\mathbf{J}_1 = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_{e1}^\top & \hat{\mathbf{r}}_{e2}^\top & \hat{\mathbf{r}}_{e3}^\top \end{bmatrix}^\top$. Indeed, we have $\dot{\mathbf{q}} = \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}}$ [from (11) and (19)] and

$$\frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} = \mathbf{M}_d^{-1} \mathbf{p}_e = \begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \begin{bmatrix} \mathbf{v} - \mathbf{R}^\top \mathbf{R}^* \mathbf{v}^* \\ \boldsymbol{\omega} - \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix}.$$

The error dynamics becomes

$$\dot{\mathbf{q}}_e = \begin{bmatrix} \dot{\mathbf{p}} - \dot{\mathbf{p}}^* \\ \dot{\mathbf{r}}_{e1} \\ \dot{\mathbf{r}}_{e2} \\ \dot{\mathbf{r}}_{e3} \end{bmatrix} = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_{e1}^\top & \hat{\mathbf{r}}_{e2}^\top & \hat{\mathbf{r}}_{e3}^\top \end{bmatrix}^\top \begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \mathbf{J}_1 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \quad (56)$$

在 $(\mathbf{q}^*, \mathbf{p}^*)$ 达到其最小值。换句话说, 将控制器应用于 (22) 中的端口-哈密顿动力学所得到的闭环系统应具有以下形式

$$\begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} = (\mathcal{J}_d(\mathbf{q}, \mathbf{p}) - \mathcal{R}_d(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}} \end{bmatrix} \quad (46)$$

以确保 $(\mathbf{q}^*, \mathbf{p}^*)$ 是一个平衡点。控制输入 $\mathbf{u}$ 应该被选择, 使得 (22) 和 (46) 相等。这种匹配方程设计并不直接适用于轨迹跟踪, 尤其是欠驱动系统 [61], [62]。

考虑系统应当跟踪的期望轨迹 $(\mathbf{q}^*(t), \mathbf{p}^*(t))$ 。设 $(\mathbf{q}_e(t), \mathbf{p}_e(t))$ 分别表示广义坐标和动量的误差, 其中 $\mathbf{q}_e = (\mathbf{q}^*)^{-1} \mathbf{q} \in \mathbf{G}$ 和 $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^* \in \mathbf{T}_{\mathbf{q}_e}^* \mathbf{G}$ 。对于轨迹跟踪, 期望的总能量 $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ 是根据误差状态定义的, 并具有期望的闭环动力学。

$$\begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} = (\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix}. \quad (47)$$

将 (47) 与 (22) 匹配会得到对控制输入的以下要求:

$$\begin{aligned} \mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u} &= (\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \\ &\quad - (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix}. \end{aligned} \quad (48)$$

控制输入可以从 (48) 中获得, 作为能量塑形分量 $\mathbf{u}_{ES}$ 和阻尼注入分量 $\mathbf{u}_{DI}$ 的和 $\mathbf{u} = \mathbf{u}_{ES} + \mathbf{u}_{DI}$

$$\mathbf{u}_{ES} = \mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \left((\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} \right) \quad (49a)$$

$$- (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} \Bigg)$$

$$\mathbf{u}_{DI} = -\mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \quad (49b)$$

其中 $\mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) = (\mathcal{G}^\top(\mathbf{q}, \mathbf{p})\mathcal{G}(\mathbf{q}, \mathbf{p}))^{-1}\mathcal{G}^\top(\mathbf{q}, \mathbf{p})$ 是 $\mathcal{G}(\mathbf{q}, \mathbf{p})$ 的伪逆。当期望的互连矩阵 $\mathcal{J}_d$ 、耗散矩阵 $\mathcal{R}_d$ 和总能量 $\mathcal{H}_d$ 对所有 $(\mathbf{q}, \mathbf{p}) \in \mathbf{T}^*\mathbf{G}$ 以及 $(\mathbf{q}_e, \mathbf{p}_e) \in \mathbf{T}^*\mathbf{G}$ 满足以下匹配条件时, 控制输入 $\mathbf{u}_{ES}$ 存在:

$$\begin{aligned} \mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \left((\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \right. \\ \left. - (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} \right) = 0 \end{aligned} \quad (50)$$

其中 $\mathcal{G}^\perp(\mathbf{q}, \mathbf{p})$ 是 $\mathcal{G}(\mathbf{q}, \mathbf{p})$ 的最大秩左消去元, 即 $\mathcal{G}^\perp(\mathbf{q}, \mathbf{p})\mathcal{G}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$ 。

B. Tracking Control for Port-Hamiltonian Dynamics on the SE(3) Manifold

考虑系统应跟踪的期望状态轨迹 $\mathbf{x}^*(t) = (\mathbf{q}^*(t), \boldsymbol{\zeta}^*(t))$, 其中 $\mathbf{q}^*(t) \in \text{SE}(3)$ 是期望姿态, $\boldsymbol{\zeta}^*(t) = [\mathbf{v}^*(t)^\top \ \boldsymbol{\omega}^*(t)^\top]^\top$ 是以期望坐标系表示的期望广义速度。令 $\mathbf{p}^* = \mathbf{M} \begin{bmatrix} \mathbf{R}^\top \mathbf{R}^* \mathbf{v}^* \\ \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix}$ 表示期望动量, 基于 (26) 定义, 期望速度以机体坐标系表示。令 $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^*$ 和 $\mathbf{R}_e = \mathbf{R}^{*\top} \mathbf{R} = [\mathbf{r}_{e1} \ \mathbf{r}_{e2} \ \mathbf{r}_{e3}]^\top$ 分别为当前方向 $\mathbf{R}$ 与期望方向 $\mathbf{R}^*$ 之间的位置误差和旋转误差。广义坐标下的向量化误差 $\mathbf{q}_e$ 为

$$\mathbf{q}_e = [\mathbf{p}_e^{*\top} \ \mathbf{r}_{e1}^\top \ \mathbf{r}_{e2}^\top \ \mathbf{r}_{e3}^\top]^\top. \quad (51)$$

广义动量的误差为 $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^*$, 在机体坐标系中描述。期望的总能量以误差状态定义为

$$\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) = \frac{1}{2} \mathbf{p}_e^\top \mathbf{M}_d^{-1}(\mathbf{q}_e) \mathbf{p}_e + V_d(\mathbf{q}_e) \quad (52)$$

其中 $\mathbf{M}_d(\mathbf{q}_e)$ 和 $V_d(\mathbf{q}_e)$ 是期望的广义质量和势能。选择以下期望的互连矩阵和耗散矩阵:

$$\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) = \begin{bmatrix} \mathbf{0} & \mathbf{J}_1 \\ -\mathbf{J}_1^\top & \mathbf{J}_2 \end{bmatrix}, \quad \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e) = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_d \end{bmatrix} \quad (53)$$

并将(28)中的 $\mathcal{J}(\mathbf{q}, \mathbf{p})$ 和 $\mathcal{R}(\mathbf{q}, \mathbf{p})$ 代入(48)中的匹配方程, 会得到

$$\mathbf{0} = \mathbf{J}_1 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{q}} - \dot{\mathbf{q}}_e \quad (54)$$

$$\begin{aligned} \mathbf{B}(\mathbf{q})\mathbf{u} &= \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \\ &\quad - \mathbf{K}_d \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} + \mathbf{D}(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}_e. \end{aligned} \quad (55)$$

假设 $\mathbf{M}_d(\mathbf{q}_e) = \mathbf{M}(\mathbf{q})$, 如果我们选择 $\mathbf{J}_1 = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_{e1}^\top & \hat{\mathbf{r}}_{e2}^\top & \hat{\mathbf{r}}_{e3}^\top \end{bmatrix}$, 则 (54) 得以满足。事实上, 我们有 $\dot{\mathbf{q}} = \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}}$ [根据 (11) 和 (19)] 并且

$$\frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} = \mathbf{M}_d^{-1} \mathbf{p}_e = \begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \begin{bmatrix} \mathbf{v} - \mathbf{R}^\top \mathbf{R}^* \mathbf{v}^* \\ \boldsymbol{\omega} - \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix}.$$

误差动态变为

$$\dot{\mathbf{q}}_e = \begin{bmatrix} \dot{\mathbf{p}} - \dot{\mathbf{p}}^* \\ \dot{\mathbf{r}}_{e1} \\ \dot{\mathbf{r}}_{e2} \\ \dot{\mathbf{r}}_{e3} \end{bmatrix} = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_{e1}^\top & \hat{\mathbf{r}}_{e2}^\top & \hat{\mathbf{r}}_{e3}^\top \end{bmatrix}^\top \begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \mathbf{J}_1 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \quad (56)$$

since $\dot{\mathbf{R}}_e = \frac{d}{dt}(\mathbf{R}_e) = \mathbf{R}_e \hat{\omega}_e$ as shown in [79, Sec. III-A] and $\dot{\mathbf{p}} - \dot{\mathbf{p}}^* = \mathbf{R}\mathbf{v} - \mathbf{R}^*\mathbf{v}^* = \mathbf{R}\mathbf{v}_e$.

The desired control input can be obtained from (55) as $\mathbf{u} = \mathbf{u}_{ES} + \mathbf{u}_{DI}$ with

$$\mathbf{u}_{ES} = \mathbf{B}^\dagger(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \mathbf{D}(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}^* \right) \quad (57a)$$

$$\mathbf{u}_{DI} = -\mathbf{B}^\dagger(\mathbf{q}) \mathbf{K}_d \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \quad (57b)$$

where $\mathbf{B}^\dagger(\mathbf{q}) = (\mathbf{B}^\top(\mathbf{q})\mathbf{B}(\mathbf{q}))^{-1}\mathbf{B}^\top(\mathbf{q})$ is the pseudoinverse of $\mathbf{B}(\mathbf{q})$. The matching condition (50) becomes

$$\mathbf{B}^\perp(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}^* \right) = 0. \quad (58)$$

In this article, we reshape the open-loop Hamiltonian $\mathcal{H}(\mathbf{q}, \mathbf{p})$ into the following desired total energy $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$, minimized along the desired trajectory:

$$\begin{aligned} \mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) &= \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^\top \mathbf{K}_p(\mathbf{p} - \mathbf{p}^*) \\ &\quad + \frac{1}{2} \text{tr}(\mathbf{K}_R(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R})) \\ &\quad + \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^\top \mathbf{M}^{-1}(\mathbf{q})(\mathbf{p} - \mathbf{p}^*) \end{aligned} \quad (59)$$

where $\mathbf{K}_p, \mathbf{K}_R \succ 0$ are positive-definite matrices. For an SE(3) rigid-body system with constant generalized mass matrix $\mathbf{M}_d = \mathbf{M}$ and $\mathbf{J}_2 = 0$, which is a common choice, the energy-shaping term in (57a) and the damping-injection term in (57b) simplify as

$$\begin{aligned} \mathbf{u}_{ES}(\mathbf{q}, \mathbf{p}) &= \mathbf{B}^\dagger(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial V}{\partial \mathbf{q}} - (\mathbf{p}^\times - \mathbf{D}(\mathbf{q}, \mathbf{p})) \mathbf{M}^{-1} \mathbf{p} - \mathbf{e}(\mathbf{q}, \mathbf{q}^*) + \dot{\mathbf{p}}^* \right) \\ \mathbf{u}_{DI}(\mathbf{q}, \mathbf{p}) &= -\mathbf{B}^\dagger(\mathbf{q}) \mathbf{K}_d \mathbf{M}^{-1}(\mathbf{p} - \mathbf{p}^*) \end{aligned} \quad (60)$$

where the generalized coordinate error between $\mathbf{q}$ and $\mathbf{q}^*$ is

$$\mathbf{e}(\mathbf{q}, \mathbf{q}^*) := \mathbf{J}_1^\top \frac{\partial V_d}{\partial \mathbf{q}_e} = \left[\begin{array}{c} \mathbf{R}^\top \mathbf{K}_p(\mathbf{p} - \mathbf{p}^*) \\ \frac{1}{2}(\mathbf{K}_R \mathbf{R}^{*\top} \mathbf{R} - \mathbf{R}^\top \mathbf{R}^* \mathbf{K}_R^\top)^\vee \end{array} \right] \quad (61)$$

and the derivative of the desired momentum is

$$\dot{\mathbf{p}}^* = \mathbf{M} \left[\begin{array}{c} \mathbf{R}^\top \dot{\mathbf{p}}^* - \hat{\omega} \mathbf{R}^\top \mathbf{p}^* \\ \mathbf{R}^\top \mathbf{R}^* \dot{\omega}^* - \hat{\omega}_e \mathbf{R}^\top \mathbf{R}^* \omega^* \end{array} \right]. \quad (62)$$

By expanding the terms in (60), we have

$$\mathbf{p}^\times \mathbf{M}^{-1} \mathbf{p} = \mathbf{p}^\times \boldsymbol{\zeta} = \left[\begin{array}{c} \hat{\mathbf{p}}_v \omega \\ \hat{\mathbf{p}}_\omega \omega + \hat{\mathbf{p}}_v \mathbf{v} \end{array} \right] \quad (63)$$

$$\mathbf{M}^{-1}(\mathbf{p} - \mathbf{p}^*) = \left[\begin{array}{c} \mathbf{v} - \mathbf{R}^\top \dot{\mathbf{p}}^* \\ \omega - \mathbf{R}^\top \mathbf{R}^* \omega^* \end{array} \right] \quad (64)$$

$$\mathbf{q}^{\times\top} \frac{\partial \mathcal{V}}{\partial \mathbf{q}} = \left[\begin{array}{c} \mathbf{R}^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{p}} \\ \sum_{i=1}^3 \hat{\mathbf{r}}_i^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{r}_i} \end{array} \right]. \quad (65)$$

Theorem 1: Consider a port-Hamiltonian system on the SE(3) manifold with dynamics (30). Assume that the matching condition (58) is satisfied, the desired momentum's derivative $\dot{\mathbf{p}}^*$ is bounded, and the matrices $\mathbf{K}_p$, $\mathbf{K}_R$, and $\mathbf{K}_d$ are positive definite. The control policy in (57a) leads to closed-loop error dynamics in (47) and (53). The tracking error $(\mathbf{q}_e, \mathbf{p}_e) = ((\mathbf{p}_e, \mathbf{R}_e), \mathbf{p}_e)$ asymptotically stabilizes to $((\mathbf{0}, \mathbf{I}), \mathbf{0})$ with Lyapunov function given by the desired Hamiltonian $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ in (59).

Proof: Since the matching condition is satisfied and the desired momentum's derivative $\dot{\mathbf{p}}^*$ is bounded, the control policy in (57a) and (57b) exists and achieves the desired closed-loop error dynamics as follows :

$$\left[\begin{array}{c} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{array} \right] = \left[\begin{array}{cc} \mathbf{0} & \mathbf{J}_1 \\ -\mathbf{J}_1^\top & \mathbf{J}_2 - \mathbf{K}_d \end{array} \right] \left[\begin{array}{c} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{array} \right]. \quad (66)$$

We have $\text{tr}(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R}) \geq 0$, as all entries in $\mathbf{R}_e \in SO(3)$ are less than 1. Since $\mathbf{M}$, $\mathbf{K}_p$, and $\mathbf{K}_R$ are positive-definite matrices, the desired Hamiltonian $\mathcal{H}_d$ is positive-definite, and achieves minimum value 0 only at $\mathbf{q}_e = (\mathbf{0}, \mathbf{I})$ and $\mathbf{p}_e = \mathbf{0}$, i.e., no position, rotation, and momentum errors. The time derivative of $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ can be computed as

$$\begin{aligned} \dot{\mathcal{H}}_d(\mathbf{q}_e, \mathbf{p}_e) &= \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e}^\top \dot{\mathbf{q}}_e + \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e}^\top \dot{\mathbf{p}}_e \\ &= -\mathbf{p}_e^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{K}_d \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p}_e. \end{aligned} \quad (67)$$

As $\mathbf{K}_d$ and $\mathbf{M}(\mathbf{q})$ are positive-definite, we have $\dot{\mathcal{H}}_d(\mathbf{q}_e, \mathbf{p}_e) \leq 0$ for all $(\mathbf{q}_e, \mathbf{p}_e)$ and equality holds at $((\mathbf{0}, \mathbf{I}), \mathbf{0})$. By LaSalle's invariance principle [80], the tracking errors $(\mathbf{q}_e, \mathbf{p}_e)$ asymptotically converge to $((\mathbf{0}, \mathbf{I}), \mathbf{0})$. $\square$

Without requiring a priori knowledge of the system parameters, the control design in (57a) offers a unified control approach for SE(3) Hamiltonian systems that achieves trajectory tracking, if permissible by the system's degree of underactuation. Thus, our control design solves Problem 2 for rigid-body robot systems, such as UGVs, UAVs, and UUVs, with tracking performance guaranteed by Theorem 1.

VII. EVALUATION

We verify the effectiveness of our port-Hamiltonian neural ODE network for dynamics learning and control on matrix Lie groups using a simulated pendulum, a simulated Crazyflie quadrotor, and a real PX4 quadrotor platform, whose states evolve on the SE(3) manifold. The implementation details for the experiments are provided in Appendix A.

自 $\dot{\mathbf{R}}_e = \frac{d}{dt}(\mathbf{R}_e) = \mathbf{R}_e \hat{\boldsymbol{\omega}}_e$ 如 [79, 第三节-A] 和 $\dot{\mathbf{p}} - \dot{\mathbf{p}}^* = \mathbf{R}\mathbf{v} - \mathbf{R}^*\mathbf{v}^* = \mathbf{R}\mathbf{v}_e$ 所示。

所需的控制输入可以从 (55) 中获得为 $\mathbf{u} = \mathbf{u}_{ES} + \mathbf{u}_{DI}$, 并且

$$\mathbf{u}_{ES} = \mathbf{B}^\dagger(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \mathbf{D}(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}_e \right) \quad (57a)$$

$$\mathbf{u}_{DI} = -\mathbf{B}^\dagger(\mathbf{q}) \mathbf{K}_d \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \quad (57b)$$

其中 $\mathbf{B}^\dagger(\mathbf{q}) = (\mathbf{B}^\top(\mathbf{q})\mathbf{B}(\mathbf{q}))^{-1}\mathbf{B}^\top(\mathbf{q})$ 是 $\mathbf{B}(\mathbf{q})$ 的伪逆。匹配条件 (50) 变为

$$\mathbf{B}^\perp(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}_e \right) = 0. \quad (58)$$

在本文中, 我们将开环哈密顿量 $\mathcal{H}(\mathbf{q}, \mathbf{p})$ 重塑为以下期望的总能量 $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$, 沿期望轨迹最小化:

$$\begin{aligned} \mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) &= \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^\top \mathbf{K}_p(\mathbf{p} - \mathbf{p}^*) \\ &\quad + \frac{1}{2} \text{tr}(\mathbf{K}_R(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R})) \\ &\quad + \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^\top \mathbf{M}^{-1}(\mathbf{q})(\mathbf{p} - \mathbf{p}^*) \end{aligned} \quad (59)$$

其中 $\mathbf{K}_p, \mathbf{K}_R \succ 0$ 是正定矩阵。对于具有恒定广义质量矩阵 $\mathbf{M}_d = \mathbf{M}$ 和 $\mathbf{J}_2 = 0$ 的 SE(3) 刚体系统, 这是一种常见选择, (57a) 中的能量塑形项和 (57b) 中的阻尼注入项简化为

$$\begin{aligned} \mathbf{u}_{ES}(\mathbf{q}, \mathbf{p}) &= \mathbf{B}^\dagger(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial V}{\partial \mathbf{q}} - (\mathbf{p}^\times - \mathbf{D}(\mathbf{q}, \mathbf{p})) \mathbf{M}^{-1} \mathbf{p} - \mathbf{e}(\mathbf{q}, \mathbf{q}^*) + \dot{\mathbf{p}}^* \right) \\ \mathbf{u}_{DI}(\mathbf{q}, \mathbf{p}) &= -\mathbf{B}^\dagger(\mathbf{q}) \mathbf{K}_d \mathbf{M}^{-1}(\mathbf{p} - \mathbf{p}^*) \end{aligned} \quad (60)$$

$\mathbf{q}$ 与 $\mathbf{q}^*$ 之间的广义坐标误差为

$$\mathbf{e}(\mathbf{q}, \mathbf{q}^*) := \mathbf{J}_1^\top \frac{\partial V_d}{\partial \mathbf{q}_e} = \begin{bmatrix} \mathbf{R}^\top \mathbf{K}_p(\mathbf{p} - \mathbf{p}^*) \\ \frac{1}{2}(\mathbf{K}_R \mathbf{R}^{*\top} \mathbf{R} - \mathbf{R}^\top \mathbf{R}^* \mathbf{K}_R)^\vee \end{bmatrix} \quad (61)$$

所需动量的导数是

$$\dot{\mathbf{p}}^* = \mathbf{M} \begin{bmatrix} \mathbf{R}^\top \dot{\mathbf{p}}^* - \hat{\boldsymbol{\omega}} \mathbf{R}^\top \dot{\mathbf{p}}^* \\ \mathbf{R}^\top \mathbf{R}^* \dot{\boldsymbol{\omega}}^* - \hat{\boldsymbol{\omega}}_e \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix}. \quad (62)$$

通过展开 (60) 中的各项, 我们得到

$$\mathbf{p}^\times \mathbf{M}^{-1} \mathbf{p} = \mathbf{p}^\times \boldsymbol{\zeta} = \begin{bmatrix} \hat{\mathbf{p}}_v \boldsymbol{\omega} \\ \hat{\mathbf{p}}_\omega \boldsymbol{\omega} + \hat{\mathbf{p}}_v \mathbf{v} \end{bmatrix} \quad (63)$$

$$\mathbf{M}^{-1}(\mathbf{p} - \mathbf{p}^*) = \begin{bmatrix} \mathbf{v} - \mathbf{R}^\top \dot{\mathbf{p}}^* \\ \boldsymbol{\omega} - \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix} \quad (64)$$

$$\mathbf{q}^{\times\top} \frac{\partial \mathcal{V}}{\partial \mathbf{q}} = \begin{bmatrix} \mathbf{R}^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{p}} \\ \sum_{i=1}^3 \hat{\mathbf{r}}_i^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{r}_i} \end{bmatrix}. \quad (65)$$

Theorem 1: 考虑一个定义在 SE(3) 流形上的端口-哈密顿系统, 其动力学为 (30)。假设匹配条件 (58) 得到满足, 期望动量的导数 $\dot{\mathbf{p}}^*$ 有界, 并且矩阵 $\mathbf{K}_p$ 、 $\mathbf{K}_R$ 和 $\mathbf{K}_d$ 为正定。(57a) 中的控制策略导致闭环误差动力学为 (47) 和 (53)。跟踪误差 $(\mathbf{q}_e, \mathbf{p}_e) = ((\mathbf{p}_e, \mathbf{R}_e), \mathbf{p}_e)$ 渐近稳定于 $((0, \mathbf{I}), 0)$, 其 Lyapunov 函数由期望的哈密顿量 $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ 在 (59) 中给出。

Proof: 由于匹配条件得到满足且所需动量的导数 $\dot{\mathbf{p}}^*$ 有界, (57a) 和 (57b) 中的控制策略存在, 并实现如下所示的期望闭环误差动力学:

$$\begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{J}_1 \\ -\mathbf{J}_1^\top & \mathbf{J}_2 - \mathbf{K}_d \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix}. \quad (66)$$

我们有 $\text{tr}(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R}) \geq 0$, 因为 $\mathbf{R}_e \in SO(3)$ 中的所有条目都小于 1。由于 $\mathbf{M}$ 、 $\mathbf{K}_p$ 和 $\mathbf{K}_R$ 是正定矩阵, 所需的哈密顿量 $\mathcal{H}_d$ 是正定的, 并且仅在 $\mathbf{q}_e = (0, \mathbf{I})$ 和 $\mathbf{p}_e = 0$ 时取得最小值 0, 即没有位置、旋转和动量误差。 $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ 的时间导数可以计算为

$$\begin{aligned} \dot{\mathcal{H}}_d(\mathbf{q}_e, \mathbf{p}_e) &= \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e}^\top \dot{\mathbf{q}}_e + \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e}^\top \dot{\mathbf{p}}_e \\ &= -\mathbf{p}_e^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{K}_d \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p}_e. \end{aligned} \quad (67)$$

由于 $\mathbf{K}_d$ 和 $\mathbf{M}(\mathbf{q})$ 是正定的, 我们有 $\dot{\mathcal{H}}_d(\mathbf{q}_e, \mathbf{p}_e) \leq 0$ 对于所有 $(\mathbf{q}_e, \mathbf{p}_e)$ 并且在 $((0, \mathbf{I}), 0)$ 处取等号。根据 LaSalle 不变性原理 [80], 跟踪误差 $(\mathbf{q}_e, \mathbf{p}_e)$ 渐近地收敛到 $((0, \mathbf{I}), 0)$ 。 $\square$

在不需要事先了解系统参数的情况下, 公式 (57a) 中的控制设计为 SE(3) 哈密顿系统提供了一种统一的控制方法, 在系统的欠驱动程度允许的情况下, 实现轨迹跟踪。因此, 我们的控制设计解决了刚体机器人系统 (如 UGV、UAV 和 UUV) 的第 2 问题, 其跟踪性能由定理 1 保证。

七、评估

我们使用一个模拟摆、一个模拟 Crazyflie 四旋翼和一个真实的 PX4 四旋翼平台验证了我们的端口-哈密顿神经网络在矩阵李群上的动力学学习和控制的有效性, 这些平台的状态在 SE(3) 流形上演化。实验的实现细节见附录 A。

A. Pendulum

In this section, we verify our port-Hamiltonian dynamics learning and control approach on the $SO(3)$ manifold. We consider a pendulum with the following dynamics:

$$\ddot{\varphi} = -15 \sin \varphi + 3u - 0.2\dot{\varphi} \quad (68)$$

where φ is the angle of the pendulum with respect to its vertically downward position, and u is a scalar control input. The ground-truth mass, potential energy, friction coefficient, and the input gain are: $m = 1/3$, $\mathcal{V}(\varphi) = 5(1 - \cos \varphi)$, $D(\varphi) = 0.2/3$, and $B(\varphi) = 1$, respectively. We collected data of the form $\{(\cos \varphi, \sin \varphi, \dot{\varphi})\}$ from an OpenAI Gym environment, provided by [27], with the dynamics in (68). To illustrate our Lie group neural ODE learning, we represent the angle φ as a rotation matrix

$$\mathbf{R} = \begin{bmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (69)$$

representing the pendulum orientation. We let $\boldsymbol{\omega} = [0, 0, \dot{\varphi}]$ and remove position $\mathbf{p}$ and linear velocity $\mathbf{v}$ from the Hamiltonian dynamics in (30), restricting the system to the $SO(3)$ manifold with generalized coordinates $\mathbf{q} = [\mathbf{r}_1^\top \ \mathbf{r}_2^\top \ \mathbf{r}_3^\top]^\top$.

As described in Section V-D, control inputs $\mathbf{u}^{(i)}$ were sampled randomly and applied to the pendulum for five time intervals of $0.05s$, forming a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{q}_{0:N}^{(i)}, \boldsymbol{\omega}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ with $N = 5$ and $D = 5120$. We trained an $SO(3)$ port-Hamiltonian neural ODE network as described in Section V-D for 5000 iterations without any nominal model, i.e., $\mathbf{M}_{\omega_0}^{-1}(\mathbf{q}) = \mathbf{0}$, $\mathbf{D}_{\omega_0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $V_0(\mathbf{q}) = 0$, and $\mathbf{B}_0(\mathbf{q}) = \mathbf{0}$.

As noted in [27] and [32], since the generalized momenta $\mathbf{p}$ are not available in the dataset, the dynamics of $\mathbf{q}$ in (68) do not change if $\mathbf{p}$ is scaled by a factor $\beta > 0$. This is also true in our formulation as scaling $\mathbf{p}$ leaves the dynamics of $\mathbf{q}$ in (30) unchanged. To emphasize this scale invariance, let $\mathbf{M}_\beta(\mathbf{q}) = \beta \mathbf{M}(\mathbf{q})$, $\mathcal{V}_\beta(\mathbf{q}) = \beta \mathcal{V}(\mathbf{q})$, $\mathbf{D}_\beta(\mathbf{q}, \mathbf{p}) = \beta \mathbf{D}(\mathbf{q}, \mathbf{p})$, $\mathbf{B}_\beta(\mathbf{q}) = \beta \mathbf{B}(\mathbf{q})$, and

$$\begin{aligned} \mathbf{p}_\beta &= \mathbf{M}_\beta(\mathbf{q})\boldsymbol{\omega} = \beta \mathbf{p}, & \dot{\mathbf{p}}_\beta &= \beta \dot{\mathbf{p}} \\ \mathcal{H}_\beta(\mathbf{q}, \mathbf{p}) &= \frac{1}{2} \mathbf{p}_\beta^\top \mathbf{M}_\beta^{-1}(\mathbf{q}) \mathbf{p}_\beta + V_\beta(\mathbf{q}) = \beta \mathcal{H}(\mathbf{q}, \mathbf{p}) \\ \frac{\partial \mathcal{H}_\beta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\beta} &= \mathbf{M}_\beta^{-1}(\mathbf{q}) \mathbf{p}_\beta = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \end{aligned} \quad (70)$$

guaranteeing that the equations of motions (30) still hold.

Fig. 3 shows the training and testing behavior of our $SO(3)$ Hamiltonian ODE network. Fig. 3(a) and (c) shows that the $[\mathbf{M}(\mathbf{q})^{-1}]_{3,3}$ entry of the mass inverse and the $[\mathbf{B}(\mathbf{q})]_3$ entry of the input matrix with scaling factor $\beta = 1.33$ are close to their correct values of 3 and 1, respectively, while the other entries are close to zero. Fig. 3(b) indicates a constant gap between the learned and the ground-truth potential energy, which can be explained by the relativity of potential energy.

We tested stabilization of the pendulum based on the learned dynamics to the unstable equilibrium at the upward position $\varphi = \pi$, with zero velocity. Since the pendulum is a fully actuated

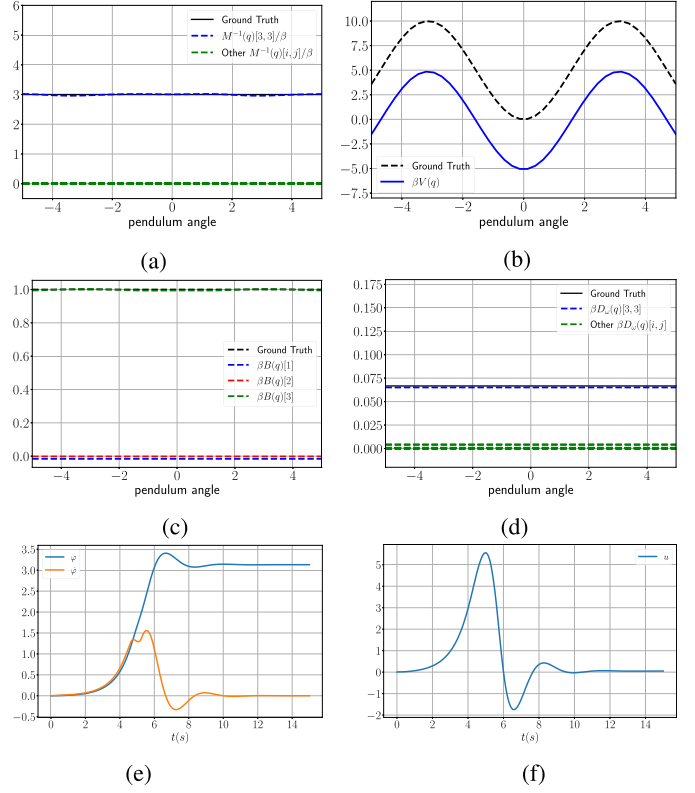


Fig. 3. Pendulum dynamics estimation using an $SO(3)$ port-Hamiltonian neural ODE network with scale factor $\beta = 1.33$. (a) $\mathbf{M}^{-1}(\mathbf{q})/\beta$ versus φ . (b) $\beta \mathcal{V}(\mathbf{q})$ versus φ . (c) $\beta \mathbf{B}(\mathbf{q})$ versus φ . (d) $\beta \mathbf{D}_\omega(\mathbf{q})$ versus φ . (e) Angle φ and velocity $\dot{\varphi}$. (f) Control input u .

system, the energy-based controller in (60) exists and is obtained by removing the position error from the desired energy

$$\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) = \frac{1}{2} \text{tr}(\mathbf{K}_R(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R})) + \frac{1}{2} \mathbf{p}_\omega^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p}_\omega. \quad (71)$$

The controlled angle φ and angular velocity $\dot{\varphi}$ as well as the control inputs $\mathbf{u}$ with gains $\mathbf{K}_R = 2\mathbf{I}$ and $\mathbf{K}_d = \mathbf{I}$ are shown over time in Fig. 3(e) and (f). We can see that the controller was able to smoothly drive the pendulum from $\phi = 0$ to $\phi = \pi$, relying only on the learned dynamics.

B. Omnidirectional Ground Vehicle

In this section, we verify our port-Hamiltonian dynamics learning and control approach on a simulated omnidirectional ground vehicle, whose states evolve on the $SE(2)$ manifold. The ground-truth dynamics of the vehicle can be obtained from (30) by keeping only the components x and y in the position $\mathbf{p}$ and the yaw angle of the rotation matrix, leading to an $SO(2)$ rotation matrix

$$\mathbf{R} = \begin{bmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{bmatrix} \quad (72)$$

where φ is the vehicle's yaw angle. The vehicle moves on a flat ground with potential energy $\mathcal{V}(\mathbf{q}) = c$, where c is a

A. Pendulum

在本节中，我们在 $SO(3)$ 流形上验证我们的端口-哈密顿动力学学习和控制方法。我们考虑一个具有以下动力学的摆：

$$\ddot{\varphi} = -15 \sin \varphi + 3u - 0.2\dot{\varphi} \quad (68)$$

其中 φ 是摆相对于垂直向下位置的角度， u 是标量控制输入。真实质量、势能、摩擦系数和输入增益分别为： $m = 1/3$ 、 $\mathcal{V}(\varphi) = 5(1 - \cos \varphi)$ 、 $D(\varphi) = 0.2/3$ 和 $B(\varphi) = 1$ 。我们由 [27] 提供的 OpenAI Gym 环境中收集了如下形式的数据： $\{(\cos \varphi, \sin \varphi, \dot{\varphi})\}$ ，其动力学如公式 (68) 所示。为了说明我们的李群神经 ODE 学习，我们将角度 φ 表示为旋转矩阵。

$$\mathbf{R} = \begin{bmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (69)$$

表示摆的方向。我们令 $\boldsymbol{\omega} = [0, 0, \dot{\varphi}]$ 并从 (30) 中的哈密顿动力学中去掉位置 $\mathbf{p}$ 和线速度 $\mathbf{v}$ ，将系统限制在带有广义坐标 $\mathbf{q} = [\mathbf{r}_1^\top \ \mathbf{r}_2^\top \ \mathbf{r}_3^\top]^\top$ 的 $SO(3)$ 流形上。

如第 V-D 节所述，控制输入 $\mathbf{u}^{(i)}$ 是随机采样的，并在摆上应用了五个 $0.05s$ 的时间间隔，形成了一个数据集 $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{q}_{0:N}^{(i)}, \boldsymbol{\omega}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ ，其中 $N = 5$ ， $D = 5120$ 。我们按照第 V-D 节的描述训练了一个 $SO(3)$ port-Hamiltonian 神经 ODE 网络，进行了 5000 次迭代，没有任何名义模型，即 $\mathbf{M}_{\omega_0}^{-1}(\mathbf{q}) = \mathbf{0}$ 、 $\mathbf{D}_{\omega_0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$ 、 $V_0(\mathbf{q}) = \mathbf{0}$ 和 $\mathbf{B}_0(\mathbf{q}) = \mathbf{0}$ 。

如 [27] 和 [32] 所述，由于数据集中没有广义动量 $\mathbf{p}$ ，若 $\mathbf{p}$ 按因子 $\beta > 0$ 缩放，(68) 中 $\mathbf{q}$ 的动力学不会发生变化。在我们的公式中也是如此，因为缩放 $\mathbf{p}$ 不会改变 (30) 中 $\mathbf{q}$ 的动力学。为了强调这种尺度不变性，设 $\mathbf{M}_\beta(\mathbf{q}) = \beta\mathbf{M}(\mathbf{q})$ 、 $\mathcal{V}_\beta(\mathbf{q}) = \beta\mathcal{V}(\mathbf{q})$ 、 $\mathbf{D}_\beta(\mathbf{q}, \mathbf{p}) = \beta\mathbf{D}(\mathbf{q}, \mathbf{p})$ 、 $\mathbf{B}_\beta(\mathbf{q}) = \beta\mathbf{B}(\mathbf{q})$ ，以及

$$\begin{aligned} \mathbf{p}_\beta &= \mathbf{M}_\beta(\mathbf{q})\boldsymbol{\omega} = \beta\mathbf{p}, & \dot{\mathbf{p}}_\beta &= \beta\dot{\mathbf{p}} \\ \mathcal{H}_\beta(\mathbf{q}, \mathbf{p}) &= \frac{1}{2}\mathbf{p}_\beta^\top \mathbf{M}_\beta^{-1}(\mathbf{q})\mathbf{p}_\beta + V_\beta(\mathbf{q}) = \beta\mathcal{H}(\mathbf{q}, \mathbf{p}) \\ \frac{\partial \mathcal{H}_\beta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\beta} &= \mathbf{M}_\beta^{-1}(\mathbf{q})\mathbf{p}_\beta = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \end{aligned} \quad (70)$$

保证运动方程 (30) 仍然成立。

图3显示了我们 $SO(3)$ Hamiltonian ODE 网络的训练和测试行为。图3(a)和(c)显示，质量逆矩阵的 $[\mathbf{M}(\mathbf{q})^{-1}]_{3,3}$ 项以及输入矩阵的 $[\mathbf{B}(\mathbf{q})]_{3,3}$ 项，缩放因子为 $\beta = 1.33$ ，分别接近它们的正确值3和1，而其他项接近零。图3(b)显示了学习到的势能与真实势能之间存在恒定差距，这可以用势能的相对性来解释。

我们基于学习到的动力学测试了钟摆在向上位置 $\varphi = \pi$ 的不稳定平衡的稳定性，初速度为零。由于钟摆是完全驱动的

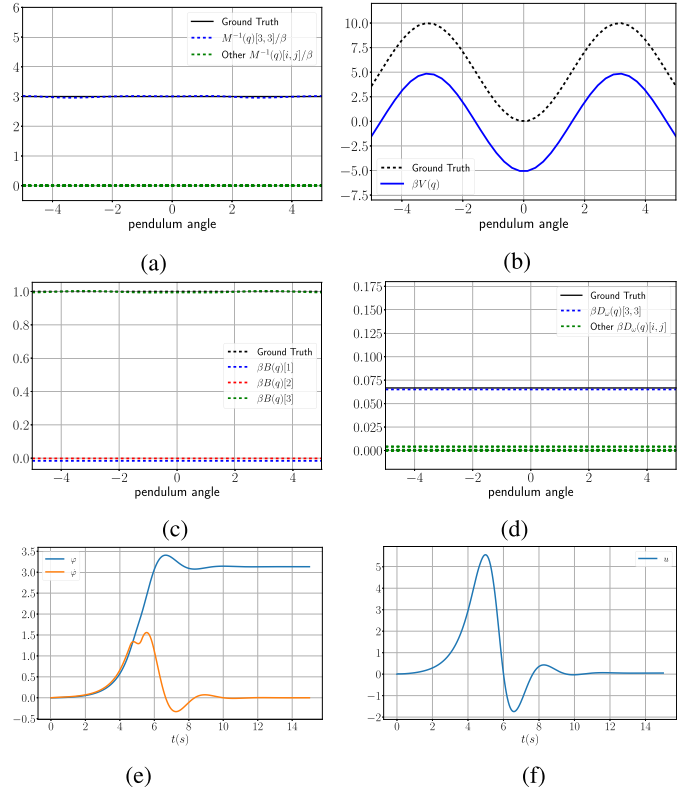


图3. 使用具有尺度因子 $\beta = 1.33$ 的 $SO(3)$ port-Hamiltonian 神经 ODE 网络进行摆的动力学估计。(a) $\mathbf{M}^{-1}(\mathbf{q})/\beta$ 对 φ 。(b) $\beta\mathcal{V}(\mathbf{q})$ 对 φ 。(c) $\beta\mathbf{B}(\mathbf{q})$ 对 φ 。(d) $\beta\mathbf{D}_{\omega}(\mathbf{q})$ 对 φ 。(e) 角度 φ 和速度 $\dot{\varphi}$ 。(f) 控制输入 u 。

系统中，式(60)中的基于能量的控制器存在，并且是通过从期望能量中去除位置误差得到的

$$\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) = \frac{1}{2}\text{tr}(\mathbf{K}_R(\mathbf{I} - \mathbf{R}^{*\top}\mathbf{R})) + \frac{1}{2}\mathbf{p}_\omega^\top \mathbf{M}^{-1}(\mathbf{q})\mathbf{p}_\omega. \quad (71)$$

受控角度 φ 和角速度 $\dot{\varphi}$ 以及带有增益 $\mathbf{K}_R = 2\mathbf{I}$ 和 $\mathbf{K}_d = \mathbf{I}$ 的控制输入 $\mathbf{u}$ 随时间的变化如图 3(e) 和 (f) 所示。我们可以看到，控制器能够平稳地将摆从 $\phi = 0$ 驱动到 $\phi = \pi$ ，仅依靠学习到的动力学。

B. Omnidirectional Ground Vehicle

在本节中，我们在一个模拟的全向地面车辆上验证我们的端口-哈密顿动力学学习与控制方法，该车辆的状态在 $SE(2)$ 流形上演化。车辆的真实动力学可以通过保留位置 $\mathbf{p}$ 和旋转矩阵的偏航角中的分量 x 和 y ，从公式 (30) 中得到，从而得到一个 $SO(2)$ 旋转矩阵。

$$\mathbf{R} = \begin{bmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{bmatrix} \quad (72)$$

其中 φ 是车辆的偏航角。车辆在具有势能 $\mathcal{V}(\mathbf{q}) = c$ 的平地上运动，其中 c 是一个

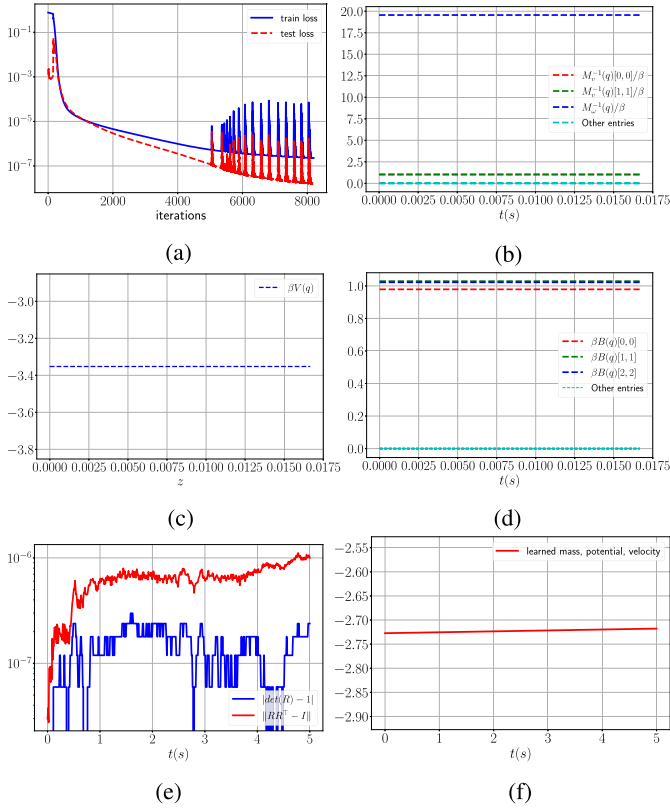


Fig. 4. Evaluation of SE(2) port-Hamiltonian neural ODE network on a simulated omnidirectional ground vehicle with scale factor $\beta = 7.1$. (a) Loss (log scale). (b) $M_v^{-1}(q)$ and $M_\omega^{-1}(q)$. (c) $V(q)$. (d) $B_v(q)$. (e) SO(3) constraints. (f) Total energy.

constant, and has ground-truth mass $M_v(q) = I \in \mathbb{S}_{>0}^{2 \times 2}$ and inertia $M_\omega(q) = 0.05 \in \mathbb{R}_{>0}$. It is fully actuated with control input $u = [f_x, f_y, \tau_\varphi]$, where f_x and f_y are forces along the x - and y -axes of the body frame and τ_φ is the yaw torque, generated by the motors.

To collect training data, the vehicle was controlled from a random initial point to nine different desired positions and yaw angles using a PID controller, providing nine one-second trajectories. The trajectories were used to generate a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, q_{0:N}^{(i)}, \zeta_{0:N}^{(i)}, u^{(i)}\}_{i=1}^D$ with $N = 5$ and $D = 432$. The SE(2) port-Hamiltonian ODE network was formulated, as described in Section V-D, by ignoring the z component of the position p , and the pitch and roll components of the rotation R . The model was trained for 8000 iterations without a nominal model, i.e., $M_{v0}^{-1}(q) = 0$, $M_{\omega0}^{-1}(q) = 0$, $D_{v0}(q, p) = 0$, $D_{\omega0}(q, p) = 0$, $V_0(q) = 0$, and $B_0(q) = 0$. We did not consider energy dissipation such as friction in the simulation, and we omitted the dissipation matrix $D_\theta(q, p)$ in the model.

Fig. 4 shows the training results for the SE(2) Hamiltonian ODE network. Fig. 4(b) and (d) shows that the mass inverse and the input gain matrix with scaling factor $\beta = 7.1$ are close to their correct values: $M_v(q)^{-1} \approx I$, $M_\omega(q)^{-1} \approx 20$ and $B(q) \approx I$. Fig. 4(c) shows a constant learned potential energy as expected.

We verified our energy-based control design in Section VI by controlling the ground robot to track horizontal lemniscate

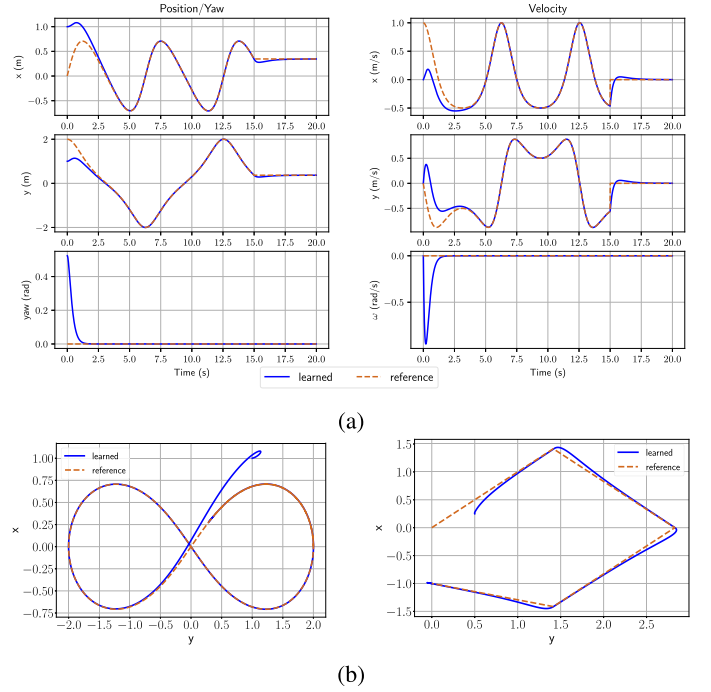


Fig. 5. Trajectories (blue) of omnidirectional ground vehicle tracking desired trajectories (orange) with our learned SE(2) port-Hamiltonian dynamics and IDA-PBC control design. (a) Tracking performance with a lemniscate trajectory. (b) Tracking lemniscate (left) and piecewise-linear (right) trajectories.

and piecewise-linear trajectories. Fig. 5(b) demonstrates that the omnidirectional ground vehicle controlled by our energy-based controller achieves successful trajectory tracking. The control gains were chosen as: $K_p = 0.72I$, $K_v = 0.8I$, $K_R = 9.1I$, and $K_\omega = 3.6$. Fig. 5(a) plots the tracking errors in the position, yaw angles, linear velocity, and angular velocity.

C. Crazyflie Quadrotor

In this section, we demonstrate that our SE(3) dynamics learning and control approach can achieve trajectory tracking for an underactuated system. We consider a Crazyflie quadrotor, shown in Fig. 8(a), simulated in the physics-based simulator PyBullet [81]. The control input $u = [f, \tau]$ includes a thrust $f \in \mathbb{R}_{\geq 0}$ and a torque vector $\tau \in \mathbb{R}^3$ generated by the four rotors. The generalized coordinates and velocity are $q = [p^\top, r_1^\top, r_2^\top, r_3^\top]^\top$ and $\zeta = [v^\top, \omega^\top]^\top$ as before.

The quadrotor was controlled from a random starting point to 18 different desired poses using a PID controller [81], providing 18 2.5-s trajectories. The trajectories were used to generate a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, q_{0:N}^{(i)}, \zeta_{0:N}^{(i)}, u^{(i)}\}_{i=1}^D$ with $N = 5$ and $D = 1080$. The SE(3) port-Hamiltonian ODE network was trained, as described in Section V-D, for 500 iterations without a nominal model, i.e., $M_{v0}^{-1}(q) = 0$, $M_{\omega0}^{-1}(q) = 0$, $D_{v0}(q, p) = 0$, $D_{\omega0}(q, p) = 0$, $V_0(q) = 0$, and $B_0(q) = 0$. We did not consider energy dissipation such as drag effect in the PyBullet simulator, and we omitted the dissipation matrix $D_\theta(q, p)$ in the model design.

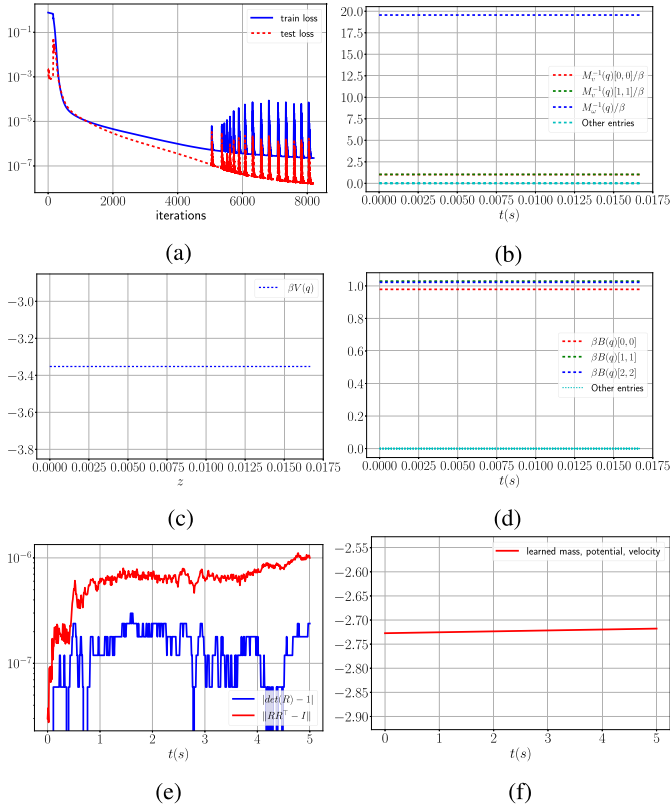


图 4. 在具有缩放因子 $\beta = 7.1$ 的模拟全向地面车辆上对 SE(2) 端口-哈密顿神经 ODE 网络的评估。(a) 损失 (对数刻度)。(b) $M_v^{-1}(q)$ 和 $M_\omega^{-1}(q)$ 。(c) $V(q)$ 。(d) $B_v(q)$ 。(e) SO(3) 约束。(f) 总能量。

恒定, 并且具有真实质量 $M_v(q) = I \in S_{>0}^{2 \times 2}$ 和惯性 $M_\omega(q) = 0.05 \in \mathbb{R}_{>0}$ 。它是全驱动的, 控制输入为 $u = [f_x, f_y, \tau_\varphi]$, 其中 f_x 和 f_y 是沿机体坐标系的 x 和 y 轴的力, τ_φ 是由电机产生的偏航力矩。

为了收集训练数据, 车辆从随机初始点被控制到九个不同的期望位置和偏航角, 使用 PID 控制器, 提供了九条一秒的轨迹。这些轨迹被用来生成一个数据集 $\mathcal{D} = \{t_{0:N}^{(i)}, q_{0:N}^{(i)}, \zeta_{0:N}^{(i)}, u^{(i)}\}_{i=1}^D$, 其中包含 $N = 5$ 和 $D = 432$ 。如第五节 D 部分所述, SE(2) 端口哈密顿常微分方程网络被构建, 通过忽略位置 p 的 z 分量, 以及旋转 R 的俯仰和滚转分量。模型在没有标称模型的情况下训练了 8000 次迭代, 即 $M_{v0}^{-1}(q) = 0$, $M_{\omega0}^{-1}(q) = 0$, $D_{v0}(q, p) = 0$, $D_{\omega0}(q, p) = 0$, $V_0(q) = 0$ 和 $B_0(q) = 0$ 。我们在模拟中没有考虑摩擦等能量损耗, 并且在模型中省略了耗散矩阵 $D_\theta(q, p)$ 。

图4显示了SE(2)哈密顿ODE网络的训练结果。图4(b)和(d)显示, 带有缩放因子 $\beta = 7.1$ 的质量逆矩阵和输入增益矩阵接近其正确值: $M_v(q)^{-1} \approx I$, $M_\omega(q)^{-1} \approx 20$ 和 $B(q) \approx I$ 。图4(c)显示了一个如预期的常数学习势能。

我们在第六节通过控制地面机器人追踪水平勒蒙斯刻曲线验证了我们的基于能量的控制设计

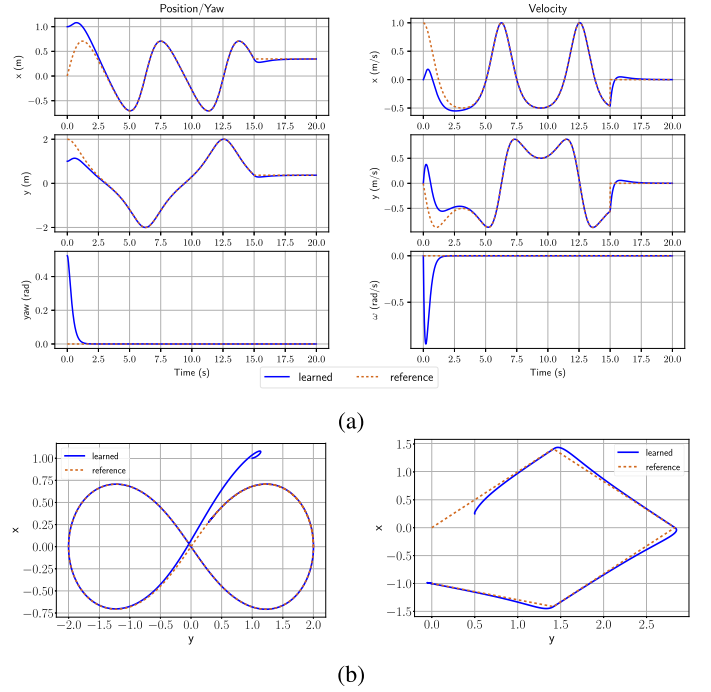


图5. 全向地面车辆使用我们学习的SE(2)端口哈密顿动力学和IDA-PBC控制设计跟踪期望轨迹 (橙色) 的轨迹 (蓝色)。(a) 使用勒蒙尼斯科特轨迹的跟踪性能。(b) 跟踪勒蒙尼斯科特轨迹 (左) 和分段线性轨迹 (右)。

以及分段线性轨迹。图 5(b) 展示了通过我们基于能量的控制器控制的全向地面车辆实现了成功的轨迹跟踪。控制增益选择为: $K_p = 0.72I$, $K_v = 0.8I$, $K_R = 9.1I$, 以及 $K_\omega = 3.6$ 。图 5(a) 绘制了位置、偏航角、线速度和角速度的跟踪误差。

C. Crazyflie Quadrotor

在本节中, 我们展示了我们的 SE(3) 动力学学习与控制方法能够实现欠驱动系统的轨迹跟踪。我们考虑了一个 Crazyflie 四旋翼, 如图 8(a) 所示, 在基于物理的仿真器 PyBullet [81] 中进行了模拟。控制输入 $u = [f, \tau]$ 包括由四个旋翼产生的推力 $f \in \mathbb{R}_{\geq 0}$ 和力矩向量 $\tau \in \mathbb{R}^3$ 。广义坐标和速度如前所示, 为 $q = [p^\top, r_1^\top, r_2^\top, r_3^\top]^\top$ 和 $\zeta = [v^\top, \omega^\top]^\top$ 。

四旋翼从随机起点使用 PID 控制器 [81] 控制到 18 个不同的目标姿态, 提供了 18 条 2.5 秒的轨迹。这些轨迹用于生成数据集 $\mathcal{D} = \{t_{0:N}^{(i)}, q_{0:N}^{(i)}, \zeta_{0:N}^{(i)}, u^{(i)}\}_{i=1}^D$, 其中 $N = 5$, $D = 1080$ 。如第 V-D 节所述, SE(3) 哈密顿常微分方程网络进行了 500 次迭代的训练, 没有使用名义模型, 即 $M_{v0}^{-1}(q) = 0$, $M_{\omega0}^{-1}(q) = 0$, $D_{v0}(q, p) = 0$, $D_{\omega0}(q, p) = 0$, $V_0(q) = 0$ 和 $B_0(q) = 0$ 。在 PyBullet 仿真器中, 我们没有考虑能量耗散, 例如阻力效应, 并且在模型设计中省略了耗散矩阵 $D_\theta(q, p)$ 。

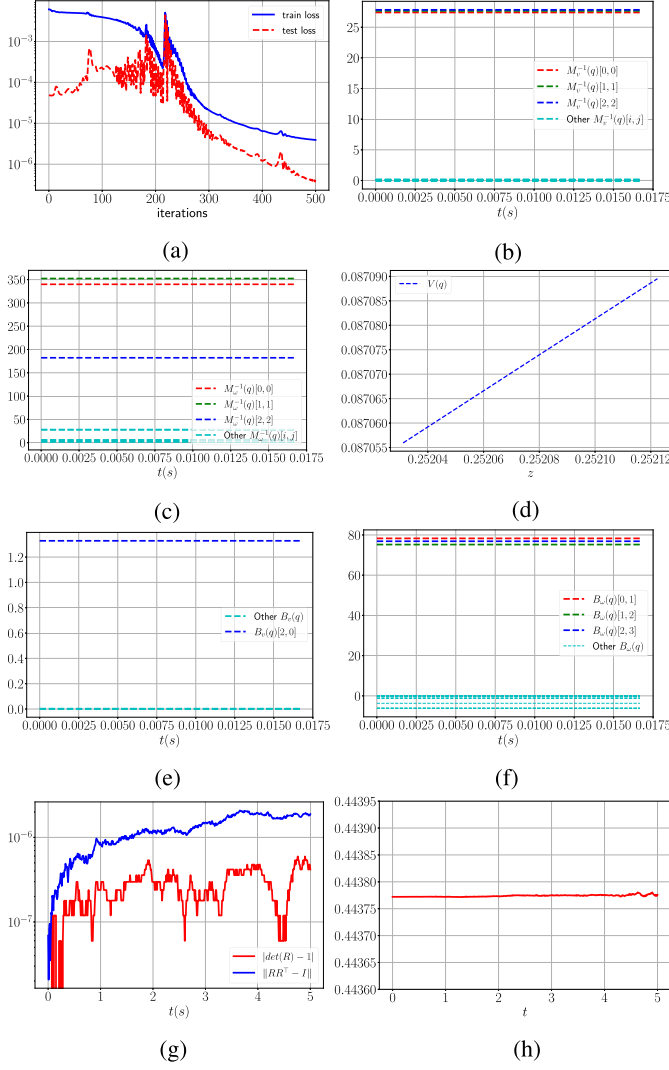


Fig. 6. SE(3) port-Hamiltonian neural ODE network on a Crazyflie quadrotor in the PyBullet simulator [81]. (a) Loss (log scale). (b) $\mathbf{M}_v^{-1}(\mathbf{q})$. (c) $\mathbf{M}_\omega^{-1}(\mathbf{q})$. (d) $\mathcal{V}(\mathbf{q})$. (e) $\mathbf{B}_v(\mathbf{q})$. (f) $\mathbf{B}_\omega(\mathbf{q})$. (g) SO(3) constraints. (h) Total energy.

The training and test results are shown in Fig. 6. The learned generalized mass and inertia converged to constant diagonal matrices: $\mathbf{M}_v^{-1}(\mathbf{q}) \approx 27.5\mathbf{I}$ and $\mathbf{M}_\omega^{-1}(\mathbf{q}) \approx \text{diag}([351, 340, 181])$. The input matrix $\mathbf{B}_v(\mathbf{q})$ converged to a constant matrix with $[\mathbf{B}_v(\mathbf{q})]_{2,0} \approx 1.32$, while the other entries were closed to 0, consistent with the fact that the quadrotor thrust only affects the linear velocity along the z -axis in the body-fixed frame. The input matrix $\mathbf{B}_\omega(\mathbf{q})$ converged to $\sim 76\mathbf{I}$ as the motor torques affects all components of the angular velocity ω . The learned potential energy $\mathcal{V}(\mathbf{q})$ was linear in the height z , agreeing with the gravitational potential.

We also verified our energy-based control design in Section VI by controlling the quadrotor to track a desired trajectory based on the learned dynamics model. Given desired position $\mathbf{p}^*$ and heading ψ^* (yaw angle), we construct an appropriate $\mathbf{R}^*$ and $\mathbf{p}^*$ to be used with the energy-based controller in (60). By expanding the terms in (60) and choosing the control gain $\mathbf{K}_d$ of the form

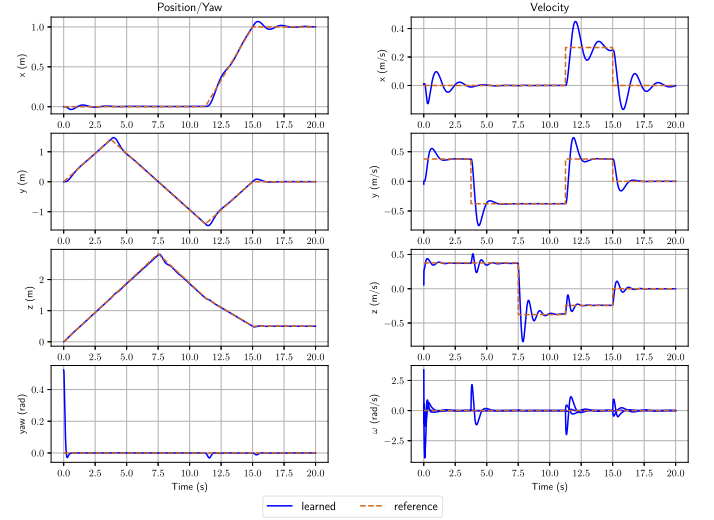


Fig. 7. Crazyflie quadrotor trajectory (blue) tracking a desired diamond-shaped trajectory (orange) shown in Fig. 8.

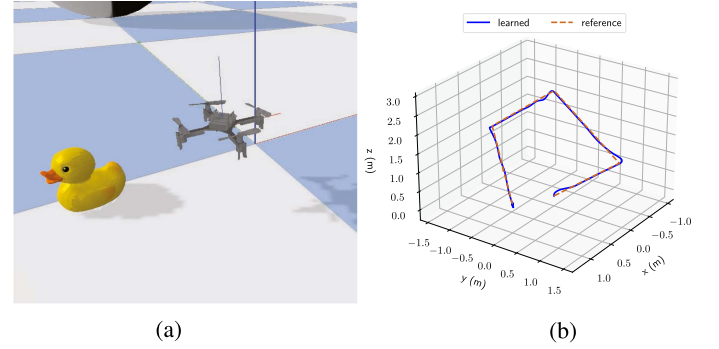


Fig. 8. Trajectory tracking experiment with a Crazyflie quadrotor in the PyBullet simulator [81]. (a) Crazyflie simulator. (b) Trajectory tracking.

TABLE I
COMPARISON OF DIFFERENT NEURAL NETWORK ARCHITECTURES FOR PENDULUM AND QUADROTOR DYNAMICS LEARNING

Metrics	Platform	Black-box	Unstructured Hamiltonian	Structured Hamiltonian
Training loss	Pendulum	0.037	6.3×10^{-5}	2.3×10^{-7}
$\ \det(\mathbf{R}) - 1\ $ (avg.)	Pendulum	1888752	1.4×10^{-3}	1.4×10^{-3}
$\ \mathbf{R}\mathbf{R}^T - \mathbf{I}\ $ (avg.)	Pendulum	267971.6	2.1×10^{-3}	2.1×10^{-3}
Total energy (std.)	Pendulum	13557.1	0.163	0.003
Prediction error (avg.)	Pendulum	1.02 (rad)	0.08 (rad)	0.008 (rad)
Training loss	Quadrotor	2.2×10^{-3}	6.4×10^{-4}	3.92×10^{-6}
$\ \det(\mathbf{R}) - 1\ $ (avg.)	Quadrotor	3741	2.9×10^{-6}	2.6×10^{-7}
$\ \mathbf{R}\mathbf{R}^T - \mathbf{I}\ $ (avg.)	Quadrotor	29336.7	7.6×10^{-6}	1.3×10^{-6}
Total energy (std.)	Quadrotor	18.1	0.074	1.64×10^{-6}
Prediction error (avg.)	Quadrotor	0.49 (m)	0.46 (m)	0.02 (m)

The bold values are to emphasize that our method outperforms other baselines, i.e. achieving smaller errors.

$\mathbf{K}_d = \begin{bmatrix} \mathbf{K}_v & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_\omega \end{bmatrix}$, the control input can be written explicitly as

$$\mathbf{u} = \mathbf{B}^\dagger(\mathbf{q}) \begin{bmatrix} \tau_v \\ \tau_\omega \end{bmatrix} \quad (73)$$

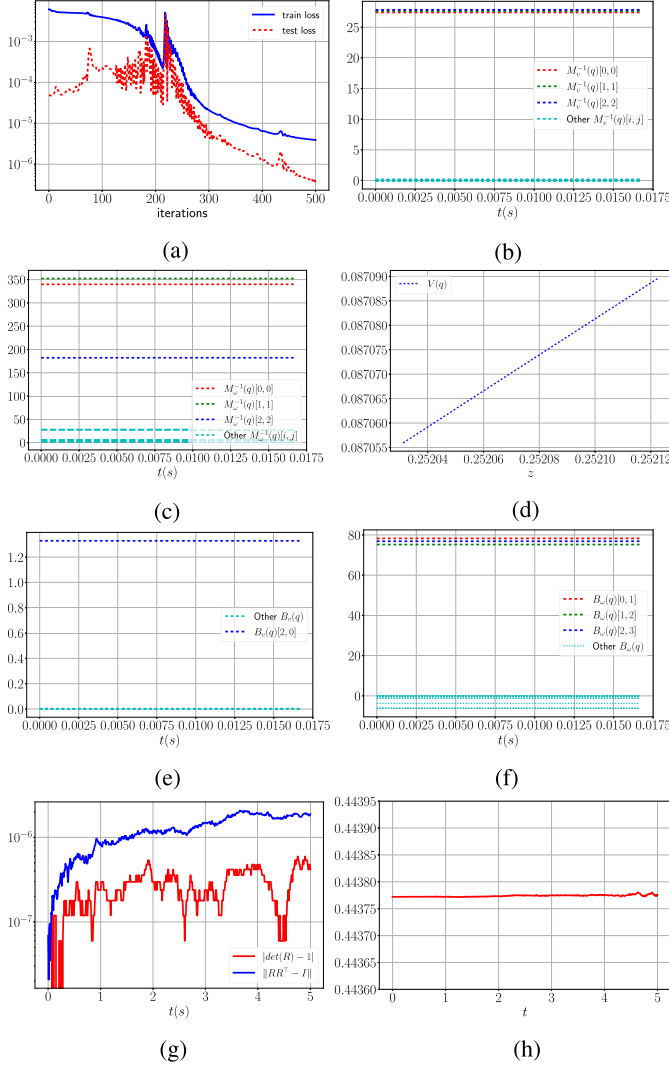


图6. 在PyBullet模拟器中的Crazyflie四旋翼上的SE(3)端口-哈密顿神经ODE网络 [81]。 (a) 损失 (对数刻度)。 (b) $M_v^{-1}(\mathbf{q})$ 。 (c) $M_\omega^{-1}(\mathbf{q})$ 。 (d) $V(\mathbf{q})$ 。 (e) $B_v(\mathbf{q})$ 。 (f) $B_\omega(\mathbf{q})$ 。 (g) SO(3)约束。 (h) 总能量。

训练和测试结果如图6所示。学习到的广义质量和惯性收敛到常数对角矩阵: $M_v^{-1}(\mathbf{q}) \approx 27.5\mathbf{I}$ 和 $M_\omega^{-1}(\mathbf{q}) \approx \text{diag}([351, 340, 181])$ 。输入矩阵 $B_v(\mathbf{q})$ 收敛到一个常数矩阵, 其中 $[B_v(\mathbf{q})]_{2,0} \approx 1.32$, 而其他项接近0, 这与四旋翼推力仅影响机体固定坐标系下 z 轴方向的线速度的事实一致。输入矩阵 $B_\omega(\mathbf{q})$ 收敛到 $\sim 76\mathbf{I}$, 因为电机扭矩影响角速度 ω 的所有分量。学习到的势能 $V(\mathbf{q})$ 在高度 z 上是线性的, 与重力势能一致。

我们还通过在第六节中基于学习到的动力学模型控制四旋翼跟踪期望轨迹来验证我们的基于能量的控制设计。给定期望位置 $\mathbf{p}^*$ 和航向 ψ^* (偏航角), 我们构建适当的 $\mathbf{R}^*$ 和 $\mathbf{p}^*$ 以与(60)中的基于能量的控制器一起使用。通过展开(60)中的项并选择形式为 $\mathbf{K}_d$ 的控制增益

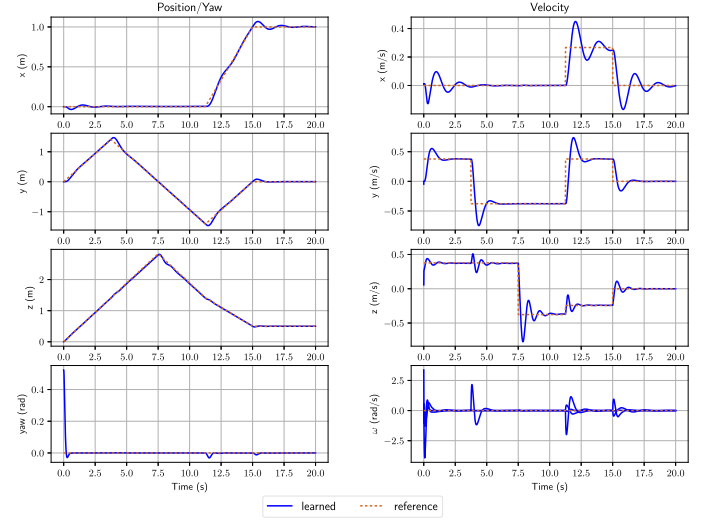


图7. Crazyflie四旋翼跟踪期望的菱形轨迹 (橙色), 实际轨迹为蓝色, 如图8所示。

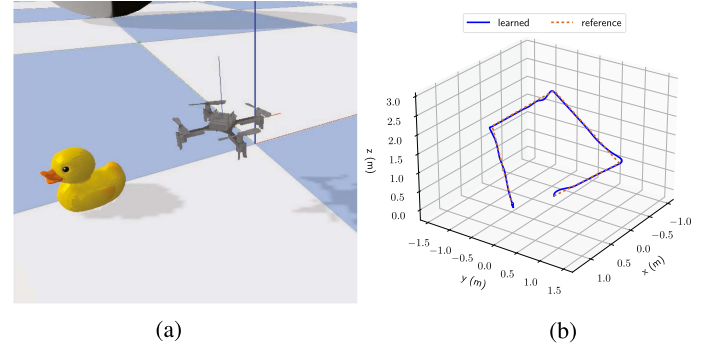


图8. 使用Crazyflie四旋翼在PyBullet模拟器中进行的轨迹跟踪实验 [81]。 (a) Crazyflie模拟器。 (b) 轨迹跟踪。

表 I 摆和四旋翼动力学学习不同神经网络架构比较

Metrics	Platform	Black-box	Unstructured Hamiltonian	Structured Hamiltonian
Training loss	Pendulum	0.037	6.3×10^{-5}	2.3×10^{-7}
$\ \det(\mathbf{R}) - 1\ $ (avg.)	Pendulum	1888752	1.4×10^{-3}	1.4×10^{-3}
$\ \mathbf{R}\mathbf{R}^T - \mathbf{I}\ $ (avg.)	Pendulum	267971.6	2.1×10^{-3}	2.1×10^{-3}
Total energy (std.)	Pendulum	13557.1	0.163	0.003
Prediction error (avg.)	Pendulum	1.02 (rad)	0.08 (rad)	0.008 (rad)
Training loss	Quadrotor	2.2×10^{-3}	6.4×10^{-4}	3.92×10^{-6}
$\ \det(\mathbf{R}) - 1\ $ (avg.)	Quadrotor	3741	2.9×10^{-6}	2.6×10^{-7}
$\ \mathbf{R}\mathbf{R}^T - \mathbf{I}\ $ (avg.)	Quadrotor	29336.7	7.6×10^{-6}	1.3×10^{-6}
Total energy (std.)	Quadrotor	18.1	0.074	1.64×10^{-6}
Prediction error (avg.)	Quadrotor	0.49 (m)	0.46 (m)	0.02 (m)

The bold values are to emphasize that our method outperforms other baselines, i.e. achieving smaller errors.

$\mathbf{K}_d = \begin{bmatrix} \mathbf{K}_v & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_\omega \end{bmatrix}$, the control input can be written explicitly as

$$\mathbf{u} = \mathbf{B}^\dagger(\mathbf{q}) \begin{bmatrix} \tau_v \\ \tau_\omega \end{bmatrix} \quad (73)$$

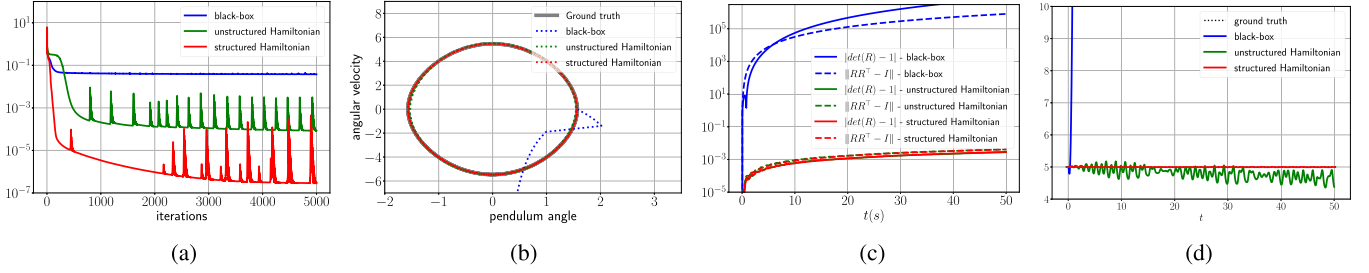


Fig. 9. Comparison of different neural network architectures to learn pendulum dynamics: black-box, i.e., the dynamics function $\mathbf{f}$ is represented by a multilayer perceptron network; unstructured Hamiltonian, i.e., the Hamiltonian function is represented by a multilayer perceptron network instead of the sum of kinetic and potential energy as shown in (27); and structured Hamiltonian, i.e., the Hamiltonian function has the form of (27). Initialized at $\phi = \pi/2$, the learned pendulum dynamics are rolled out, showing that our approach with structured Hamiltonian preserves the phase portraits, SO(3) constraints, and the conservation of energy better than the other models. (a) Training loss. (b) Phase portraits. (c) SO(3) constraints. (d) Total energy.

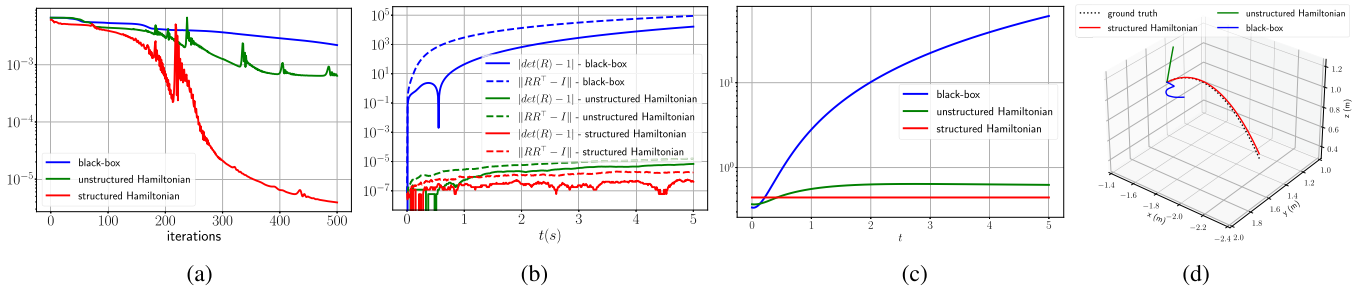


Fig. 10. Comparison of different neural network architectures to learn quadrotor dynamics: black-box, i.e., the dynamics function $\mathbf{f}$ is represented by a multilayer perceptron network; unstructured Hamiltonian, i.e., the Hamiltonian function is represented by a multilayer perceptron network instead of the sum of kinetic and potential energy as shown in (27); and structured Hamiltonian, i.e., the Hamiltonian function has the form of (27). Initialized at a random pose and twist, the learned quadrotor dynamics are rolled out, showing that our approach with structured Hamiltonian preserves SO(3) constraints and the conservation of energy better than the other models, and provides better state predictions. (a) Training loss. (b) SO(3) constraints. (c) Total energy. (d) Predicted trajectories.

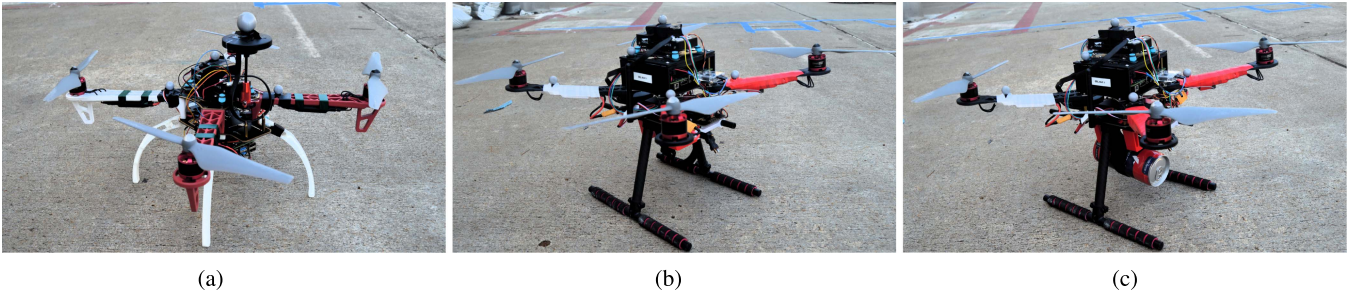


Fig. 11. Quadrotor robots used in the experiments. (a) RaspberryPi quadrotor whose mass and inertia matrix serve as nominal values for our learning framework. (b) Intel NUC quadrotor with a different frame. (c) Intel NUC quadrotor carrying a coffee can as payload.

where

$$\begin{aligned} \tau_v = & \mathbf{R}^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{p}} - \hat{\mathbf{p}}_v \omega - \mathbf{R}^\top \mathbf{K}_p (\mathbf{p} - \mathbf{p}^*) \\ & - \mathbf{K}_v (\mathbf{v} - \mathbf{R}^\top \dot{\mathbf{p}}^*) + \mathbf{M}_1 (\mathbf{R}^\top \ddot{\mathbf{p}}^* - \dot{\omega} \mathbf{R}^\top \dot{\mathbf{p}}^*) \end{aligned} \quad (74)$$

$$\begin{aligned} \tau_\omega = & \sum_{i=1}^3 \hat{\mathbf{r}}_i \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{r}_i} - \mathbf{K}_\omega (\omega - \mathbf{R}^\top \mathbf{R}^* \omega^*) \\ & - (\hat{\mathbf{p}}_\omega \omega + \hat{\mathbf{p}}_v \mathbf{v}) - \frac{1}{2} (\mathbf{K}_R \mathbf{R}^{*\top} \mathbf{R} - \mathbf{R}^\top \mathbf{R}^* \mathbf{K}_R^\top)^\vee \\ & + \mathbf{M}_2 (\mathbf{R}^\top \mathbf{R}^* \dot{\omega}^* - \dot{\omega}_e \mathbf{R}^\top \mathbf{R}^* \omega^*). \end{aligned} \quad (75)$$

Note that $\tau_v \in \mathbb{R}^3$ is the desired thrust in the body frame and depends only on the desired position $\mathbf{p}^*$ and the current pose. The desired thrust is transformed to the world frame as $\mathbf{R} \tau_v$. Inspired by [79], the vector $\mathbf{R} \tau_v$ should be along the z -axis of the body frame, i.e., the third column $\mathbf{r}_3^*$ of the desired rotation matrix $\mathbf{R}^*$. The second column $\mathbf{r}_2^*$ of the desired rotation matrix $\mathbf{R}^*$ can be chosen so that it has the desired yaw angle ψ^* and is perpendicular to $\mathbf{r}_3^*$. This can be done by projecting the second column of the yaw's rotation matrix $\mathbf{r}_2^\psi = [-\sin \psi, \cos \psi, 0]$ onto the plane perpendicular to $\mathbf{r}_3^*$. We have $\mathbf{R}^* = [\mathbf{r}_1^* \ \mathbf{r}_2^* \ \mathbf{r}_3^*]$ where

$$\mathbf{r}_3^* = \frac{\mathbf{R} \tau_v}{\|\mathbf{R} \tau_v\|}, \mathbf{r}_1^* = \frac{\mathbf{r}_2^\psi \times \mathbf{r}_3^*}{\|\mathbf{r}_2^\psi \times \mathbf{r}_3^*\|}, \mathbf{r}_2^* = \mathbf{r}_3^* \times \mathbf{r}_1^* \quad (76)$$

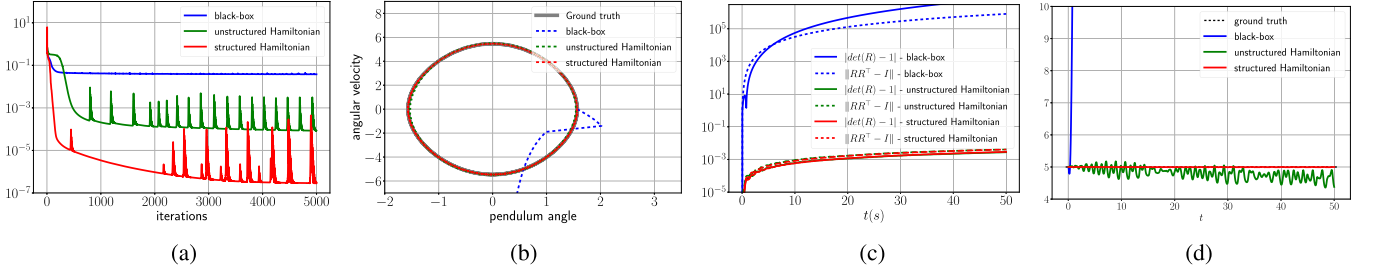


图9. 不同神经网络架构学习摆动力学的比较：黑箱，即动力学函数 $\mathbf{f}$ 由多层感知机网络表示；无结构哈密顿，即哈密顿函数由多层感知机网络表示，而不是如公式(27)所示的动能和势能之和；以及结构化哈密顿，即哈密顿函数具有公式(27)的形式。在 $\phi = \pi/2$ 初始化后，展开学习到的摆动力学，显示出我们使用结构化哈密顿的方法比其他模型更好地保持相位图、SO(3)约束和能量守恒。(a) 训练损失。(b) 相位图。(c) SO(3)约束。(d) 总能量

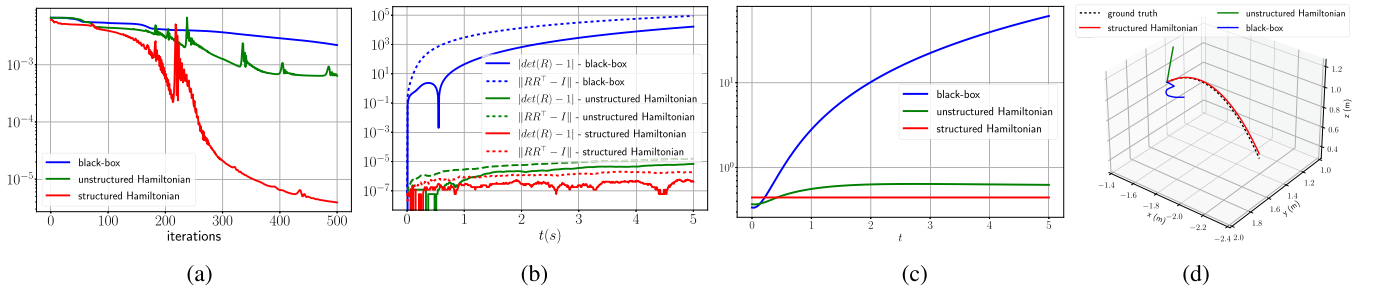


图 10. 不同神经网络架构学习四旋翼动力学的比较：黑箱模型，即动力学函数 $\mathbf{f}$ 由多层感知器网络表示；非结构化哈密顿量，即哈密顿量函数由多层感知器网络表示，而不是如公式 (27) 所示的动能与势能之和；以及结构化哈密顿量，即哈密顿量函数具有公式 (27) 的形式。在随机姿态和角速度初始化下，展开学习到的四旋翼动力学，结果表明我们采用结构化哈密顿量的方法比其他模型更好地保持了 SO(3) 约束和能量守恒，并提供了更好的状态预测。(a) 训练损失。(b) SO(3) 约束。(c) 总能量。(d) 预测轨迹。

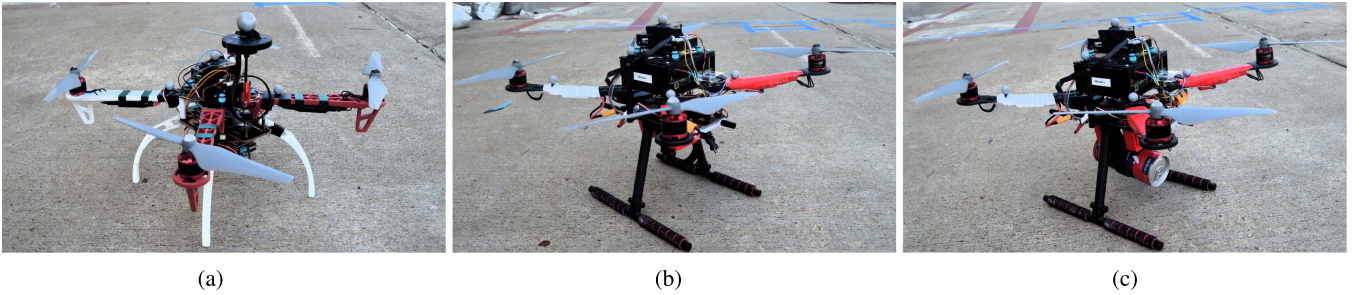


图11. 实验中使用的四旋翼机器人。(a) RaspberryPi 四旋翼，其质量和惯性矩阵作为我们学习框架的标称值。(b) 具有不同机架的 Intel NUC 四旋翼。(c) 携带咖啡罐作为有效载荷的 Intel NUC 四旋翼。

哪里

$$\begin{aligned} \tau_v = \mathbf{R}^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{p}} - \hat{\mathbf{p}}_v \omega - \mathbf{R}^\top \mathbf{K}_p(\mathbf{p} - \mathbf{p}^*) \\ - \mathbf{K}_v(\mathbf{v} - \mathbf{R}^\top \dot{\mathbf{p}}^*) + \mathbf{M}_1(\mathbf{R}^\top \ddot{\mathbf{p}}^* - \dot{\omega} \mathbf{R}^\top \dot{\mathbf{p}}^*) \end{aligned} \quad (74)$$

$$\begin{aligned} \tau_\omega = \sum_{i=1}^3 \hat{\mathbf{r}}_i \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{r}_i} - \mathbf{K}_\omega(\omega - \mathbf{R}^\top \mathbf{R}^* \omega^*) \\ - (\hat{\mathbf{p}}_\omega \omega + \hat{\mathbf{p}}_v \mathbf{v}) - \frac{1}{2} (\mathbf{K}_R \mathbf{R}^{*\top} \mathbf{R} - \mathbf{R}^\top \mathbf{R}^* \mathbf{K}_R^\top)^\vee \\ + \mathbf{M}_2(\mathbf{R}^\top \mathbf{R}^* \dot{\omega}^* - \dot{\omega}_e \mathbf{R}^\top \mathbf{R}^* \omega^*). \end{aligned} \quad (75)$$

请注意， $\tau_v \in \mathbb{R}^3$ 是机体坐标系下的期望推力，仅取决于期望位置 $\mathbf{p}^*$ 和当前姿态。期望推力被转换到世界坐标系下为 $\mathbf{R}\tau_v$ 。参考文献 [79]，向量 $\mathbf{R}\tau_v$ 应该沿着机体坐标系的 z 轴，即期望旋转矩阵 $\mathbf{R}^*$ 的第三列 $\mathbf{r}_3^*$ 。期望旋转矩阵 $\mathbf{R}^*$ 的第二列 $\mathbf{r}_2^*$ 可以选择，使其具有期望偏航角 ψ^* 并且与 $\mathbf{r}_3^*$ 垂直。这可以通过将偏航旋转矩阵 $\mathbf{r}_2^\psi$ 的第二列 $[-\sin \psi, \cos \psi, 0]$ 投影到与 $\mathbf{r}_3^*$ 垂直的平面上来实现。我们有 $\mathbf{R}^* = [\mathbf{r}_1^* \ \mathbf{r}_2^* \ \mathbf{r}_3^*]$ ，其中

$$\mathbf{r}_3^* = \frac{\mathbf{R}\tau_v}{\|\mathbf{R}\tau_v\|}, \mathbf{r}_1^* = \frac{\mathbf{r}_2^\psi \times \mathbf{r}_3^*}{\|\mathbf{r}_2^\psi \times \mathbf{r}_3^*\|}, \mathbf{r}_2^* = \mathbf{r}_3^* \times \mathbf{r}_1^* \quad (76)$$

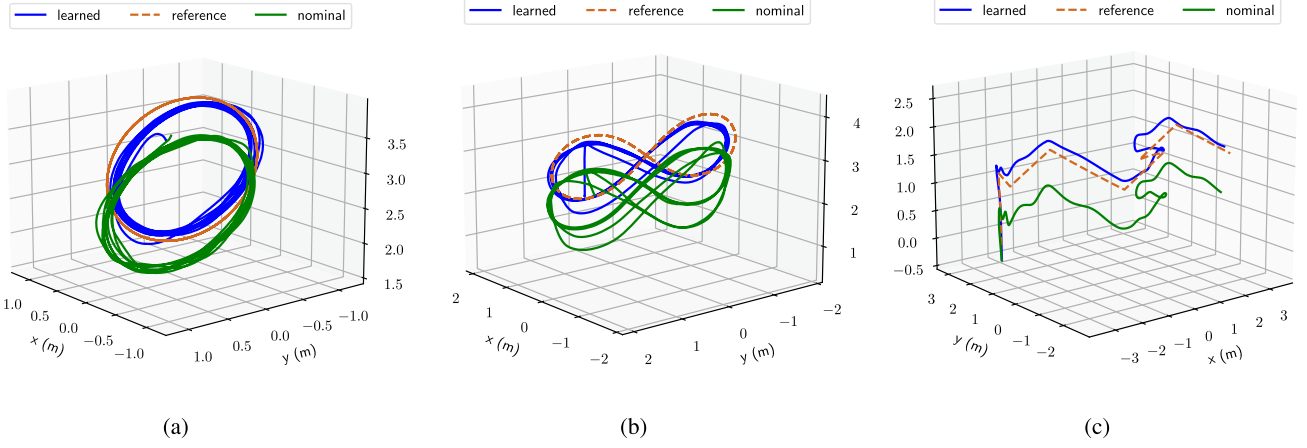


Fig. 12. Trajectory tracking with real quadrotors. (a) Vertical circle. (b) Vertical lemniscate. (c) Piecewise linear trajectory.

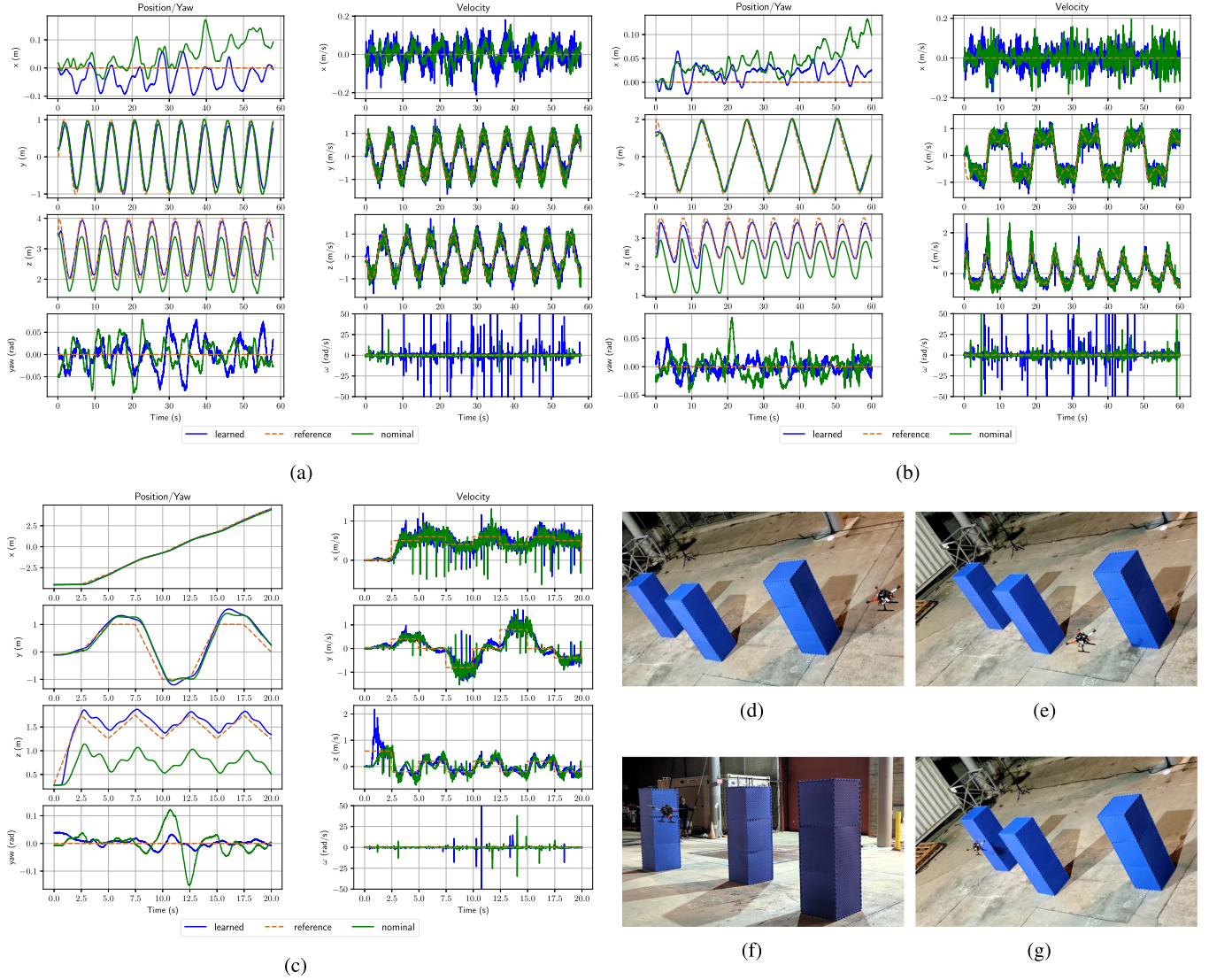


Fig. 13. Real quadrotor trajectory using our learned model and controller (blue) and using a nominal model and a geometric controller [79] (green) tracking a desired trajectory. (a) Vertical circle. (b) Vertical lemniscate. (c) Piecewise linear. (d) Tracking at $t = 0$ s. (e) Tracking at $t = 3$ s. (f) Tracking at $t = 9$ s. (g) Tracking at $t = 12$ s.

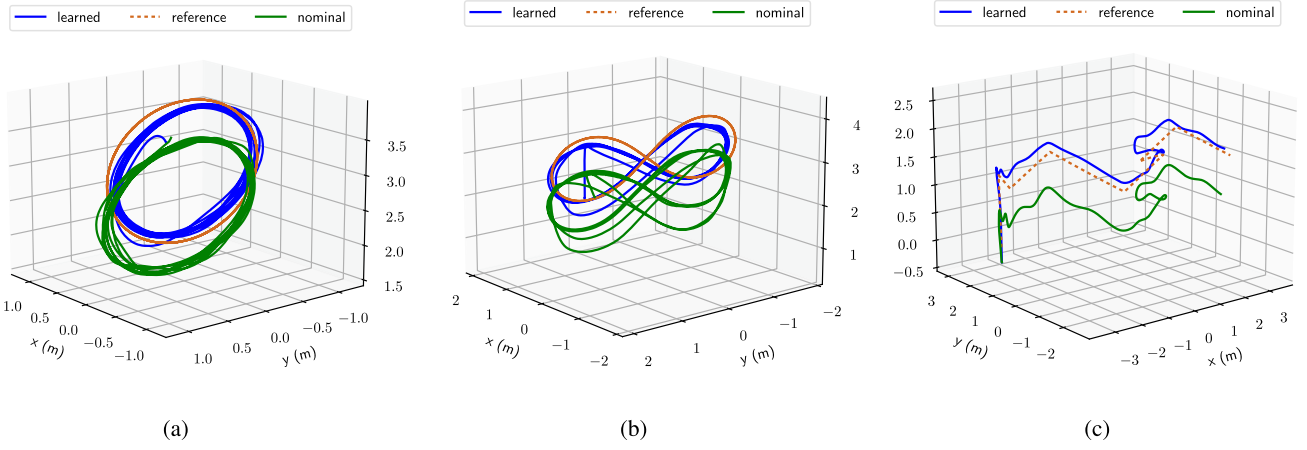


图12. 使用真实四旋翼的轨迹跟踪。(a) 垂直圆。(b) 垂直勒姆尼萨特曲线。(c) 分段线性轨迹。

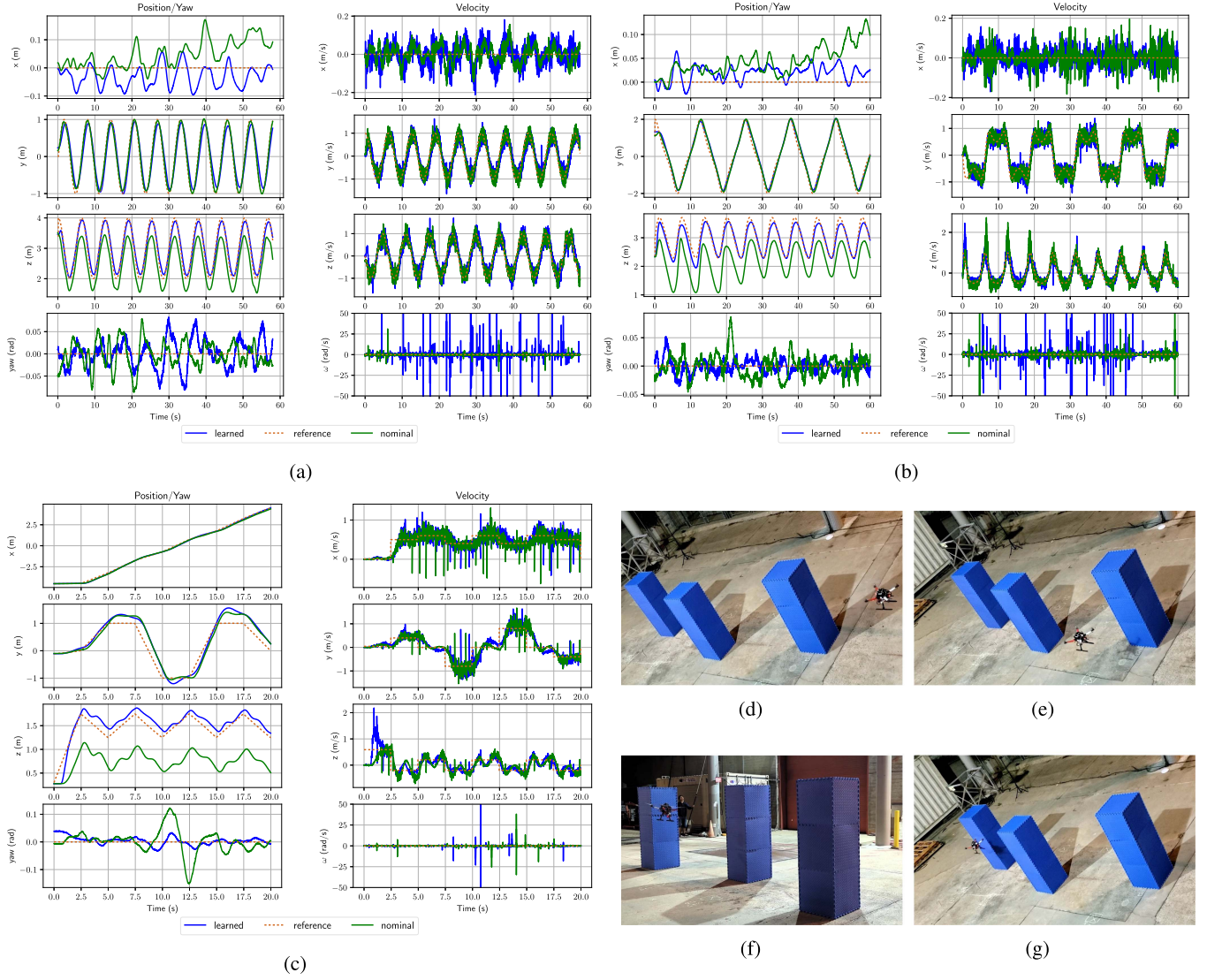


图13. 使用我们学习的模型和控制器 (蓝色) 以及使用名义模型和几何控制器[79] (绿色) 跟踪期望轨迹的真实四旋翼轨迹。(a) 垂直圆。(b) 垂直莱蒙尼形。(c) 分段线性。(d) 在 $t = 0$ 秒时跟踪。(e) 在 $t = 3$ 秒时跟踪。(f) 在 $t = 9$ 秒时跟踪。(g) 在 $t = 12$ 秒时跟踪。

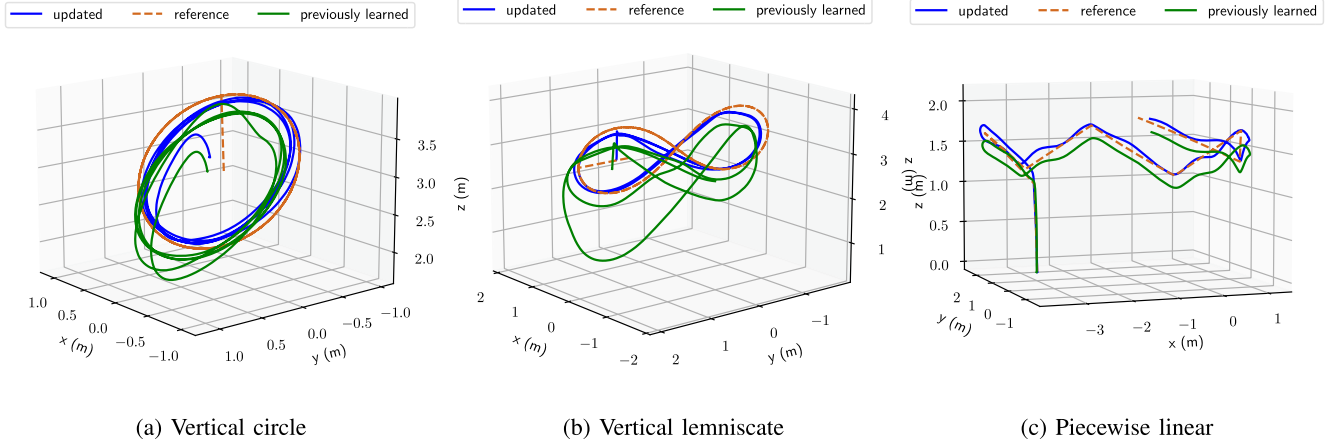


Fig. 14. Quadrotor trajectory tracking experiment with extra payload: (a) vertical circle, (b) vertical lemniscate, and (c) piecewise linear trajectory.

and $\dot{\omega}^* = \mathbf{R}^{*\top} \dot{\mathbf{R}}^*$. The derivative $\dot{\mathbf{R}}^*$ is calculated as follows:

$$\dot{\mathbf{r}}_3^* = \mathbf{r}_3^* \times \frac{\mathbf{R} \dot{\boldsymbol{\tau}}_{\mathbf{v}}}{\|\mathbf{R} \boldsymbol{\tau}_{\mathbf{v}}\|} \times \mathbf{r}_3^* \quad (77)$$

$$\dot{\mathbf{r}}_1^* = \mathbf{r}_1^* \times \frac{\dot{\mathbf{r}}_2^{\psi} \times \mathbf{r}_3^* + \mathbf{r}_2^{\psi} \times \dot{\mathbf{r}}_3^*}{\|\mathbf{r}_2^{\psi} \times \mathbf{r}_3^*\|} \times \mathbf{r}_1^* \quad (78)$$

$$\dot{\mathbf{r}}_2^* = \dot{\mathbf{r}}_3^* \times \mathbf{r}_1^* + \mathbf{r}_3^* \times \dot{\mathbf{r}}_1^*. \quad (79)$$

Plugging $\mathbf{R}^*$ and ω^* back in $\boldsymbol{\tau}_{\omega}$, we obtain the complete control input $\mathbf{u}$ in (73).

Fig. 8(b) shows qualitatively that the quadrotor controlled by our energy-based controller achieves successful trajectory tracking. The control gains were chosen as: $\mathbf{K}_{\mathbf{p}} = \text{diag}([0.8, 0.8, 3.9])$, $\mathbf{K}_{\mathbf{v}} = 0.23\mathbf{I}$, $\mathbf{K}_{\mathbf{R}} = \text{diag}([3.6, 3.6, 6.9])$, and $\mathbf{K}_{\omega} = \text{diag}([0.3, 0.3, 0.6])$. Fig. 7 shows quantitatively the tracking errors in the position, yaw angles, linear velocity, and angular velocity. On an Intel i9 3.1-GHz CPU with 32-GB RAM, our controller's computation time in Python 3.7 was about 2.5 ms per control input, including forward passes through the neural networks, showing that it is suitable for fast real-time applications.

D. Comparison to Unstructured Neural ODE Models

In this section, we show the benefits of our neural ODE architecture by comparing the following: 1) our *structured Hamiltonian model*; 2) a *black-box model*, i.e., the approximated dynamics $\mathbf{f}$ is represented by a multilayer perceptron network; and 3) an *unstructured Hamiltonian model*, i.e., the Hamiltonian function is represented by a multilayer perceptron network instead of using the structure in (27), in terms of training convergence rates, satisfaction of energy conservation principle, and Lie group constraints. To verify energy conservation, we rolled out the learned dynamics and calculated the Hamiltonian via (27) along the predicted trajectories for the following:

- 1) the black-box model using ground-truth mass and potential energy with the predicted states;
- 2) the unstructured Hamiltonian model using the output of the multilayer perceptron Hamiltonian network;

- 3) the structured Hamiltonian model using the learned mass and potential energy networks.

We check the $\text{SO}(3)$ constraints by verifying that two quantities $|\det \mathbf{R} - 1|$ and $\|\mathbf{R}\mathbf{R}^{\top} - \mathbf{I}\|$ remain small along the predicted trajectories.

We first use a pendulum as described in Section VII-A without energy dissipation. The models are trained for 5000 iterations from 512 0.2-s state-control trajectories and rolled out for a significantly longer horizon of 50 s. Fig. 9 plots the training loss, the phase portraits, the $\text{SO}(3)$ constraints, and the total energy (Hamiltonian) of the learned models for a pendulum system. As the Hamiltonian structure is imposed in the neural ODE network architecture, our model is able to converge faster with lower loss [see Fig. 9(a) and Table I], preserves the phase portraits for state predictions [see Fig. 9(b)], and achieves the lowest angle prediction error in Table I. Fig. 9(c) and Table I show that the $\text{SO}(3)$ constraints are satisfied by our structured and unstructured Hamiltonian models as their values of $|\det \mathbf{R} - 1|$ and $\|\mathbf{R}\mathbf{R}^{\top} - \mathbf{I}\|$ remain small along a 50-s trajectory rollout initialized at $\phi = \pi/2$. The constant Hamiltonian in Fig. 9(d) of our structured Hamiltonian with lowest standard derivation value in Table I verifies that our model obeys the energy conservation law with high precision, given no control input and no energy dissipation. The Hamiltonian of the black-box model increases along the trajectory while that of the unstructured Hamiltonian model fluctuates and slightly decreases over time.

We also tested the models using the simulated Crazyflie quadrotor with the same dataset $\mathcal{D}$ of 18 trajectories as described in Section VII-C. The $\text{SE}(3)$ port-Hamiltonian ODE network was trained, as described in Section V-D, for 500 iterations. Our structured Hamiltonian model converges faster with significantly lower loss as seen in Fig. 10(a) and in Table I. We verified that the predicted orientation trajectories from our learned models satisfy the $\text{SO}(3)$ constraints. Fig. 10(b) and Table I show two near-zero quantities $|\det \mathbf{R} - 1|$ and $\|\mathbf{R}\mathbf{R}^{\top} - \mathbf{I}\|$, obtained by rolling out our learned dynamics for 5 s, while the learned black-box model significantly violates the constraints after a very short time. Fig. 10(c) shows a constant total energy

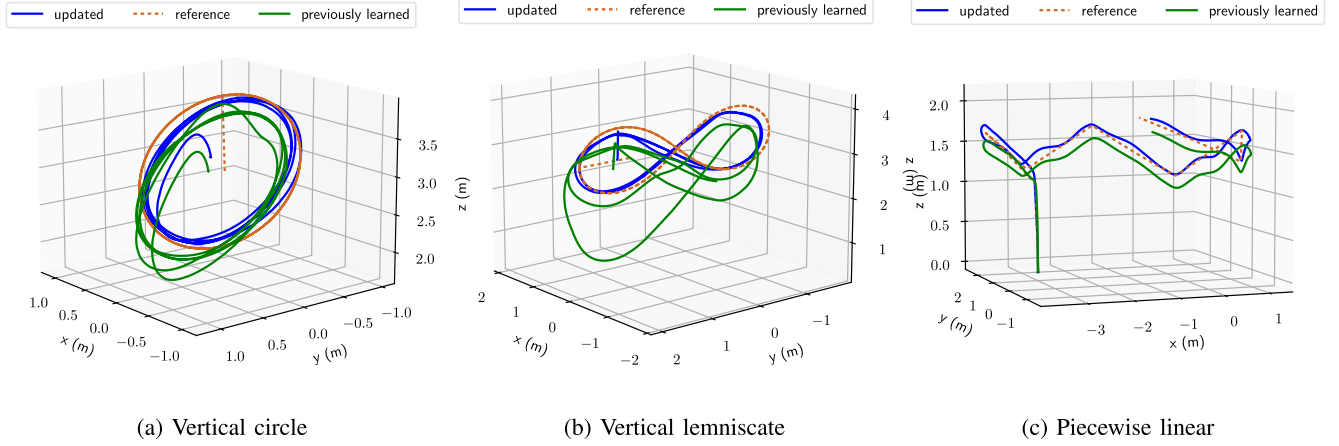


图14. 携带额外负载的四旋翼轨迹跟踪实验：(a) 竖直圆，(b) 竖直双纽线，和(c) 分段线性轨迹。

以及 $\dot{\omega}^* = \mathbf{R}^{*\top} \dot{\mathbf{R}}^*$ 。导数 $\dot{\mathbf{R}}^*$ 计算如下：

$$\dot{\mathbf{r}}_3^* = \mathbf{r}_3^* \times \frac{\mathbf{R}^* \dot{\boldsymbol{\tau}}_{\mathbf{v}}}{\|\mathbf{R}^* \dot{\boldsymbol{\tau}}_{\mathbf{v}}\|} \times \mathbf{r}_3^* \quad (77)$$

$$\dot{\mathbf{r}}_1^* = \mathbf{r}_1^* \times \frac{\dot{\mathbf{r}}_2^* \times \mathbf{r}_3^* + \mathbf{r}_2^* \times \dot{\mathbf{r}}_3^*}{\|\mathbf{r}_2^* \times \mathbf{r}_3^*\|} \times \mathbf{r}_1^* \quad (78)$$

$$\dot{\mathbf{r}}_2^* = \dot{\mathbf{r}}_3^* \times \mathbf{r}_1^* + \mathbf{r}_3^* \times \dot{\mathbf{r}}_1^*. \quad (79)$$

将 $\mathbf{R}^*$ 和 ω^* 代回 $\boldsymbol{\tau}_{\omega}$ ，我们得到 (73) 中的完整控制输入 $\mathbf{u}$ 。

图 8(b) 定性地显示了由我们基于能量的控制器控制的四旋翼实现了成功的轨迹跟踪。控制增益选择如下： $\mathbf{K}_{\mathbf{p}} = \text{diag}([0.8, 0.8, 3.9])$ ， $\mathbf{K}_{\mathbf{v}} = 0.23\mathbf{I}$ ， $\mathbf{K}_{\mathbf{R}} = \text{diag}([3.6, 3.6, 6.9])$ ，以及 $\mathbf{K}_{\omega} = \text{diag}([0.3, 0.3, 0.6])$ 。图 7 定量地显示了位置、偏航角、线速度和角速度的跟踪误差。在配备 32-GB RAM 的 Intel i9 3.1-GHz CPU 上，我们控制器在 Python 3.7 中的计算时间约为每个控制输入 2.5 毫秒，包括神经网络的前向传播，这表明其适用于快速实时应用。

D. Comparison to Unstructured Neural ODE Models

在本节中，我们通过比较以下内容展示我们神经ODE架构的优势：1) 我们的 *structured Hamiltonian model*；2) 一个 *black-box model*，即近似动力学 $\mathbf{f}$ 由多层感知器网络表示；以及 3) 一个 *unstructured Hamiltonian model*，即哈密顿函数由多层感知器网络表示，而不是使用(27)中的结构，从训练收敛速度、能量守恒原理的满足情况和李群约束方面进行比较。为了验证能量守恒，我们展开了学习到的动力学，并通过(27)沿预测轨迹计算哈密顿如下：

- 1) 使用真实质量和势能与预测状态的黑箱模型；2) 使用多层感知器哈密顿网络输出的非结构化哈密顿模型；

- 3) 使用学习到的质量和势能网络的结构化哈密顿模型。

我们通过验证两个量 $|\det \mathbf{R} - 1|$ 和 $\|\mathbf{R}\mathbf{R}^{\top} - \mathbf{I}\|$ 在预测的轨迹上保持较小来检查 SO(3) 约束。

我们首先使用第 VII-A 节中描述的摆系统，并且不考虑能量耗散。这些模型从 512 条 0.2 秒的状态-控制轨迹中训练 5000 次迭代，并在显著更长的 50 秒时间范围内进行展开。图 9 绘制了摆系统学习模型的训练损失、相位图、SO(3) 约束和总能量 (Hamiltonian)。由于在神经 ODE 网络结构中加入 Hamiltonian 结构，我们的模型能够以更低的损失更快收敛 [见图 9(a) 和表 I]，在状态预测中保持相位图 [见图 9(b)]，并在表 I 中实现最低角度预测误差。图 9(c) 和表 I 显示，SO(3) 约束在我们的结构化和非结构化 Hamiltonian 模型中得到满足，因为在初始化为 $\phi = \pi/2$ 的 50 秒轨迹展开过程中，其 $|\det \mathbf{R} - 1|$ 和 $\|\mathbf{R}\mathbf{R}^{\top} - \mathbf{I}\|$ 的值保持较小。图 9(d) 中的结构化 Hamiltonian 的恒定 Hamiltonian 表明，具有最低标准偏差的模型在

我们还使用模拟的 Crazyflie 四旋翼，利用第 VII-C 节中描述的同 18 条轨迹数据集 $\mathcal{D}$ 对模型进行了测试。如第 V-D 节所述，SE(3) port-Hamiltonian ODE 网络进行了 500 次迭代的训练。我们的结构化哈密顿模型收敛更快，并且如图 10(a) 和表 I 所示损失显著较低。我们验证了从学习到的模型预测出的姿态轨迹满足 SO(3) 约束。图 10(b) 和表 I 显示了两个接近零的量 $|\det \mathbf{R} - 1|$ 和 $\|\mathbf{R}\mathbf{R}^{\top} - \mathbf{I}\|$ ，这是通过将学习到的动力学展开 5 秒获得的，而学习的黑箱模型在很短时间后明显违反了约束。图 10(c) 显示了一个恒定的总能量。

along the predicted trajectory, i.e., lowest standard derivation value in Table I, from our structured Hamiltonian model without control input and dissipation networks, verifying that the learned model obeys the law of energy conservation. Fig. 10(d) and the prediction error in Table I show that our structured Hamiltonian model provides better trajectory predictions compared to the other methods.

E. Real Quadrotor Experiments

In this section, we verify our approach using a real quadrotor robot, equipped with an onboard i7 Intel NUC computer and a PX4 flight controller [see Fig. 11(b)]. The quadrotor's pose and twist were provided by a motion capture system.

1) *Learning Quadrotor Dynamics After Upgrade:* We consider a scenario in which the quadrotor is upgraded with a new frame and a new onboard computer, leading to changes in the robot dynamics that we aim to learn from data. The nominal model was obtained from a computer-aided design (CAD) model of another much lighter weight Raspberry Pi quadrotor with a Raspberry Pi computer and an F450 frame [see Fig. 11(a)], which is less accurate and far from the unknown ground-truth model of our upgraded quadrotors. Specifically, the nominal mass and inertia matrix are $\mathbf{M}_{v0} = 1.3\mathbf{I}$ and $\mathbf{M}_{\omega0} = \text{diag}([0.12, 0.12, 0.2])$, respectively, for the upgraded quadrotor in Fig. 11(b). The other nominal matrices were set to zero: $\mathbf{D}_{v0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $\mathbf{D}_{\omega0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $\mathbf{V}_0(\mathbf{q}) = \mathbf{0}$ and $\mathbf{B}_0(\mathbf{q}) = \mathbf{0}$. We modified the PX4 firmware [82] to expose the normalized thrust and torque being sent to the motors. The firmware's normalization of thrust and torque is unknown, and in fact, is learned from data via the input gain matrix $\mathbf{B}(\mathbf{q})$. We collected 12 state-control trajectories by flying the quadrotor from a starting pose to 12 different poses using a PID controller provided by the PX4 flight controller [82]. The trajectories were used to generate a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{q}_{0:N}^{(i)}, \dot{\mathbf{q}}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ with $N = 1$ and $D = 10000$. We trained our model as described in Section V-D for 5000 steps.

The trained model was used with the control policy in Section VII-C to track different trajectories: a vertical circle, a vertical lemniscate, and a 3-D piecewise-linear trajectories. Figs. 12 and 13 show that we achieve better tracking performance using our learned dynamics model and energy-based control compared to the nominal model and the geometric controller in [79]. The tracking errors of our controller with a learned model improve by 2 – 4 times compared to those of geometric control based on the nominal model, as shown in Table II.

2) *Learning Quadrotor Dynamics With Extra Payload:* In this section, we demonstrate that after our dynamics model is trained, if there is a change in the quadrotor dynamics, e.g., an extra payload is added, we are able to update the dynamics quickly starting from the previously trained model. We attached a coffee can to the quadrotor frame [see Fig. 11(c)] to change the mass and inertia matrix of the robot. We, then, collected a new dataset by driving the quadrotor to 12 different poses, and trained our dynamics model for only 100 steps, initialized with the trained model in Section VII-E1.

TABLE II
POSITION TRACKING ERRORS USING NOMINAL AND LEARNED QUADROTOR MODELS WITH AND WITHOUT PAYLOAD

Model	Train with payload	Test with payload	Circle	Lemniscate	Piecewise-linear
Nominal	-	No	0.26 (m)	0.52 (m)	0.62 (m)
Learned	No	No	0.13(m)	0.14(m)	0.22(m)
Learned	No	Yes	0.20 (m)	0.40 (m)	0.30 (m)
Learned	Yes	Yes	0.13 (m)	0.12 (m)	0.21 (m)

The bold values are to emphasize that our method outperforms other baselines, i.e. achieving smaller errors.

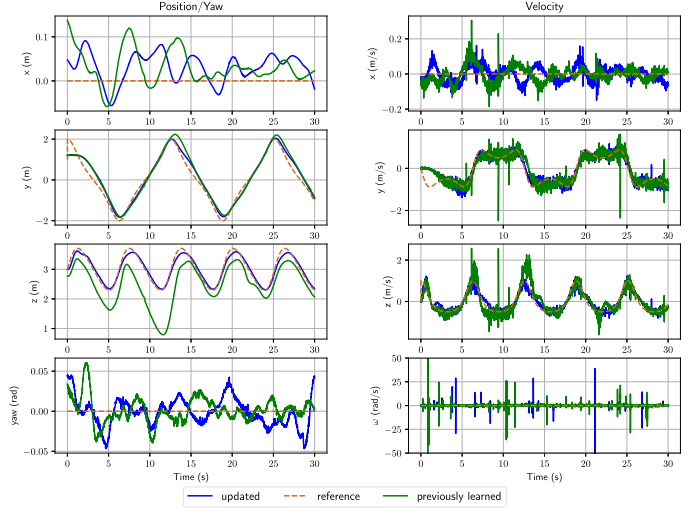


Fig. 15. Tracking a piecewise-linear trajectory with extra payload using our previously learned and updated models.

In the presence of the coffee can payload, the tracking performance of the controller with the previously learned model degrades as shown in Figs. 14 and 15. Meanwhile, after a quick model update, the robot is able to track desired trajectories accurately again. Table II shows that our updated model improves the tracking errors by 1.5–4 times compared to the previously learned model.

VIII. CONCLUSION

This article proposed a neural ODE network design for robot dynamics learning that captures Lie group kinematics, e.g., SE(3), and port-Hamiltonian dynamics constraints by construction. It also developed a general control approach for trajectory tracking based on the learned Lie group port-Hamiltonian dynamics. The learning and control designs were not system specific, and thus, can be applied to different types of robots whose states evolve on Lie group. These techniques had the potential to enable robots to quickly adapt their models online, in response to changing operational conditions or structural damage, and continue to maintain stability during autonomous operation. Future work will focus on extending our formulation to allow learning multirigid-body dynamics, handling contact, and online adaptation to disturbances and structural changes in the dynamics.

沿着预测轨迹，即表 I 中的最低标准差值，来自我们没有控制输入和耗散网络的结构化哈密顿模型，验证了所学习的模型遵守能量守恒定律。图 10(d) 和表 I 中的预测误差表明，与其他方法相比，我们的结构化哈密顿模型提供了更好的轨迹预测。

E. Real Quadrotor Experiments

在本节中，我们使用一台真实的四旋翼机器人验证我们的方法，该机器人配备了板载 i7 Intel NUC 计算机和 PX4 飞控器 [见图 11(b)]。四旋翼的姿态和速度由动作捕捉系统提供。

1) *Learning Quadrotor Dynamics After Upgrade:* 我们考虑一种方案，其中四旋翼升级了新的机架和新的机载计算机，从而导致机器人动力学发生变化，我们的目标是从数据中学习这些变化。标称模型是从另一款重量更轻的配有 Raspberry Pi 计算机和 F450 机架的 Raspberry Pi 四旋翼 [见图 11(a)] 的计算机辅助设计 (CAD) 模型获得的，该模型精度较低，并且远未达到我们升级后的四旋翼的未知真实模型。具体来说，对于图 11(b) 中的升级四旋翼，标称质量和惯性矩阵分别为 $\mathbf{M}_{v0} = 1.3\mathbf{I}$ 和 $\mathbf{M}_{\omega0} = \text{diag}([0.12, 0.12, 0.2])$ 。其他标称矩阵设为零： $\mathbf{D}_{v0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$ 、 $\mathbf{D}_{\omega0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$ 、 $\mathbf{V}_0(\mathbf{q}) = \mathbf{0}$ 和 $\mathbf{B}_0(\mathbf{q}) = \mathbf{0}$ 。我们修改了 PX4 固件 [82] 以显示发送到电机的归一化推力和力矩。固件对推力和力矩的归一化是未知的，实际上是通过输入增益矩阵 $\mathbf{B}(\mathbf{q})$ 从数据中学习得到的。我们通过飞行收集了 12 条状态-控制轨迹

训练好的模型与第 VII-C 节中的控制策略一起使用，用于跟踪不同的轨迹：垂直圆、垂直数字 8 曲线以及三维分段线性轨迹。图 12 和图 13 显示，相比于基于名义模型的几何控制器 [79]，使用我们学习到的动力学模型和基于能量的控制可以实现更好的跟踪性能。如表 II 所示，使用学习模型的控制器的跟踪误差相比基于名义模型的几何控制提高了 2 到 4 倍。

2) *Learning Quadrotor Dynamics With Extra Payload:* 在本节中，我们展示了在我们的动力学模型训练完成后，如果四旋翼动力学发生变化，例如增加额外负载，我们能够从先前训练的模型快速更新动力学。我们在四旋翼机架上附加了一个咖啡罐 [见图 11(c)] 来改变机器人质量和惯性矩阵。然后，我们通过将四旋翼机驾驶到 12 个不同的姿态收集了一个新的数据集，并在第 VII-E1 节训练的模型初始化下，仅训练了 100 步我们的动力学模型。

表 II 使用名义和学习到的四旋翼模型在有无有效载荷情况下的位置跟踪误差

Model	Train with payload	Test with payload	Circle	Lemniscate	Piecewise-linear
Nominal	-	No	0.26 (m)	0.52 (m)	0.62 (m)
Learned	No	No	0.13(m)	0.14(m)	0.22(m)
Learned	No	Yes	0.20 (m)	0.40 (m)	0.30 (m)
Learned	Yes	Yes	0.13 (m)	0.12 (m)	0.21 (m)

The bold values are to emphasize that our method outperforms other baselines, i.e. achieving smaller errors.

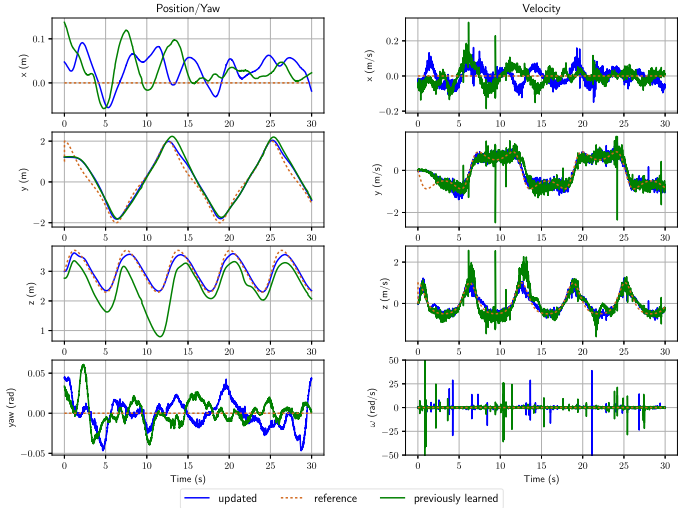


图15. 使用我们之前学习并更新的模型跟踪带额外载荷的分段线性轨迹。

在存在咖啡罐载荷的情况下，如图14和图15所示，使用先前学习的模型的控制器的跟踪性能下降。与此同时，在快速模型更新后，机器人能够再次准确地跟踪期望轨迹。表II显示，与先前学习的模型相比，我们更新的模型将跟踪误差改善了1.5至4倍。

八、结论

本文提出了一种用于机器人动力学学习的神经ODE网络设计，该设计在构建时捕捉李群运动学（例如SE(3)）和端口Hamiltonian动力学约束。它还开发了一种基于学习到的李群端口Hamiltonian动力学的通用轨迹跟踪控制方法。学习和控制设计不是系统特定的，因此可以应用于其状态在李群上演化的不同类型的机器人。这些技术有可能使机器人在面对变化的操作条件或结构损伤时快速在线适应其模型，并在自主操作期间继续维持稳定性。未来工作将重点扩展我们的方案，以允许学习多刚体动力学、处理接触，以及在线适应动力学中的扰动和结构变化。

APPENDIX

A. Implementation Details

We used fully connected neural networks whose architecture is shown as follows. The first number is the input dimension, while the last number is the output dimension. The numbers in between are the hidden layers' dimensions and activation functions. The value of ε_v and ε_ω in (40) is set to 0.01.

- 1) Pendulum
 - a) Input dimension: 9. Action dimension: 1.
 - b) $\mathbf{L}_v(\mathbf{q})$: 9 - 300 Tanh - 300 Tanh - 300 Tanh - 300 Linear - 6.
 - c) $\mathcal{V}_\theta(\mathbf{q})$: 9 - 50 Tanh - 50 Tanh - 50 Linear - 1.
 - d) $\mathbf{B}_\theta(\mathbf{q})$: 9 - 300 Tanh - 300 Tanh - 300 Linear - 3.
- 2) Pybullet quadrotor
 - a) Input dimension: 12. Action dimension: 4.
 - b) $\mathbf{L}_v(\mathbf{q})$ only takes the position $\mathbf{p} \in \mathbb{R}^3$ as input: 3 - 400 Tanh - 400 Tanh - 400 Tanh - 400 Linear - 6.
 - c) $\mathbf{L}_\omega(\mathbf{q})$ only takes the rotation matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ as input: 9 - 400 Tanh - 400 Tanh - 400 Tanh - 400 Linear - 6.
 - d) $\mathcal{V}_\theta(\mathbf{q})$: 12 - 400 Tanh - 400 Tanh - 400 Linear - 1.
 - e) $\mathbf{B}_\theta(\mathbf{q})$: 12 - 400 Tanh - 400 Tanh - 400 Linear - 24.
- 3) Real PX4 quadrotor
 - a) Input dimension: 12. Action dimension: 4.
 - b) $\mathbf{L}_v(\mathbf{q})$ only takes the position $\mathbf{p} \in \mathbb{R}^3$ as input: 3 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - c) $\mathbf{L}_\omega(\mathbf{q})$ only takes the rotation matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ as input: 9 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - d) $\mathbf{D}_{v;\theta}(\mathbf{q})$ only takes the position $\mathbf{p} \in \mathbb{R}^3$ as input: 3 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - e) $\mathbf{D}_{\omega;\theta}(\mathbf{q})$ only takes the rotation matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ as input: 9 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - f) $V(\mathbf{q})$: 12 - 20 Tanh - 20 Tanh - 20 Linear - 1.
 - g) $\mathbf{B}_\theta(\mathbf{q})$: 12 - 20 Tanh - 20 Tanh - 20 Linear - 24.

B. Derivation of Hamiltonian Dynamics on SE(3) From Hamiltonian Dynamics on a Matrix Lie Group

The Hamiltonian dynamics on SE(3) in (30) can be obtained from the general matrix Lie group Hamiltonian dynamics in (18) by computing explicit expressions for the terms $\text{ad}_\xi^*(\mathbf{p})$ and $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta})$ with $\boldsymbol{\eta} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}}$.

To obtain an explicit expression for $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta})$, we use (6) and the pairing in Definition 1:

$$\begin{aligned} \langle \mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta}), \boldsymbol{\xi} \rangle &= \langle \boldsymbol{\eta}, \mathbf{T}_e \mathbf{L}_q(\boldsymbol{\xi}) \rangle = \langle \boldsymbol{\eta}, \mathbf{q} \boldsymbol{\xi} \rangle \\ &= \text{tr}(\boldsymbol{\eta}^\top \mathbf{q} \boldsymbol{\xi}) = \langle \mathbf{q}^\top \boldsymbol{\eta}, \boldsymbol{\xi} \rangle. \end{aligned} \quad (80)$$

Thus, $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta}) = \mathbf{P}_{\mathbf{g}^*}(\mathbf{q}^\top \boldsymbol{\eta})$, where $\mathbf{P}_{\mathbf{g}^*}$ is an orthogonal projector on $\mathbf{g}^*$ [83, Def. 3.60], which depends on the specific matrix Lie group. For example, on SE(3) [84] with $\mathbf{A} \in \mathbb{R}^{3 \times 3}$,

$\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$, and $c \in \mathbb{R}$

$$\mathbf{P}_{\mathbf{g}^*} \left(\begin{bmatrix} \mathbf{A} & \mathbf{a} \\ \mathbf{b}^\top & c \end{bmatrix} \right) = \begin{bmatrix} \frac{1}{2}(\mathbf{A} - \mathbf{A}^\top) & \mathbf{a} \\ \mathbf{0}^\top & 0 \end{bmatrix}. \quad (81)$$

To obtain an explicit expression for $\text{ad}_\xi^*(\mathbf{p})$, we use Definition 9 and the pairing in Definition 1 as

$$\begin{aligned} \langle \text{ad}_\xi^*(\mathbf{p}), \boldsymbol{\psi} \rangle &= \langle \mathbf{p}, \text{ad}_\xi(\boldsymbol{\psi}) \rangle = \langle \mathbf{p}, [\boldsymbol{\xi}, \boldsymbol{\psi}] \rangle \\ &= \text{tr}(\mathbf{p}^\top (\boldsymbol{\xi} \boldsymbol{\psi} - \boldsymbol{\psi} \boldsymbol{\xi})) = \langle [\boldsymbol{\xi}^\top, \mathbf{p}], \boldsymbol{\psi} \rangle. \end{aligned} \quad (82)$$

Thus, $\text{ad}_\xi^*(\mathbf{p}) = \mathbf{P}_{\mathbf{g}^*}([\boldsymbol{\xi}^\top, \mathbf{p}])$.

1) Expression for $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta})$ on SE(3): On SE(3), we have

$$\mathbf{q} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \boldsymbol{\xi} = \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \boldsymbol{\eta} = \begin{bmatrix} \boldsymbol{\eta}_R & \boldsymbol{\eta}_p \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (83)$$

and

$$\begin{aligned} \langle \boldsymbol{\eta}, w \mathbf{q} \boldsymbol{\xi} \rangle &= \langle \boldsymbol{\eta}_R, \mathbf{R} \hat{\boldsymbol{\omega}} \rangle + \langle \boldsymbol{\eta}_p, \mathbf{R} \mathbf{v} \rangle \\ &= \left\langle \frac{1}{2}(\mathbf{R}^\top \boldsymbol{\eta}_R - \boldsymbol{\eta}_R^\top \mathbf{R}), \hat{\boldsymbol{\omega}} \right\rangle + \langle \mathbf{R}^\top \boldsymbol{\eta}_p, \mathbf{v} \rangle \\ &= \left\langle \begin{bmatrix} \frac{1}{2}(\mathbf{R}^\top \boldsymbol{\eta}_R - \boldsymbol{\eta}_R^\top \mathbf{R}) & \mathbf{R}^\top \boldsymbol{\eta}_p \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix} \right\rangle \\ &= \langle \mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta}), \boldsymbol{\xi} \rangle \end{aligned} \quad (84)$$

where we used the properties $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$ and $\text{tr}(\hat{\mathbf{x}}\mathbf{A}) = \frac{1}{2}\text{tr}(\hat{\mathbf{x}}(\mathbf{A} - \mathbf{A}^\top))$ of the hat map in the second equality.

2) Expression for $\text{ad}_\xi^*(\mathbf{p})$ on SE(3): On $\mathfrak{se}(3)$, we have

$$\mathbf{p} = \begin{bmatrix} \hat{\mathbf{a}} & \mathbf{b} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \boldsymbol{\xi} = \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \boldsymbol{\psi} = \begin{bmatrix} \hat{\mathbf{c}} & \mathbf{d} \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (85)$$

and

$$\begin{aligned} \langle \mathbf{p}, [\boldsymbol{\xi}, \boldsymbol{\psi}] \rangle &= \left\langle \begin{bmatrix} \hat{\mathbf{a}} & \mathbf{b} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} [\hat{\boldsymbol{\omega}}, \hat{\mathbf{c}}] & \hat{\boldsymbol{\omega}} \mathbf{d} + \hat{\mathbf{v}} \mathbf{c} \\ \mathbf{0} & 0 \end{bmatrix} \right\rangle \\ &= \langle \hat{\mathbf{a}}, [\hat{\boldsymbol{\omega}}, \hat{\mathbf{c}}] \rangle + \langle \mathbf{b}, \hat{\boldsymbol{\omega}} \mathbf{d} + \hat{\mathbf{v}} \mathbf{c} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}], \hat{\mathbf{c}} \rangle + \langle \mathbf{b}, \hat{\mathbf{v}} \mathbf{c} \rangle + \langle \mathbf{b}, \hat{\boldsymbol{\omega}} \mathbf{d} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}], \hat{\mathbf{c}} \rangle - \text{tr}(\mathbf{v} \mathbf{b}^\top \hat{\mathbf{c}}) + \langle \hat{\boldsymbol{\omega}}^\top \mathbf{b}, \mathbf{d} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}], \hat{\mathbf{c}} \rangle + \left\langle \frac{1}{2}(\mathbf{v} \mathbf{b}^\top - \mathbf{b} \mathbf{v}^\top), \hat{\mathbf{c}} \right\rangle + \langle \hat{\mathbf{b}} \boldsymbol{\omega}, \mathbf{d} \rangle \\ &= \left\langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}] + \frac{1}{2}[\hat{\mathbf{b}}, \hat{\mathbf{v}}], \hat{\mathbf{c}} \right\rangle + \langle \hat{\mathbf{b}} \boldsymbol{\omega}, \mathbf{d} \rangle \\ &= \left\langle \begin{bmatrix} [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}] + \frac{1}{2}[\hat{\mathbf{b}}, \hat{\mathbf{v}}] & \hat{\mathbf{b}} \boldsymbol{\omega} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} \hat{\mathbf{c}} & \mathbf{d} \\ \mathbf{0}^\top & 0 \end{bmatrix} \right\rangle \\ &= \langle \text{ad}_\xi^*(\mathbf{p}), \boldsymbol{\psi} \rangle \end{aligned} \quad (86)$$

where we used the hat map properties $\hat{\mathbf{x}}^\top = -\hat{\mathbf{x}}$, $\hat{\mathbf{x}}\mathbf{y} = -\hat{\mathbf{y}}\mathbf{x}$, and $\text{tr}(\hat{\mathbf{x}}\mathbf{A}) = \frac{1}{2}\text{tr}(\hat{\mathbf{x}}(\mathbf{A} - \mathbf{A}^\top))$.

3) Consistency Between Hamiltonian Dynamics on a Matrix Lie Group and on SE(3): Denote the momentum in (17) as

附录

A. Implementation Details

我们使用了全连接神经网络，其架构如下所示。第一个数字是输入维度，而最后一个数字是输出维度。中间的数字是隐藏层的维度和激活函数。(40) 中 ε_v 和 ε_ω 的值设为 0.01。

1) 钟摆

a) 输入维度: 9。动作维度: 1。b) $\mathbf{L}(\mathbf{q})$: 9 - 300 Tanh - 300 Tanh - 300 Tanh - 300 线性 - 6。c) $\mathcal{V}_\theta(\mathbf{q})$: 9 - 50 Tanh - 50 Tanh - 50 线性 - 1。d) $\mathbf{B}_\theta(\mathbf{q})$: 9 - 300 Tanh - 300 Tanh - 300 线性 - 3。2) Pybullet 四旋翼

a) 输入维度: 12。动作维度: 4。b) $\mathbf{L}_v(\mathbf{q})$ 仅将位置 $\mathbf{p} \in \mathbb{R}^3$ 作为输入: 3 - 400 Tanh - 400 Tanh - 400 Tanh - 400 Linear - 6。c) $\mathbf{L}_\omega(\mathbf{q})$ 仅将旋转矩阵 $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ 作为输入: 9 - 400 Tanh - 400 Tanh - 400 Tanh - 400 Linear - 6。d) $\mathcal{V}_v(\mathbf{q})$: 12 - 400 Tanh - 400 Tanh - 400 Linear - 1。e) $\mathbf{B}_\theta(\mathbf{q})$: 12 - 400 Tanh - 400 Tanh - 400 Linear - 24。

3) 真实 PX4 四旋翼

a) 输入维度: 12。动作维度: 4。b) $\mathbf{L}_v(\mathbf{q})$ 仅以位置 $\mathbf{p} \in \mathbb{R}^3$ 作为输入: 3 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6。c) $\mathbf{L}_\omega(\mathbf{q})$ 仅以旋转矩阵 $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ 作为输入: 9 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6。d) $\mathbf{D}_{v,\theta}(\mathbf{q})$ 仅以位置 $\mathbf{p} \in \mathbb{R}^3$ 作为输入: 3 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6。e) $\mathbf{D}_{\omega,\theta}(\mathbf{q})$ 仅以旋转矩阵 $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ 作为输入: 9 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6。f) $V(\mathbf{q})$: 12 - 20 Tanh - 20 Tanh - 20 Linear - 1。g) $\mathbf{B}_\theta(\mathbf{q})$: 12 - 20 Tanh - 20 Tanh - 20 Linear - 24。

B. Derivation of Hamiltonian Dynamics on SE(3) From Hamiltonian Dynamics on a Matrix Lie Group

方程 (30) 中的 SE(3) 哈密顿动力学可以通过对矩阵李群哈密顿动力学 (18) 中的项 $\text{ad}_\xi^*(\mathbf{p})$ 和 $\mathbf{T}_e^* \mathbf{L}_q(\eta)$ 计算出 $\eta = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}}$ 的显式表达式来获得。

为了得到 $\mathbf{T}_e^* \mathbf{L}_q(\eta)$ 的显式表达式，我们使用 (6) 和定义 1 中的配对：

$$\begin{aligned} \langle \mathbf{T}_e^* \mathbf{L}_q(\eta), \xi \rangle &= \langle \eta, \mathbf{T}_e \mathbf{L}_q(\xi) \rangle = \langle \eta, \mathbf{q} \xi \rangle \\ &= \text{tr}(\eta^\top \mathbf{q} \xi) = \langle \mathbf{q}^\top \eta, \xi \rangle. \end{aligned} \quad (80)$$

因此， $\mathbf{T}_e^* \mathbf{L}_q(\eta) = \mathbf{P}_{\mathbf{g}^*}(\mathbf{q}^\top \eta)$ ，其中 $\mathbf{P}_{\mathbf{g}^*}$ 是 $\mathbf{g}^*$ 上的正交投影算子 [83, 定义 3.60]，它依赖于具体的矩阵李群。例如，在 SE(3) [84] 上有 $\mathbf{A} \in \mathbb{R}^{3 \times 3}$ ，

$\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ ，和 $c \in \mathbb{R}$

$$\mathbf{P}_{\mathbf{g}^*} \left(\begin{bmatrix} \mathbf{A} & \mathbf{a} \\ \mathbf{b}^\top & c \end{bmatrix} \right) = \begin{bmatrix} \frac{1}{2}(\mathbf{A} - \mathbf{A}^\top) & \mathbf{a} \\ \mathbf{0}^\top & 0 \end{bmatrix}. \quad (81)$$

为了得到 $\text{ad}_\xi^*(\mathbf{p})$ 的显式表达式，我们使用定义 9 和定义 1 中的配对为

$$\begin{aligned} \langle \text{ad}_\xi^*(\mathbf{p}), \psi \rangle &= \langle \mathbf{p}, \text{ad}_\xi(\psi) \rangle = \langle \mathbf{p}, [\xi, \psi] \rangle \\ &= \text{tr}(\mathbf{p}^\top (\xi \psi - \psi \xi)) = \langle [\xi^\top, \mathbf{p}], \psi \rangle. \end{aligned} \quad (82)$$

因此， $\text{ad}_\xi^*(\mathbf{p}) = \mathbf{P}_{\mathbf{g}^*}([\xi^\top, \mathbf{p}])$ 。

1) Expression for $\mathbf{T}_e^* \mathbf{L}_q(\eta)$ on SE(3): 在 SE(3) 上，我们有

$$\mathbf{q} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \xi = \begin{bmatrix} \hat{\omega} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \eta = \begin{bmatrix} \eta_{\mathbf{R}} & \eta_{\mathbf{p}} \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (83)$$

和

$$\begin{aligned} \langle \eta, w \mathbf{q} \xi \rangle &= \langle \eta_{\mathbf{R}}, \mathbf{R} \hat{\omega} \rangle + \langle \eta_{\mathbf{p}}, \mathbf{R} \mathbf{v} \rangle \\ &= \left\langle \frac{1}{2}(\mathbf{R}^\top \eta_{\mathbf{R}} - \eta_{\mathbf{R}}^\top \mathbf{R}), \hat{\omega} \right\rangle + \langle \mathbf{R}^\top \eta_{\mathbf{p}}, \mathbf{v} \rangle \\ &= \left\langle \begin{bmatrix} \frac{1}{2}(\mathbf{R}^\top \eta_{\mathbf{R}} - \eta_{\mathbf{R}}^\top \mathbf{R}) & \mathbf{R}^\top \eta_{\mathbf{p}} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} \hat{\omega} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix} \right\rangle \\ &= \langle \mathbf{T}_e^* \mathbf{L}_q(\eta), \xi \rangle \end{aligned} \quad (84)$$

在第二个等式中，我们使用了帽映射的性质

$\text{tr}(\mathbf{A}\mathbf{B}) = \text{tr}(\mathbf{B}\mathbf{A})$ 和 $\text{tr}(\hat{\mathbf{x}}\mathbf{A}) = \frac{1}{2}\text{tr}(\hat{\mathbf{x}}(\mathbf{A} - \mathbf{A}^\top))$ 。

2) Expression for $\text{ad}_\xi^*(\mathbf{p})$ on SE(3): 在 $\mathfrak{se}(3)$ ，我们有

$$\mathbf{p} = \begin{bmatrix} \hat{\mathbf{a}} & \mathbf{b} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \xi = \begin{bmatrix} \hat{\omega} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \psi = \begin{bmatrix} \hat{\mathbf{c}} & \mathbf{d} \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (85)$$

和

$$\begin{aligned} \langle \mathbf{p}, [\xi, \psi] \rangle &= \left\langle \begin{bmatrix} \hat{\mathbf{a}} & \mathbf{b} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} [\hat{\omega}, \hat{\mathbf{c}}] & \hat{\omega} \mathbf{d} + \hat{\mathbf{v}} \mathbf{c} \\ \mathbf{0} & 0 \end{bmatrix} \right\rangle \\ &= \langle \hat{\mathbf{a}}, [\hat{\omega}, \hat{\mathbf{c}}] \rangle + \langle \mathbf{b}, \hat{\omega} \mathbf{d} + \hat{\mathbf{v}} \mathbf{c} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\omega}], \hat{\mathbf{c}} \rangle + \langle \mathbf{b}, \hat{\mathbf{v}} \mathbf{c} \rangle + \langle \mathbf{b}, \hat{\omega} \mathbf{d} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\omega}], \hat{\mathbf{c}} \rangle - \text{tr}(\mathbf{v} \mathbf{b}^\top \hat{\mathbf{c}}) + \langle \hat{\omega}^\top \mathbf{b}, \mathbf{d} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\omega}], \hat{\mathbf{c}} \rangle + \left\langle \frac{1}{2}(\mathbf{v} \mathbf{b}^\top - \mathbf{b} \mathbf{v}^\top), \hat{\mathbf{c}} \right\rangle + \langle \hat{\mathbf{b}} \omega, \mathbf{d} \rangle \\ &= \left\langle [\hat{\mathbf{a}}, \hat{\omega}] + \frac{1}{2}[\hat{\mathbf{b}}, \hat{\mathbf{v}}], \hat{\mathbf{c}} \right\rangle + \langle \hat{\mathbf{b}} \omega, \mathbf{d} \rangle \\ &= \left\langle \begin{bmatrix} [\hat{\mathbf{a}}, \hat{\omega}] + \frac{1}{2}[\hat{\mathbf{b}}, \hat{\mathbf{v}}] & \hat{\mathbf{b}} \omega \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} \hat{\mathbf{c}} & \mathbf{d} \\ \mathbf{0}^\top & 0 \end{bmatrix} \right\rangle \\ &= \langle \text{ad}_\xi^*(\mathbf{p}), \psi \rangle \end{aligned} \quad (86)$$

我们使用了帽子映射属性 $\hat{\mathbf{x}}^\top = -\hat{\mathbf{x}}$ 、 $\hat{\mathbf{x}}\mathbf{y} = -\hat{\mathbf{y}}\mathbf{x}$ 和 $\text{tr}(\hat{\mathbf{x}}\mathbf{A}) = \frac{1}{2}\text{tr}(\hat{\mathbf{x}}(\mathbf{A} - \mathbf{A}^\top))$ 的地方。

3) Consistency Between Hamiltonian Dynamics on a Matrix Lie Group and on SE(3): 将(17)中的动量表示为

$\mathbf{p}_\xi = \begin{bmatrix} \hat{\mathbf{p}}_\omega & \mathbf{p}_v \\ \mathbf{0}^\top & 0 \end{bmatrix}$ and the momentum in (26) as $\mathbf{p}_\zeta = \begin{bmatrix} \mathbf{p}_\omega \\ \mathbf{p}_v \end{bmatrix}$, where $\xi = \hat{\zeta}$. Let $\mathbf{p}_\omega = \frac{\partial \mathcal{L}}{\partial \omega} = \hat{\mu}$. By the chain rule, we have

$$\mathbf{p}_{\omega_i} = \frac{\partial \mathcal{L}}{\partial \omega_i} = \left\langle \hat{\mu}, \frac{\partial \hat{\omega}}{\partial \omega_i} \right\rangle = 2\mu_i \quad (87)$$

where $\omega = [\omega_1 \ \omega_2 \ \omega_3]^\top$, $\mu = [\mu_1 \ \mu_2 \ \mu_3]^\top$, or

$$\mathbf{p}_\omega = \frac{1}{2} \hat{\mathbf{p}}_\omega. \quad (88)$$

Therefore, we have

$$\mathbf{p}_\xi = \begin{bmatrix} \frac{1}{2} \hat{\mathbf{p}}_\omega & \mathbf{p}_v \\ \mathbf{0}^\top & 0 \end{bmatrix}, \text{ i.e., } \mathbf{a} = \frac{1}{2} \hat{\mathbf{p}}_\omega, \mathbf{b} = \mathbf{p}_v \quad (89)$$

leading to

$$\text{ad}_\xi^*(\mathbf{p}_\xi) = \begin{bmatrix} \left(\frac{1}{2} \hat{\mathbf{p}}_\omega \omega + \frac{1}{2} \hat{\mathbf{p}}_v \mathbf{v} \right)^\wedge & \hat{\mathbf{p}}_v \omega \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (90)$$

By plugging in (89), (84), and (90) in the matrix Lie group Hamiltonian dynamics (18), we obtain the Hamiltonian dynamics on SE(3) in (30).

REFERENCES

- [1] L. Ljung, "System identification," in *Wiley Encyclopedia of Electrical and Electronics Engineering*. Hoboken, NJ, USA: Wiley, 1999, pp. 1–19.
- [2] D. Nguyen-Tuong and J. Peters, "Model learning for robot control: A survey," *Cogn. Process.*, vol. 12, pp. 319–340, 2011.
- [3] M. Deisenroth and C. Rasmussen, "PILCO: A model-based and data-efficient approach to policy search," in *Proc. Int. Conf. Mach. Learn.*, 2011, pp. 465–472.
- [4] G. Williams et al., "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 1714–1721.
- [5] M. Raissi, P. Perdikaris, and G. Karniadakis, "Multistep neural networks for data-driven discovery of nonlinear dynamical systems," 2018, *arXiv:1801.01236*.
- [6] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4759–4770.
- [7] M. Lutter and J. Peters, "Combining physics and deep learning to learn continuous-time dynamics models," *Int. J. Robot. Res.*, vol. 42, pp. 83–107, 2023.
- [8] J. Gupta, K. Menda, Z. Manchester, and M. Kochenderfer, "A general framework for structured learning of mechanical systems," 2019, *arXiv:1902.08705*.
- [9] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho, "Lagrangian neural networks," in *Proc. ICLR Workshop Integration Deep Neural Models Differ. Equ.*, 2020.
- [10] S. Greydanus, M. Dzamba, and J. Yosinski, "Hamiltonian neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 15379–15389.
- [11] Z. Chen, J. Zhang, M. Arjovsky, and L. Bottou, "Symplectic recurrent neural networks," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [12] M. Roehrl, T. Runkler, V. Brandstetter, M. Tokic, and S. Obermayer, "Modeling system dynamics with physics-informed neural networks based on Lagrangian mechanics," *IFAC-PapersOnLine*, vol. 53, pp. 9195–9200, 2020.
- [13] Y. Lu, S. Lin, G. Chen, and J. Pan, "ModLaNets: Learning generalisable dynamics via modularity and physical inductive bias," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 14384–14397.
- [14] C. Neary and U. Topcu, "Compositional learning of dynamical system models using port-Hamiltonian neural networks," in *Proc. Learn. Dyn. Control Conf.*, 2023, pp. 679–691.
- [15] A. Sanchez-Gonzalez et al., "Graph networks as learnable physics engines for inference and control," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4470–4479.
- [16] M. Lutter, C. Ritter, and J. Peters, "Deep lagrangian networks: Using physics as model prior for deep learning," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [17] B. Hall, *Lie Groups, Lie Algebras, and Representations*. New York, NY, USA: Springer, 2013.
- [18] K. Lynch and F. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [19] L. Falorsi and P. Forré, "Neural ordinary differential equations on manifolds," 2020, *arXiv:2006.06663*.
- [20] K. Elamvazhuthi, X. Zhang, S. Oymak, and F. Pasqualetti, "Learning on manifolds: Universal approximations properties using geometric controllability conditions for neural ODEs," in *Proc. Learn. Dyn. Control Conf.*, 2023, pp. 1–11.
- [21] A. Lou et al., "Neural manifold ordinary differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 17548–17558.
- [22] Y. Wotte, F. Califano, and S. Stramigioli, "Optimal potential shaping on SE(3) via neural ordinary differential equations on Lie groups," *Int. J. Robot. Res.*, 2024, doi: [10.1177/02783649241256044](https://doi.org/10.1177/02783649241256044).
- [23] A. I. Lurie, *Analytical Mechanics*. Berlin, Germany: Springer, 2013.
- [24] D. Holm, *Geometric Mechanics*. Singapore: World Scientific, 2008.
- [25] T. Bertalan, F. Dietrich, I. Mezić, and I. Kevrekidis, "On learning Hamiltonian systems from data," *Chaos, Interdiscipl. J. Nonlinear Sci.*, vol. 12, 2019, Art. no. 121107.
- [26] M. Finzi, K. A. Wang, and A. Wilson, "Simplifying Hamiltonian and Lagrangian neural networks via explicit constraints," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 13880–13889.
- [27] Y. D. Zhong, B. Dey, and A. Chakraborty, "Symplectic ODE-Net: Learning Hamiltonian dynamics with control," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [28] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating Scientific Knowledge with Machine Learning for Engineering and Environmental Systems," *ACM Comput. Surv.*, vol. 55, no. 4, 2022, Art. no. 66, doi: [10.1145/3514228](https://doi.org/10.1145/3514228).
- [29] A. Maciejewski, "Hamiltonian formalism for euler parameters," *Celestial Mechanics*, vol. 37, pp. 47–57, 1985.
- [30] R. Shivarama and E. Fahrendthold, "Hamilton's equations with euler parameters for rigid body dynamics modeling," *J. Dyn. Syst., Meas., Control*, vol. 126, pp. 124–130, 2004.
- [31] R. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6572–6583.
- [32] T. Duong and N. Atanasov, "Hamiltonian-based neural ODE networks on the SE(3) manifold for dynamics learning and control," in *Proc. Robot., Sci. Syst.*, 2021.
- [33] T. Lee, M. Leok, and N. H. McClamroch, *Global Formulations of Lagrangian and Hamiltonian Dynamics on Manifolds*. Berlin, Germany: Springer, 2017.
- [34] Y. D. Zhong, B. Dey, and A. Chakraborty, "Dissipative SymODEN: Encoding Hamiltonian dynamics with dissipation and control into deep learning," in *Proc. ICLR Workshop Integration Deep Neural Models Differ. Equ.*, 2020.
- [35] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Sci. Robot.*, vol. 8, 2023, Art. no. eadg1462.
- [36] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll, "Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms," *IEEE Robot. Automat. Lett.*, vol. 8, no. 4, pp. 2397–2404, Apr. 2023.
- [37] D. Scaramuzza and E. Kaufmann, "Learning agile, vision-based drone flight: From simulation to reality," in *Proc. Int. Symp. Robot. Res.*, 2022, pp. 11–18.
- [38] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Sci. Robot.*, vol. 6, 2021, Art. no. eabg5810.
- [39] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine, "How to train your robot with deep reinforcement learning: Lessons we have learned," *Int. J. Robot. Res.*, vol. 40, pp. 698–721, 2021.
- [40] T. X. Nghiem et al., "Physics-informed machine learning for modeling and control of dynamical systems," in *Proc. Amer. Control Conf.*, 2023, pp. 3735–3750.
- [41] F. Djeumou, C. Neary, E. Goubault, S. Putot, and U. Topcu, "Neural networks with physics-informed architectures and constraints for dynamical systems modeling," in *Proc. Learn. Dyn. Control Conf.*, 2022, pp. 263–277.
- [42] A. Thorpe, C. Neary, F. Djeumou, M. Oishi, and U. Topcu, "Physics-informed kernel embeddings: Integrating prior system knowledge with data-driven control," in *Proc. Amer. Control Conf.*, 2024.

$\mathbf{p}_\xi = \begin{bmatrix} \mathbf{p}_{\hat{\omega}} & \mathbf{p}_v \\ \mathbf{0}^\top & 0 \end{bmatrix}$ 和 (26) 中的动量为 $\mathbf{p}_\zeta = \begin{bmatrix} \mathbf{p}_\omega \\ \mathbf{p}_v \end{bmatrix}$, 其中 $\xi = \hat{\zeta}$ 。令 $\mathbf{p}_{\hat{\omega}} = \frac{\partial \mathcal{L}}{\partial \hat{\omega}} = \hat{\boldsymbol{\mu}}$ 。根据链式法则, 我们有

$$\mathbf{p}_{\omega_i} = \frac{\partial \mathcal{L}}{\partial \omega_i} = \left\langle \hat{\boldsymbol{\mu}}, \frac{\partial \hat{\omega}}{\partial \omega_i} \right\rangle = 2\mu_i \quad (87)$$

在哪里 $\boldsymbol{\omega} = [\omega_1 \ \omega_2 \ \omega_3]^\top$, $\boldsymbol{\mu} = [\mu_1 \ \mu_2 \ \mu_3]^\top$, 或

$$\mathbf{p}_{\hat{\omega}} = \frac{1}{2} \hat{\mathbf{p}}_\omega. \quad (88)$$

因此, 我们有

$$\mathbf{p}_\xi = \begin{bmatrix} \frac{1}{2} \hat{\mathbf{p}}_\omega & \mathbf{p}_v \\ \mathbf{0}^\top & 0 \end{bmatrix}, \text{ i.e., } \mathbf{a} = \frac{1}{2} \hat{\mathbf{p}}_\omega, \mathbf{b} = \mathbf{p}_v \quad (89)$$

导致

$$\text{ad}_\xi^*(\mathbf{p}_\xi) = \begin{bmatrix} \left(\frac{1}{2} \hat{\mathbf{p}}_\omega \boldsymbol{\omega} + \frac{1}{2} \hat{\mathbf{p}}_v \mathbf{v} \right)^\wedge & \hat{\mathbf{p}}_v \boldsymbol{\omega} \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (90)$$

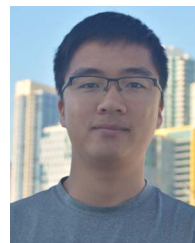
通过将 (89)、(84) 和 (90) 代入矩阵李群哈密顿动力学 (18), 我们得到 SE(3) 上的哈密顿动力学 (30)。

参考文献

[1] L. Ljung, “系统辨识,” 收录于 *Wiley Encyclopedia of Electrical and Electronics Engineering*. 新泽西州霍博肯, 美国: Wiley, 1999, 第 1–19 页。[2] D. Nguyen-Tuong 和 J. Peters, “用于机器人控制的模型学习: 综述,” *Cogn. Process.*, 第 12 卷, 第 319–340 页, 2011。[3] M. Deisenroth 和 C. Rasmussen, “PILCO: 一种基于模型和数据高效的策略搜索方法,” 收录于 *Proc. Int. Conf. Mach. Learn.*, 2011, 第 465–472 页。[4] G. Williams 等, “基于信息理论的模型预测控制在基于模型强化学习中的应用,” 收录于 *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, 第 1714–1721 页。[5] M. Raissi, P. Perdikaris, 和 G. Karniadakis, “用于数据驱动发现非线性动力系统的多步神经网络,” 2018, *arXiv:1801.01236*。[6] K. Chua, R. Calandra, R. McAllister, 和 S. Levine, “使用概率动力学模型在少量试验中进行深度强化学习,” 收录于 *Proc. Adv. Neural Inf. Process. Syst.*, 2018, 第 4759–4770 页。[7] M. Lutter 和 J. Peters, “结合物理与深度学习以学习连续时间动力学模型,” *Int. J. Robot. Res.*, 第 42 卷, 第 83–107 页, 2023。[8] J. Gupta, K. Men

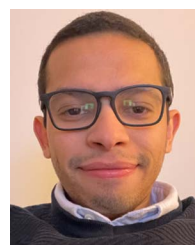
[16] M. Lutter, C. Ritter 和 J. Peters, “深拉格朗日网络: 使用物理作为深度学习的模型先验,” 载于 *Proc. Int. Conf. Learn. Representations*, 2018。[17] B. Hall, *Lie Groups, Lie Algebras, and Representations*. 美国纽约, NY: Springer, 2013。[18] K. Lynch 和 F. Park, *Modern Robotics: Mechanics, Planning, and Control*. 英国剑桥: 剑桥大学出版社, 2017。[19] L. Falorsi 和 P. Forré, “流形上的神经常微分方程,” 2020, *arXiv:2006.06663*。[20] K. Elamvazhuthi, X. Zhang, S. Oymak 和 F. Pasqualetti, “流形学习: 使用神经ODE的几何可控性条件的通用逼近性质,” 载于 *Proc. Learn. Dyn. Control Conf.*, 2023, 第1–11页。[21] A. Lou 等, “神经流形常微分方程,” 载于 *Proc. Adv. Neural Inf. Process. Syst.*, 2020, 第17548–17558页。[22] Y. Wotte, F. Califano 和 S. Stramigioli, “通过李群上神经常微分方程在SE(3)上的最优势能塑形,” *Int. J. Robot. Res.*, 2024, doi: 10.1177/02783649241256044。[23] A. I. Lurie, *Analytical Mechanics*. 德国柏林: Springer, 2013。[24] D. Holm, *Geometric Mechanics*. 新加坡: 世界科学出版社

- [43] L. Ruthotto and E. Haber, "Deep neural networks motivated by partial differential equations," *J. Math. Imag. Vis.*, vol. 62, pp. 352–364, 2019.
- [44] R. Wang, R. Walters, and R. Yu, "Incorporating symmetry into deep dynamics models for improved generalization," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [45] M. Lutter, K. Listmann, and J. Peters, "Deep Lagrangian networks for end-to-end learning of energy-based control for under-actuated systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 7718–7725.
- [46] T. Beckers, J. Seidman, P. Perdikaris, and G. Pappas, "Gaussian process port-Hamiltonian systems: Bayesian learning with physics prior," in *Proc. IEEE Conf. Decis. Control*, 2022, pp. 1447–1453.
- [47] T. Beckers, "Data-driven Bayesian control of port-Hamiltonian systems," in *Proc. IEEE Conf. Decis. Control*, 2023, pp. 8708–8713.
- [48] B. Leimkuhler and S. Reich, *Simulating Hamiltonian Dynamics*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [49] P. Toth, D. J. Rezende, A. Jaegle, S. Racanière, A. Botev, and I. Higgins, "Hamiltonian generative networks," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [50] J. Mason, C. Allen-Blanchette, N. Zolman, E. Davison, and N. Leonard, "Learning interpretable dynamics from images of a freely rotating 3D rigid body," in *Proc. Third Symp. Knowl. guided Mach. Learn.*, 2022. [Online]. Available: <https://aaai.org/conference/fall-symposia/fss22/>.
- [51] A. Van Der Schaft and D. Jeltsema, "Port-Hamiltonian systems theory: An introductory overview," *Found. Trends Syst. Control*, vol. 1, pp. 173–378, 2014.
- [52] J. E. Marsden and M. West, "Discrete mechanics and variational integrators," *Acta Numerica*, vol. 10, pp. 357–514, 2001.
- [53] A. Havens and G. Chowdhary, "Forced variational integrator networks for prediction and control of mechanical systems," in *Proc. Learn. Dyn. Control*, 2021, pp. 1142–1153.
- [54] V. Duruisseaux, T. P. Duong, M. Leok, and N. Atanasov, "Lie group forced variational integrator networks for learning and control of robot systems," in *Proc. Learn. Dyn. Control Conf.*, 2023, pp. 731–744.
- [55] R. Chen and M. Tao, "Data-driven prediction of general Hamiltonian dynamics via learning exactly-symplectic maps," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1717–1727.
- [56] O. So, G. Li, E. Theodorou, and M. Tao, "Data-driven discovery of non-newtonian astronomy via learning non-euclidean hamiltonian," in *Proc. ICML Mach. Learn. Phys. Sci. Workshop*, 2022.
- [57] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [58] R. Ortega, M. Spong, F. Gómez-Estern, and G. Blankenstein, "Stabilization of a class of underactuated mechanical systems via interconnection and damping assignment," *IEEE Trans. Autom. Control*, vol. 47, no. 8, pp. 1218–1233, Aug. 2002.
- [59] J. Á. Acosta, M. Sanchez, and A. Ollero, "Robust control of underactuated aerial manipulators via IDA-PBC," in *Proc. IEEE Conf. Decis. Control*, 2014, pp. 673–678.
- [60] O. Cieza and J. Reger, "IDA-PBC for underactuated mechanical systems in implicit port-hamiltonian representation," in *Proc. Eur. Control Conf.*, 2019, pp. 614–619.
- [61] Z. Wang and P. Goldsmith, "Modified energy-balancing-based control for the tracking problem," *IET Control Theory Appl.*, vol. 2, pp. 310–322, 2008.
- [62] C. Souza, V. Raffo, and E. Castelan, "Passivity-based control of a quadrotor UAV," *IFAC Proc. Vols.*, vol. 47, pp. 3196–3201, 2014.
- [63] L. Furieri, C. L. Galimberti, M. Zakwan, and G. Ferrari-Trecate, "Distributed neural network control with dependability guarantees: A compositional port-Hamiltonian approach," in *Proc. Learn. Dyn. Control Conf.*, 2022, pp. 571–583.
- [64] C. L. Galimberti, L. Furieri, L. Xu, and G. Ferrari-Trecate, "Hamiltonian deep neural networks guaranteeing nonvanishing gradients by design," *IEEE Trans. Autom. Control*, vol. 68, no. 5, pp. 3155–3162, May 2023.
- [65] E. Sebastián, T. Duong, N. Atanasov, E. Montijano, and C. Sagüés, "LEMURS: Learning distributed multi-robot interactions," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 7713–7719.
- [66] J. Marsden and T. Ratiu, *Introduction to Mechanics and Symmetry: A Basic Exposition of Classical Mechanical Systems*. Berlin, Germany: Springer, 2013.
- [67] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 5745–5753.
- [68] T. Barfoot, *State Estimation for Robotics*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [69] E. G. Hemingway and O. M. O'Reilly, "Perspectives on euler angle singularities, gimbal lock, and the orthogonality of applied forces and applied moments," *Multibody Syst. Dyn.*, vol. 44, pp. 31–56, 2018.
- [70] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE Robot. Automat. Lett.*, vol. 3, no. 2, pp. 620–626, Apr. 2018.
- [71] P. Forni, D. Jeltsema, and G. Lopes, "Port-Hamiltonian formulation of rigid-body attitude control," *IFAC-PapersOnLine*, vol. 48, pp. 164–169, 2015.
- [72] R. Rashad, F. Califano, and S. Stramigioli, "Port-hamiltonian passivity-based control on SE(3) of a fully actuated UAV for aerial physical interaction near-hovering," *IEEE Robot. Automat. Lett.*, vol. 4, no. 4, pp. 4378–4385, Oct. 2019.
- [73] J. Delmerico and D. Scaramuzza, "A benchmark comparison of monocular visual-inertial odometry algorithms for flying robots," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 2502–2509.
- [74] S. A. S. Mohamed, M. Haghbayan, T. Westerlund, J. Heikkonen, H. Tenhunen, and J. Plosila, "A survey on odometry for autonomous navigation systems," *IEEE Access*, vol. 7, pp. 97466–97486, 2019.
- [75] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 8026–8037.
- [76] J. R. Dormand and P. J. Prince, "A family of embedded Runge-Kutta formulae," *J. Comput. Appl. Math.*, vol. 6, pp. 19–26, 1980.
- [77] M. Lienen and S. Günnemann, "torchode: A parallel ODE solver for PyTorch," in *Proc. Symbiosis Deep Learn. Differ. Equ. II*, 2022.
- [78] A. Iserles, H. Z. Munthe-Kaas, S. P. Nørsett, and A. Zanna, "Lie-group methods," *Acta Numerica*, vol. 9, pp. 215–365, 2000.
- [79] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *Proc. IEEE Conf. Decis. Control*, 2010, pp. 5420–5425.
- [80] H. K. Khalil, *Nonlinear Systems*. Upper Saddle River, NJ, USA: Prentice-Hall, 2002.
- [81] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. Schöllig, "Learning to fly: A PyBullet gym environment to learn the control of multiple nano-quadcopters," 2020. [Online]. Available: <https://github.com/utiasDSL/gym-pybullet-drones>
- [82] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multi-threaded open source robotics framework for deeply embedded platforms," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 6235–6240.
- [83] N. Boumal, *An Introduction to Optimization on Smooth Manifolds*. Cambridge, U.K.: Cambridge Univ. Press, 2023. [Online]. Available: <https://www.nicolasboumal.net/book>
- [84] P. Arathoon, "Coadjoint orbits of the special Euclidean group," Master's thesis, Dept. Math., Univ. of Manchester, Manchester, U.K., 2015.



Thai Duong (Graduate Student Member, IEEE) received the B.S. degree in electronics and telecommunications from the Hanoi University of Science and Technology, Hanoi, Vietnam, in 2011, and the M.S. degree in electrical and computer engineering from Oregon State University, Corvallis, OR, USA, in 2013. He is currently working toward the Ph.D. degree in electrical and computer engineering with the University of California San Diego, La Jolla, CA, USA.

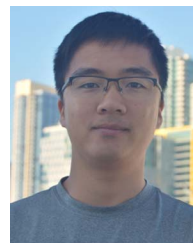
His research interests include machine learning with applications to robotics, mapping, and active exploration using mobile robots, robot dynamics learning, planning, and control.



Abdullah Altawaitan (Student Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Arizona State University, Tempe, AZ, USA, in 2017 and 2019, respectively. He is currently working toward the Ph.D. degree in electrical and computer engineering with the University of California San Diego, La Jolla, CA, USA.

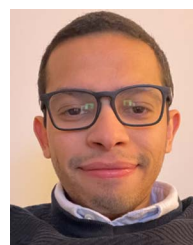
He is affiliated with the Electrical Engineering Department, College of Engineering and Petroleum, Kuwait University, Safat, Kuwait. His research interests include machine learning, control theory, and their applications to robotics.

- [43] L. Ruthotto 和 E. Haber, “受偏微分方程启发的深度神经网络,” *J. Math. Imag. Vis.*, 卷 62, 页 352–364, 2019. [44] R. Wang, R. Walters 和 R. Yu, “将对称性引入深度动力学模型以提升泛化能力,” 收录于 *Proc. Int. Conf. Learn. Representations*, 2021. [45] M. Lutter, K. Listmann 和 J. Peters, “用于欠驱动系统基于能量控制的端到端学习的深度拉格朗日网络,” 收录于 *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, 页 7718–7725. [46] T. Beckers, J. Seidman, P. Perdikaris 和 G. Pappas, “高斯过程端口-哈密顿系统: 具有物理先验的贝叶斯学习,” 收录于 *Proc. IEEE Conf. Decis. Control*, 2022, 页 1447–1453. [47] T. Beckers, “端口-哈密顿系统的数据驱动贝叶斯控制,” 收录于 *Proc. IEEE Conf. Decis. Control*, 2023, 页 8708–8713. [48] B. Leimkuhler 和 S. Reich, *Simulating Hamiltonian Dynamics*. 英国剑桥: 剑桥大学出版社, 2004. [49] P. Toth, D. J. Rezende, A. Jaegle, S. Racanière, A. Botev 和 I. Higgins, “哈密顿生成网络,” 收录于 *Proc. Int. Conf. Learn. Representations*, 2019. [50] J. Mason, C. Allen-Blanchette, N. Zolma
- [69] E. G. 赫明威 和 O. M. 奥赖利, “欧拉角奇异性、万向锁以及施加力与施加力矩的正交性观点”, *Multibody Syst. Dyn.*, 第44卷, 第31–56页, 2018年. [70] M. 法斯勒, A. 弗兰奇, 和 D. 斯卡拉穆扎, “考虑转子阻力的四旋翼动力学微分平坦性, 用于高速度轨迹的精确跟踪”, *IEEE Robot. Automat. Lett.*, 第3卷, 第2期, 第620–626页, 2018年4月. [71] P. 福尔尼, D. 耶尔采马, 和 G. 洛佩斯, “刚体姿态控制的端口-哈密顿形式”, *IFAC-PapersOnLine*, 第48卷, 第164–169页, 2015年. [72] R. 拉沙德, F. 卡利法诺, 和 S. 斯特拉米焦里, “全驱动无人机在近悬停状态下空中物理交互的 SE(3) 端口-哈密顿惯性基础控制”, *IEEE Robot. Automat. Lett.*, 第4卷, 第4期, 第4378–4385页, 2019年10月. [73] J. 德尔梅里科 和 D. 斯卡拉穆扎, “飞行机器人单目视觉惯性测程算法的基准比较”, 见 *Proc. IEEE Int. Conf. Robot. Automat.*, 2018年, 第2502–2509页. [74] S. A. S. 穆罕默德, M. 哈格巴扬, T. 韦斯特伦德, J. 海科宁, H



Thai Duong (IEEE研究生会员) 于2011年获得越南河内河内科技大学的电子与通信学士学位, 并于2013年在美国俄勒冈州科瓦利斯的俄勒冈州立大学获得电气与计算机工程硕士学位。他目前正在美国加利福尼亚州拉霍亚的加利福尼亚大学圣地亚哥分校攻读电气与计算机工程博士学位。

他的研究兴趣包括机器学习用于机器人技术、地图绘制以及使用移动机器人进行主动探索, 机器人动力学学习、规划和控制的应用。



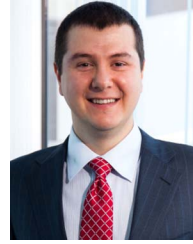
Abdullah Altawaitan (IEEE学生会员) 于2017年和2019年分别获得美国亚利桑那州坦佩市亚利桑那州立大学的电气工程学士和硕士学位。他目前正在美国加利福尼亚州拉霍亚市加利福尼亚大学圣地亚哥分校攻读电气与计算机工程博士学位。

他隶属于科威特大学工程与石油学院电气工程系, 科威特萨法特。他的研究兴趣包括机器学习、控制理论和它们在机器人技术中的应用。



Jason Stanley (Student Member, IEEE) received the B.S. degree in computer engineering in 2024 from the University of California San Diego (UCSD), La Jolla, CA, USA, where he is currently working toward the master's degree with the Department of Electrical and Computer Engineering.

His research interests include machine learning for robotics, control theory, and mobile robots.



Nikolay Atanasov (Senior Member, IEEE) received the B.S. degree in electrical engineering from Trinity College, Hartford, CT, USA, in 2008, and the M.S. and Ph.D. degrees in electrical and systems engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2012 and 2015, respectively.

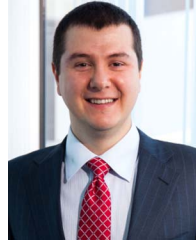
He is an Associate Professor of electrical and computer engineering with the University of California San Diego, La Jolla, CA, USA. He works on probabilistic models that unify geometric and semantic information in simultaneous localization and mapping (SLAM) and on optimal control and reinforcement learning algorithms for minimizing probabilistic model uncertainty. His research interests include robotics, control theory, and machine learning, applied to active perception problems for autonomous mobile robots.

Dr. Atanasov was the recipient of the Joseph and Rosaline Wolf award for the best Ph.D. dissertation in electrical and systems engineering at the University of Pennsylvania in 2015, the Best Conference Paper Award at the IEEE International Conference on Robotics and Automation (ICRA) in 2017, the NSF CAREER Award in 2021, and the IEEE RAS Early Academic Career Award in Robotics and Automation in 2023.



Jason Stanley (IEEE 学生会员) 于 2024 年获得美国加利福尼亚州拉荷亚加州大学圣地亚哥分校 (UCSD) 计算机工程学士学位, 目前正在该校电气与计算机工程系攻读硕士学位。

他的研究兴趣包括机器人领域的机器学习、控制理论和移动机器人。



尼古拉·阿塔纳索夫 (高级会员, IEEE) 于 2008 年在美国康涅狄格州哈特福德的三一学院获得电气工程学士学位, 并于 2012 年和 2015 年分别在美国宾夕法尼亚州费城的宾夕法尼亚大学获得电气与系统工程硕士和博士学位。

他是美国加利福尼亚大学圣地亚哥分校 (La Jolla, CA, USA) 电气与计算机工程副教授。他从事概率模型的研究, 这些模型将几何和语义信息统一应用于同时定位与建图中

研究SLAM和用于最小化概率模型不确定性的最优控制及强化学习算法。他的研究兴趣包括机器人技术、控制理论和机器学习, 应用于自主移动机器人主动感知问题。

阿塔纳索夫博士在 2015 年获得宾夕法尼亚大学电气与系统工程最佳博士论文奖——约瑟夫和罗莎琳·沃尔夫奖; 在 2017 年获得 IEEE 国际机器人与自动化会议 (ICRA) 最佳会议论文奖; 在 2021 年获得国家科学基金会 (NSF) 职业早期发展奖 (CAREER Award); 在 2023 年获得 IEEE 机器人与自动化协会 (RAS) 机器人与自动化早期学术生涯奖。